



SECJ2203: Software Engineering

System Documentation (SD)

Software Requirements Specification (SRS)

Hast Travel Car Rental

Version 3.0

Date: 14/01/2025

Faculty of Computing

Prepared by: Hephaestus

Revision Page

1. Overview

The 3.0 version of System Documentation (SD) contains the content 1 .0 version which are the Chapter 1 Introduction and Chapter 2 Specific Requirements of the Hasta Travel Car Rental System and 2.0 version which covering Chapters 3 to 8. Version 2.0 expands the documentation to include Chapter 3 Architectural Rationale, Chapter 4 Architectural Views, Chapter 5 Data Design, Chapter 6 User Interface Design and Chapter 7 Traceability. Version 3.0 further extends the System Documentation by adding Chapter 8 Test Cases and Chapter 9 Traceability Matrix. Chapter 8 presents detailed test cases to verify system functionality against requirements while Chapter 9 provides a traceability matrix linking requirements to their corresponding test scenarios. This version offers a complete view of the system's lifecycle from requirement specification through design and implementation to testing and validation.

2. Target Audience

This SD is intended for individuals and teams involved in the design, development, and evaluation of the Hasta Car Booking System. The main audience includes the project stakeholders, academic supervisor and system development team members. This documentation serves as a reference for the project team members to ensure consistent understanding of the system requirements and design specifications throughout the development process.

3. Project Team Members.

The project team divided the SRS project documentation parts for each member to handle. Each member is assigned to specific tasks to ensure the documentation process is undergone efficiently and effectively. The table below shows the overview of team members, roles, task handles and the status of the tasks.

Member Name	Role	Task	Status
Muhammad Hafiz Bin Reepei	Project Manager	Module 1: Vehicle Management and Vehicle Booking Module All done	System Testing
Ahmad Irfan Bin Azahan	Frontend Developer	Module 2: Customer Module & Booking Design All done	System Testing
Cheong Yi Shien	System Analyst	Module 3: Staff Module, Admin & Reporting Modules and Booking cancellation All done	System testing
Ling Yu Qian	Payment System Engineer	Module 4: Payment & Billing Module All done	System Testing

4. Version Control History

The Version Control History records all modifications made to this SRS document, including updates to requirements, corrections, and additions. Each entry notes the version number, date, author, and a brief description of the changes. This helps maintain clarity and traceability of revisions over time. The table below summarizes the version history of the document

Version	Primary Author(s)	Sprint/Iteration	Description of Version	Date Completed
1.0	Team leader 1 (Muhammad Hafiz Bin Reepei)	Sprint 1	Completed Version 1.0 including Chapter 1 until Chapter 2, Section 2.6	29/11/2025
2.0	Team leader 1 (Muhammad Hafiz Bin Reepei)	Sprint 2	Completed Version 2.0 including Chapter 1 until Chapter 7	15/12/2025
3.0	Team leader 1 (Muhammad Hafiz Bin Reepei)	Sprint 3	Completed Version 3.0 including Chapter 1 until Chapter 9	15/01/2025

Note:

This System Documentation (SD) template is adapted from IEEE Recommended Practice for Software Requirements Specification (SRS) (IEEE Std. 830-1998), Software Design Descriptions (SDD) (IEEE Std. 1016-1998 1), and Software Test Documentation (IEEE Std. 829-2008) that are simplified and customized to meet the need of SECJ2203 course at School of Computing, UTM. Examples of models are from Arlow and Neustadt (2002) and other sources stated accordingly.

Table of Contents

1	Introduction	7
	1.1 Purpose	7
	1.2 Scope	7
	1.3 Definitions, Acronyms and Abbreviations	8
	1.4 References	8
	1.5 Overview	9
2	Specific Requirements	9
	2.1 Persona	9
	2.1.1 Persona 1 Customer	9
	2.1.2 Persona 2 Staff	10
	2.1.3 Persona 3 Admin	11
	2.2 System Features	12
	2.3 Launch Phase	15
	2.4 User Story Details	15
	2.4.1 US001: User Story <Staff>	
	User Story description of US001	16
	System Sequence Diagram of US001	18
	Activity Diagram of US001	19
	2.4.2 US002: User Story <Customer>	
	User Story description of US002	20
	System Sequence Diagram of US002	21
	Activity Diagram of US002	22
	2.4.3 US002: User Story <Customer>	
	User Story description of US003	23

	System Sequence Diagram of US003	25
	Activity Diagram of US003	26
	2.4.4 US004: User Story <Customer>	
	User Story description of US004	27
	System Sequence Diagram of US004	28
	Activity Diagram of US004	29
	2.4.5 US005: User Story <Customer>	
	User Story description of US005	30
	System Sequence Diagram of US005	31
	Activity Diagram of US005	32
	2.4.6 US006: User Story <Customer>	
	User Story description of US006	33
	System Sequence Diagram of US006	34
	Activity Diagram of US006	35
	2.4.7 US007: User Story <Administrator>	
	User Story description of US007	36
	System Sequence Diagram of US007	37
	Activity Diagram of US007	38
	2.4.8 US008: User Story <Staff>	
	User Story description of US008	39
	System Sequence Diagram of US008	41
	Activity Diagram of US008	42
	2.4.9 US009: User Story <Customer>	
	User Story description of US009	43
	System Sequence Diagram of US009	44
	Activity Diagram of US009	45
	2.4.10 US010: User Story <Staff>	

		User Story description of US010	46
		System Sequence Diagram of US010	48
		Activity Diagram of US010	49
	2.5	Performance and Other Requirements	50
	2.6	Design Constraints	51
3		Architectural Rationale	53
	3.1	Architectural Style and Rationale	53
4		Architectural Views	54
	4.1	Use Case View	58
	4.2	Implementation View	59
	4.3	Logical View	67
	4.4	Process View	68
	4.5	Physical/ Deployment View	85
5		Data Design	86
	5.1	Data Dictionary	88
		5.1.1 Entity: <Admin>	88
		5.1.2 Entity: <Booking>	89
		5.1.3 Entity: <Car>	89
		5.1.4 Entity: <Customer>	89
		5.1.5 Entity: <car_img>	89
		5.1.6 Entity: <Fuel>	90
		5.1.7 Entity: <International>	90
		5.1.8 Entity: <InternationalStaff>	90
		5.1.9 Entity: <International_UTMStaff>	90
		5.1.10 Entity: <Invoice>	90

		5.1.11 Entity: <Local>	90
		5.1.12 Entity: <LocalStudent>	91
		5.1.13 Entity: <Local_UTMStaff>	91
		5.1.14 Entity: <LoyaltyCard>	91
		5.1.15 Entity: <Motorcycle>	91
		5.1.16 Entity: <OwnerCar>	92
		5.1.17 Entity: <Payment>	92
		5.1.18 Entity: <Runner>	93
		5.1.19 Entity: <Staff>	93
		5.1.20 Entity: <StaffIT>	93
		5.1.21 Entity: <User>	93
		5.1.22 Entity: <Vehicle>	94
		5.1.23 Entity: <VehicleConditionForm>	94
		5.1.24 Entity: <VehicleConditionImage>	94
		5.1.25 Entity: <VehicleDocument>	95
		5.1.26 Entity: <VehicleMaintenance>	95
		5.1.27 Entity: <Voucher>	96
		5.1.28 Entity: <WalletAccount>	96
6		User Interface Design	97
	6.1	Screen Images	97
7		Traceability	104
8		Test Cases	105
	8.1	TC001: Test <Customer Management> Subsystem: <Register Account (US003)>	105

	8.1.1	TC001_01: Test <Register Account Scenario (SD001)>	105
	8.2	TC002: Test <Customer Management> Subsystem: <Login Account (US004)>	107
	8.2.1	TC002_01: Test <Login Scenario (SD001)>	107
	8.3	TC003: Test Customer Management Subsystem: View Booking History (US005)	109
	8.3.1	TC003_01: Test View History Scenario (SD001)	109
	8.4	TC004: Test Booking Subsystem: <Book Vehicle(US002)>	111
	8.4.1	TC004_01: Test <Create New Booking Scenario (SD002)>	111
	8.4.2	TC004_02 Test <Return Car (SD003)>	114
	8.5	TC005: Test <Vehicle> Subsystem: <Manage Vehicle (US001)>	117
	8.5.1	TC005_01: Test <Add Vehicle (SD004)>	117
	8.6	TC006: Test <Booking> Subsystem: <Cancel Order (US006)>	120
	8.6.1	TC006_01: Test <Cancel Order Scenario (SD005)>	120
	8.7	TC007 Test <Generate Report Scenario> Subsystem: <Generate Report (US007)>	122
	8.7.1	TC006_01: Test <Generate Report (SD006)>	122
	8.8	TC008: Test <Invoice & Receipt> Subsystem: Make Payment (US009)	129
	8.8.1	TC008_01: Test Payment Scenario (SD007)	129
	8.9	TC009: Test <Billing Module> Subsystem: Generate Invoice (US010)	135
	8.9.1	TC009_01: Test <Generate Invoice Scenario (SD008)>	135
	8.10	TC010: Test <Payment Status & Billing> Subsystem: <Verify Payment (US008)>	137

		8.10.1	TC010_01: Test Approve Payment Scenario (SD009)	137
		8.10.2	TC010_02: Test Reject Payment Scenario (SD009)	139
9	Traceability Matrix			141

Introduction

1.1 Purpose

This SD provides all the details related to the requirements, design, and documentation for developing the Hasta Travel Car Rental system. The purpose of this System Documentation (SD) is to put into place a clear and unified reference that shall guide the analysis, design, implementation, and evaluation of the software during its development life cycle. It includes the functional and non-functional requirements of the system, user characteristics, models of the system, constraints, and general expectations from the operation of the system for uniformity among the various contributors. The SD is targeted at project stakeholders such as the academic supervisor, the developers, designers, and testers of the system, who depend on this document for an understanding of the system objectives to realize the intended goals in the final output of their work. It also serves to communicate among decision-makers and evaluators about the completeness, quality, and appropriateness regarding specified requirements.

1.2 Scope

The software product to be developed is the Hasta Car Booking System, a customized web-based car rental administration platform designed specifically for Hasta Travel to replace its current semi-manual and error-prone workflow. The system is built using Laravel as the primary backend framework and MySQL as the database management system, supported by HTML, CSS, and JavaScript for the front-end interface. It is designed to centralize and automate key operations such as online booking, document submission, payment proof upload, real-time vehicle availability tracking, booking approval, and administrative reporting. The system will not integrate external payment gateways in this phase; instead, it will rely on a manual proof-of-payment upload mechanism to control development costs and scope. The application will benefit the organization by reducing redundancy, eliminating miscommunication, improving staff productivity, and enhancing customer satisfaction through faster processing, clearer workflows, and greater transparency. Additionally, the system provides supervisors with consolidated data and analytics to support data-driven decision-making. This scope is consistent with the NABC proposal, where Laravel and MySQL were identified as cost-effective, open-source technologies suitable for delivering an efficient, scalable, and reliable digital solution tailored to Hasta Travel's operational needs.

1.3 Definitions, Acronyms and Abbreviation

Term	Definition
Constraint	Limitation or restriction on system behavior and resources.
CRUD	Create, Read, Update, and Delete, the basic operation for managing data in the system.
SD	System Documentation
SRS	Software Requirement Specification
Invoice/Receipt	Digital document generated by system as a proof of payment for a booking
Sprint	A fixed length of period in Agile development to complete the specific work within period and ready to review the completed work.
Dashboard	A centralized user interface that provides an overview of relevant information.

1.4 References

1. Nuñez, R. (2014, October 13). UML - and the Unified Process. Academia.edu.
https://www.academia.edu/8751505/UML_and_the_Unified_Process
2. Software Requirements | Software Engineering Tutorial | Minigranth. (2022). Minigranth.in.
<https://minigranth.in/software-engineering-tutorial/software-requirements-sdlc>
3. Sommerville, I. (2016). Software Engineering Tenth Edition.
<https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf>
4. UML 2 Sequence Diagrams: An Agile Introduction – The Agile Modeling (AM) Method. (n.d.).
<https://agilemodeling.com/artifacts/sequencediagram.htm>

1.5 Overview

This system documentation is organized into two chapters. The first chapter is an introduction that contains purpose, scope, and definitions for the projects. The second chapter includes persona, system features, user stories with diagrams, performance, requirements, and constraints of the system.

2. Specific Requirements

This section defines all the software requirements of the Hasta Travel Car Rental system in detail to allow system designers to design the system and testers to verify that it meets the specified requirements. These requirements include all actions that the system must perform in response to user inputs or requests from other systems, as well as the outputs the system must produce. By clearly defining what the system should do in observable ways, this section ensures that the system's functionality can be understood, implemented, and tested effectively, providing confidence that the final product meets user needs and expectations.

2.1 Persona

This section describes the intended users of the Hasta Travel Car Rental system and their key characteristics. Understanding these users helps the development team design and develop a system that meets their needs, is easy to use, and is accessible to the users.

2.1.1 Persona 1: Customer

User Need

Name: Umar bin Abu

Age: 20

Knowledge: Tech-savvy and comfortable with online booking services. He is frustrated by the current semi-manual process of renting cars, which requires him to use online forms that redirect to WhatsApp for confirmation. He finds this fragmented communication inefficient and slow.

User Stories

- 1) As a customer, I want to log in and register an account so that I can access the booking system.
- 2) As a customer, I want to book a selected car with my preferred date and duration so that I can plan my travel efficiently.
- 3) As a customer, I want to cancel my booking if my plans change so that I do not occupy a vehicle I would not use.
- 4) As a customer, I want to make payments for my booking through the system so that my reservation is confirmed.
- 5) As a customer, I want to view or download my receipt or invoice so that I have proof of payment.

2.1.2 Persona 2: Staff

Name: Fatin binti Adam

Age: 30

Knowledge: He is an administrative staff member who currently manages bookings using the semi-manual process. He is frustrated by the constant switching between the web form, WhatsApp, and bank statements to confirm a single booking.

User Need

An admin staff member needs a centralized dashboard to efficiently manage all booking requests, track vehicle availability in real-time, and automate the payment verification and invoicing process. This is to reduce administrative errors, speed up customer confirmations, and maintain an accurate, consolidated record of all transactions.

User Stories

- 1) As a staff member, I want to log into the system so that I can perform booking-related functions.
- 2) As a staff member, I want to check car availability so that I can ensure the on-time availability is always match to the booking lists.
- 3) As a staff member, I want to verify incoming payments so that bookings are confirmed only when payment is valid
- 4) As a staff member, I want the system to generate invoices and receipts after payment verification so that customers receive official records and the accounting is accurate.
- 5) As a staff member, I want to add, delete, read, and update vehicle records new vehicle records so that the car owner can register for their car for rent.

2.1.3 Persona 3: Administrator

Name: Puan Haslinda binti Othman

Age: 48

Knowledge: As the manager of Hasta Travel, she is focused on strategic growth and profitability. She lacks access to consolidated data, which prevents her from making data-driven decisions to improve revenue. She needs to monitor overall performance, not just individual bookings.

User Need

A administrator needs a high-level, real-time view of the business's performance through a centralized system. This is to analyze sales trends, monitor staff efficiency, and access customer feedback to make strategic decisions that improve sales and customer satisfaction.

User Stories

- 1) As an administrator, I want to log into the system so that I can access system-wide management functions.
- 2) As an administrator I want to verify incoming payments so that bookings are confirmed only when payment is valid
- 3) As an administrator, I want to add, update, and delete vehicle records, so that I can maintain an accurate list of cars available for booking.
- 4) As an administrator, I want to generate rental and booking reports, so that I can review system performance, rental trends, and peak session.

1.1 System Features

The Hasta System Travel Car Rental is a software application designed to operate on both web and mobile platforms. The system provides a convenient way for users to book, manage, and cancel car reservations, while also enabling administrators to monitor bookings and maintain the system efficiently. The system is integrated with a secure, cloud-based database to ensure real-time updates for bookings, cancellations, and user account management. Payment processing is also handled through a reliable external gateway for secure transactions.

The system is designed to serve three main types of users: regular users and system administrators. Figure 1.1 shows the use case diagram for the Hasta Travel Car Rental System which shows the interaction between the user and the system. The detailed use case descriptions in each module are explained in Table 1.0. The domain model diagram in Figure 1.2 is the class diagram showing the classes in the system and associations between the classes while Figure 1.3 shows the state diagram of Hasta Travel car Rental System which shows the different states of the system and how it transitions between the states based on conditions.

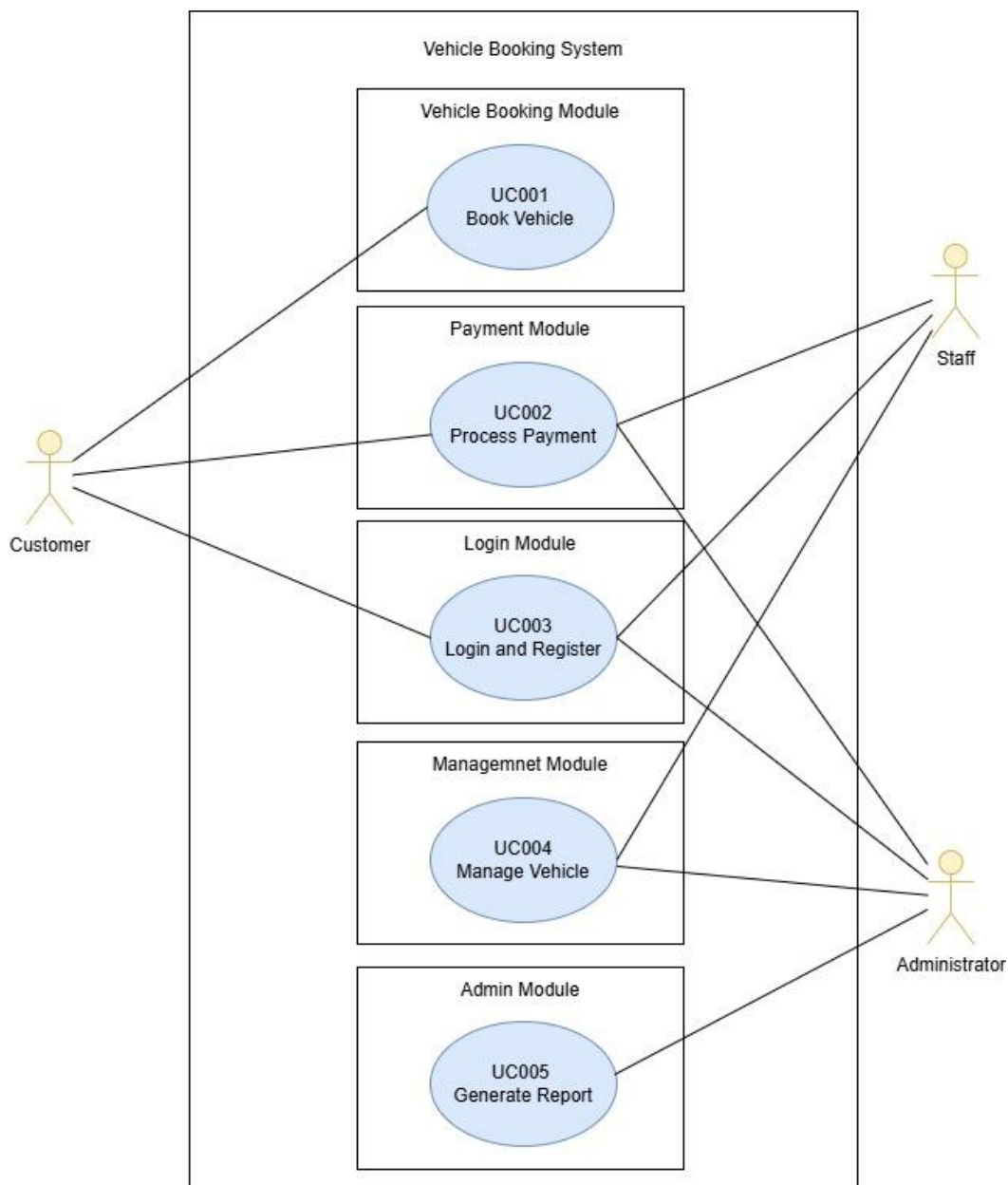


Figure 1.1: Use Case Diagram for <Hasta Travel Car Rental System>

Table 1.0: Description of Module and Functions for <Hasta Travel Car Rental>

Use case	Function	Description
UC001	Book Vehicle	To select available car, fill details and do payment to book. To see a list of all past and current car rental bookings.
UC002	Process Payment	To confirm payment transaction was successful legitimate. To pay and confirm car rental booking.
UC003	Login and Register	Allows user to create new account and book the car in the system
UC004	Manage Vehicle	To perform CRUD on the car inventory of the system.
UC005	Generate Report	To generate report based on specific criteria.

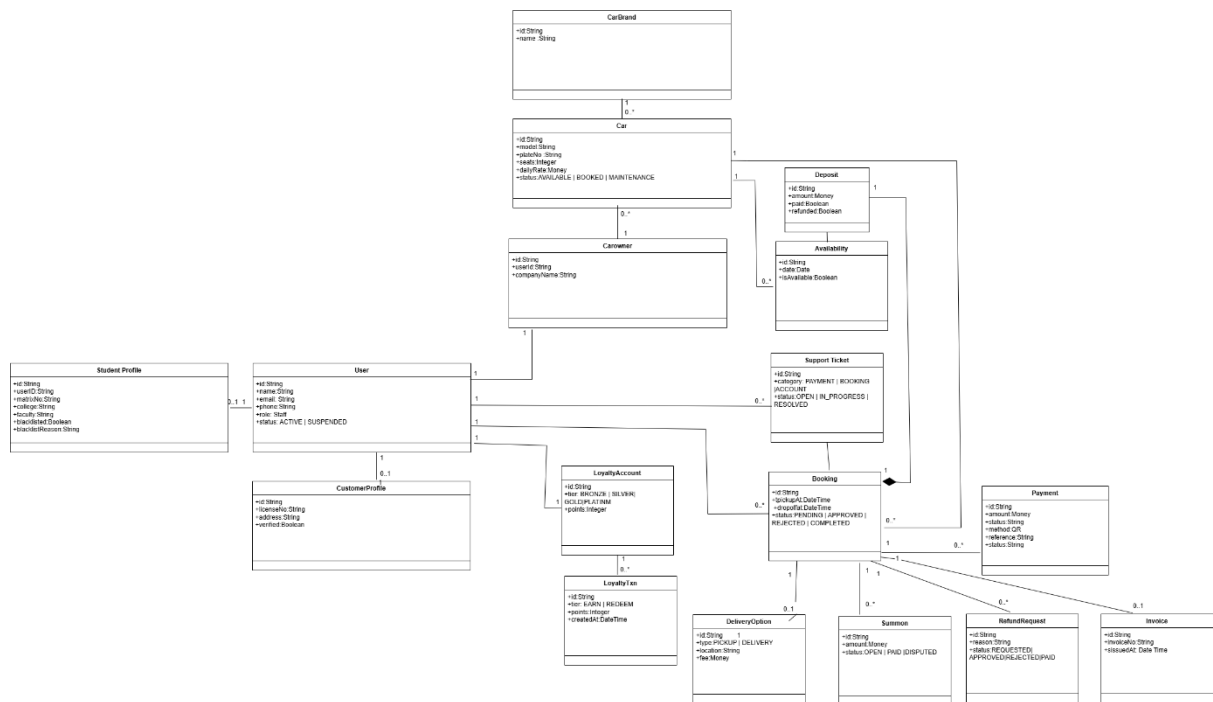


Figure 1.2: Domain Model for <Hasta Travel Car Rental>

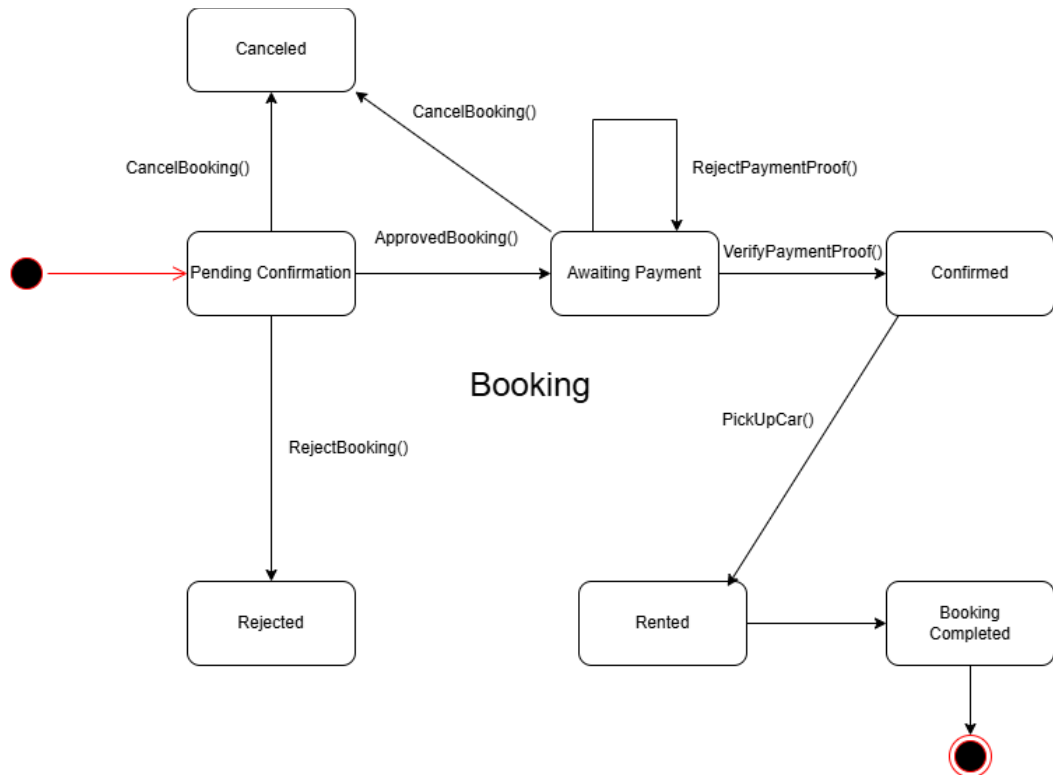


Figure 1.3: State Machine Diagram for Hasta Travel Car Rental

1.2 Launch Phase

The launch phase highlights the Agile development plan for Hasta Travel Car Rental system based on sprints involved during system development. This part includes the product backlogs, sprint planning and team members in charge to ensure development works are organized and completed iteratively to deliver functional increments of the system.

The product backlog lists all features, enhancements and system requirements needed for Hasta Car Booking System. Items in the backlog are derived from personas, user stories, and system features identified in earlier sections. Table below shows the sprint plan with user stories and team members assigned.

Sprint	Team members assigned
Sprint #1: Management Module UC004: Manage Vehicle	Muhammad Hafiz Bin Reepei
Sprint #2: Vehicle Booking Module UC001: Book Vehicle	Muhammad Hafiz Bin Reepei
Sprint #3: Customer Module UC003: Register and Login	Ahmad Irfan Bin Azahan
Sprint #4: Admin Module UC005: Generate Report	Cheong Yi Shien
Sprint #5: Payment Module UC002: Process Payment	Ling Yu Qian

1.3 User Story Details

This section provides details of user stories for the Hasta Travel Car Rental system. Each of the user stories include flow of event, alternative flow, acceptance criteria, exception flow, sequence diagram and activity diagram are documented to ensure clear understanding and accurate implementation.

2.4.1 US001: User Story < Staff > Sprint 1 – Vehicle Management Module

Table 2.1 depicts User Story Description for <Manage Vehicle> use case. Figure 2.1.1 describes step-by-step interaction of staff with systems to manage cars based on Table 2.1. The staff are required to be logged into system first and can proceed to choose between adding, delete, read, or update the vehicle records to manage the car. Figure 2.1.2 illustrates the workflow and decisions for staff to interact accordingly to add, delete, read, or update vehicles records.

Table 2.1: User Story Description for <Manage Vehicle>

User story: Manage Vehicle
ID: US001
User Story Description As a staff member, I want to add, delete, read, and update vehicle records So that the car owner can register for their car for rent.
Flow of events: 1. Staff logs into the system 2. Staff navigates to Car Management section 3. Staff can select one of following options: • Add Car • Edit Car • Delete Car 4.If Add Car: • Staff enter car details • Staff clicks Save to store new car record. 5.If Edit Car: • Staff selects existing car. • Staff updates required fields. • Staff clicks Update. 6.If Delete Car: • Staff selects existing car. • Staff clicks Delete and confirms deletion. 7.Display updated car list
Alternative flow: Staff click Cancel, system will navigate to car list without changes.
Acceptance Criteria Post-condition: • Car list updated Pre-condition: • Staff is logged into the system.

Exception flow:

1. If the picture of the car is not loaded, the system will prompt "Please upload a PNG/JPEG file".
2. If the staff cannot save the car details, the system prompts "Unable to save car details. Please try again."

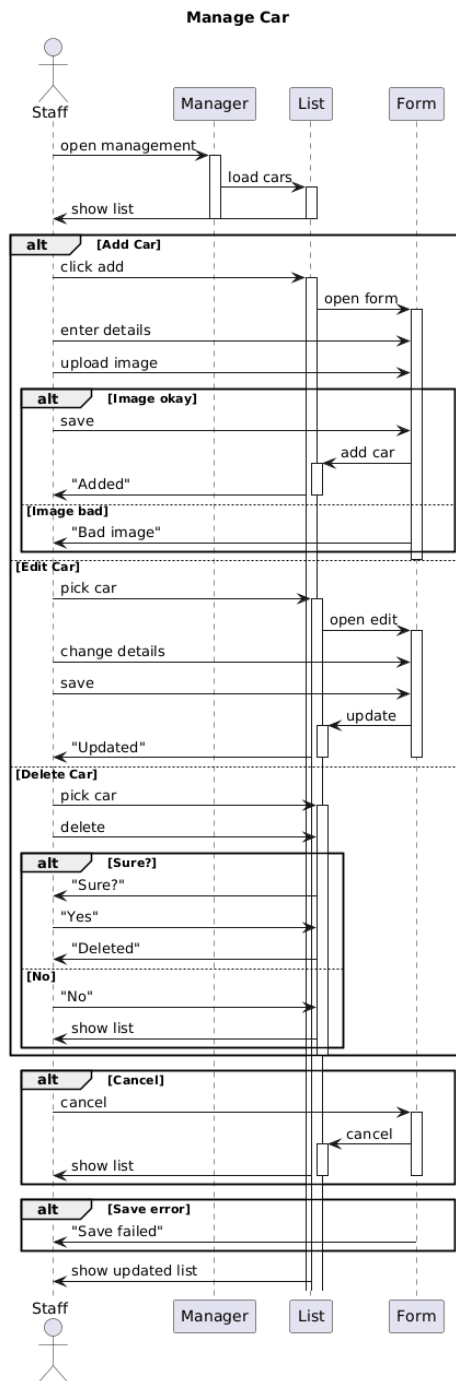


Figure 2.1.1: Sequence Diagram for <Manage Vehicle>

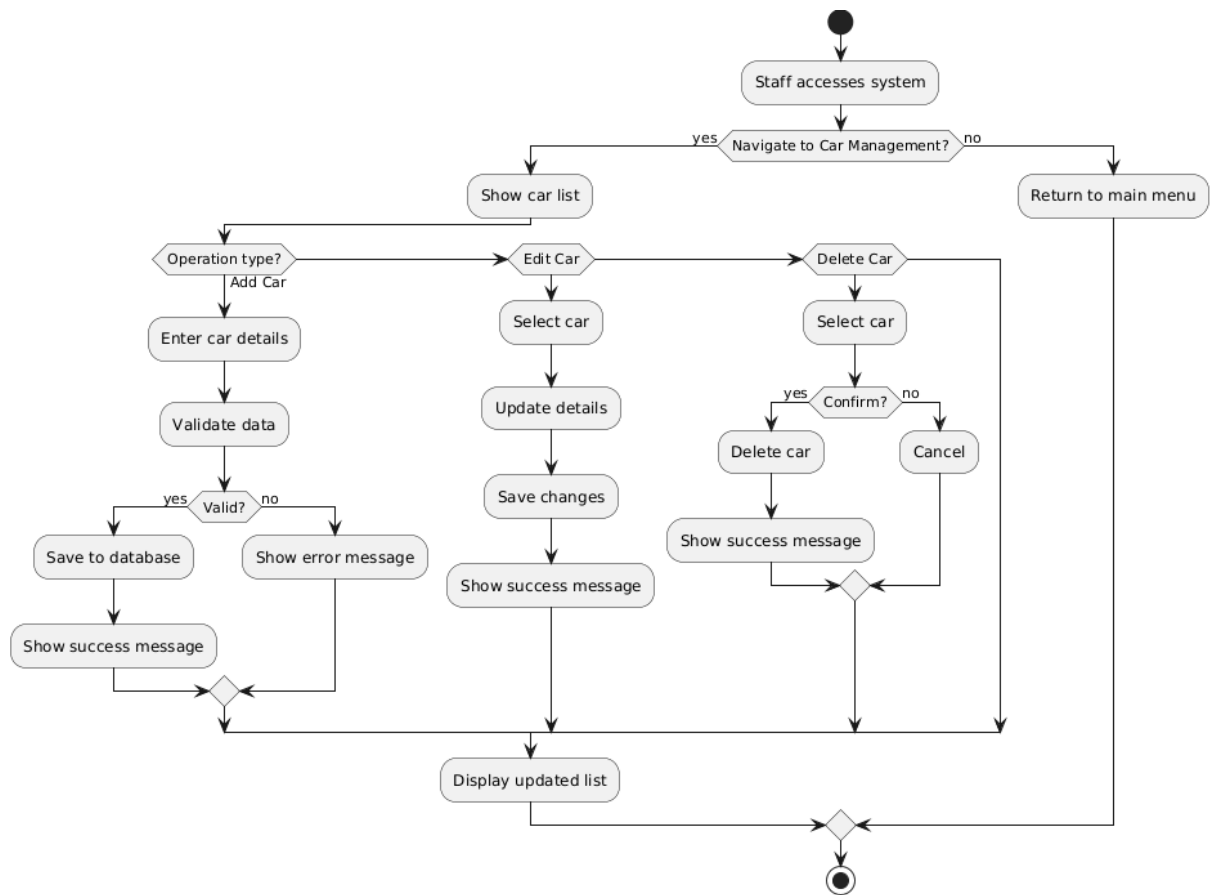


Figure 2.1.2: Activity Diagram for <Manage Vehicle>

2.4.2 US002: User Story < Customer > Sprint 2 – Vehicle Booking Module

Table 2.2 shows User Story Description for <Book Vehicle> use case of flow events for customers to interact in the system to book a car. Figure 2.2.1 and Figure 2.2.2 provides clarity process on how customers can interact with the system to book a car. The customer required to fill out the form first, before proceeding to payment. A message will be displayed to prevent proceeding payment if one of the required details is incomplete.

Table 2.2: User Story Description for <Book Vehicle>

User story: Book Vehicle
ID: US002
Customer Story Description As a customer, I want to book a selected car with my preferred date and duration So that I can plan my travel efficiently.
Flow of events: 1. Customer logs into the system. 2. Customer selects a car and views car details. 3. Customer selects preferred date and duration and clicks Book Now. 3. Customer fills out the booking form and clicks Submit Booking. 4. System displays selected car information, pickup date, rental duration, deposit, and rental price. 5. System displays instructions to proceed with payment.
Alternative flow: If the Customer changes their mind or picks the wrong date, they can click Cancel and the system will return to the car details page.
Acceptance Criteria Post-condition: • Booking records will be stored in the system; Customer will proceed to payment. Pre-condition: • The car must be available within the selected date range.
Exception flow: System prompts Customer to complete the details if the Customer missed fill the required fields.

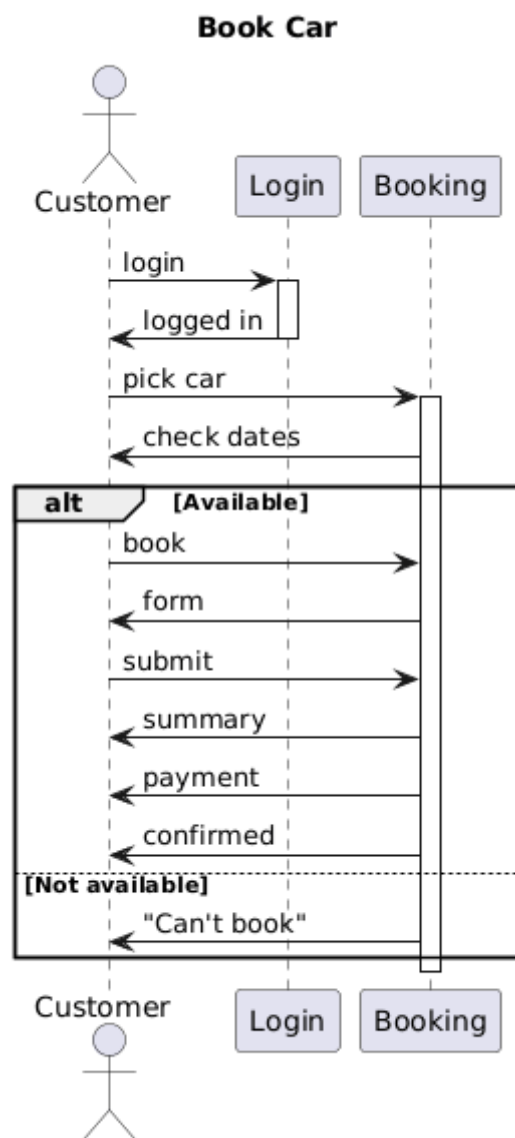


Figure 2.2.1: Sequence Diagram for <Book Vehicle>

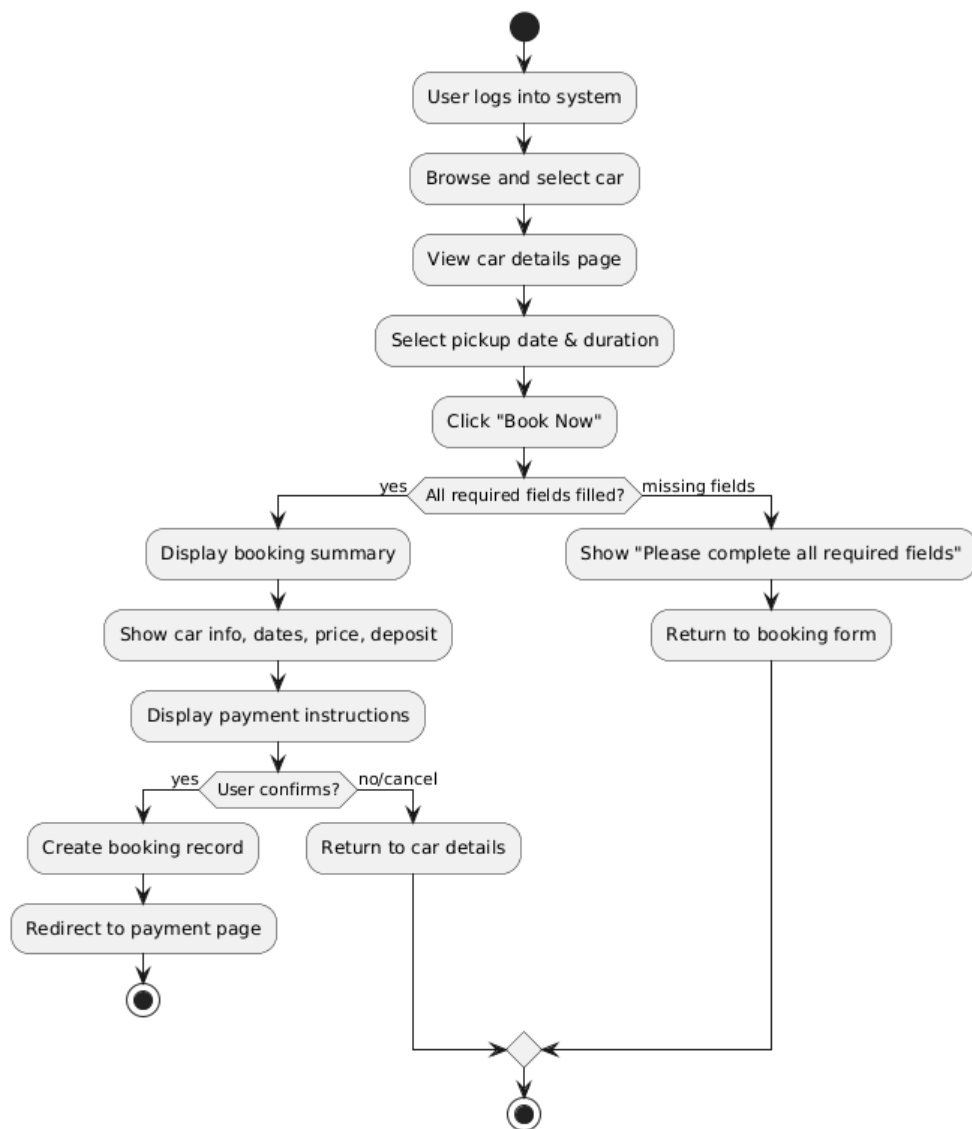


Figure 2.2.2: Activity Diagram for <Book Vehicle>

2.4.3 US003: User Story <Customer> Sprint 3 – Customer Module

Table 2.3 depicts User Story Description for <Register Account> use case. Figure 2.3.1 describes step-by-step interaction of the customer with the system to register an account based on Table 2.3. It shows the complete workflow for creating a new user account, beginning with the user entering their personal details into the registration interface. Figure 2.3.2 shows the sequential steps from data entry, validation, and email uniqueness checking to account creation and confirmation, including alternative paths for invalid input and duplicate emails, ultimately concluding with a successful registration.

Table 2.3: User Story Description for < Register Account >

User story: <Register Account>
ID: US003
User Story Description As a customer, I want to register an account So that I can access the booking system.
Flow of events: 1. Customer clicks Register button. 2. System displays the registration form. 3. Customer enters name, email, password, and confirm password. 4. Customer submits the form. 5. System validates all inputs: Required fields not empty - Email format valid - Password meets security requirements - Password and Confirm Password match 6. System checks whether the email already exists. 7. If valid, system creates a new account in the database. 8. System displays a successful registration message. 9. Customer is redirected to the login page.

Alternative flow:**Alternative flow - Empty fields**

1. System displays "Please fill in all required fields."
2. Returns user to the registration form.

Alternative flow - Invalid Email Format

1. System displays "Invalid email format."

Alternative flow - Passwords Do Not Match

1. System displays "Passwords do not match."

Alternative flow - Weak Password

1. System displays "Password must have at least 8 characters."

Alternative flow - Email Already Registered

1. System displays "This email is already registered."

Acceptance Criteria

Postcondition:

- A new account is created and stored in the system.

Precondition: none

other conditions: none

Exception flow:

The system detects one or more empty fields. System displays: "Please fill in all required fields."

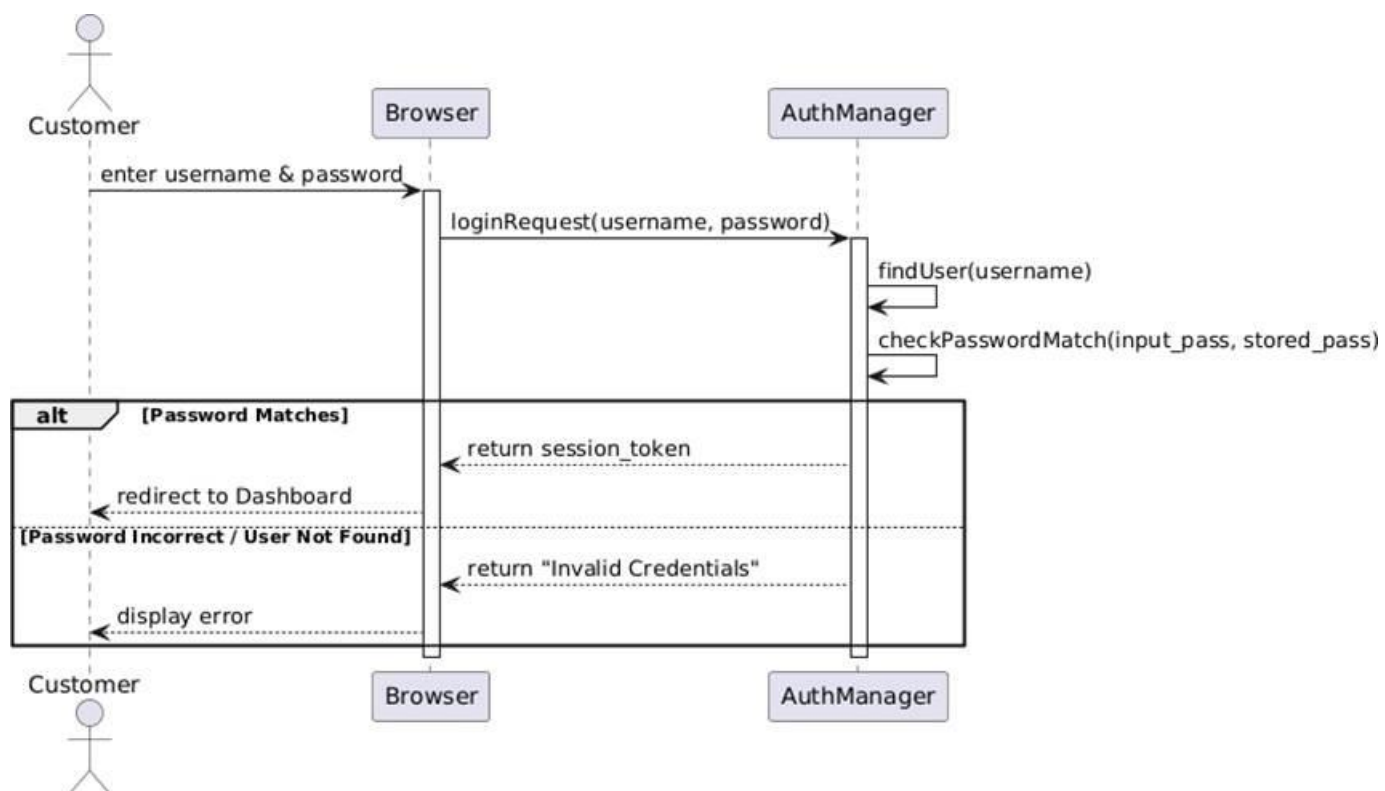


Figure 2.3.1: Sequence Diagram for <Register Account>

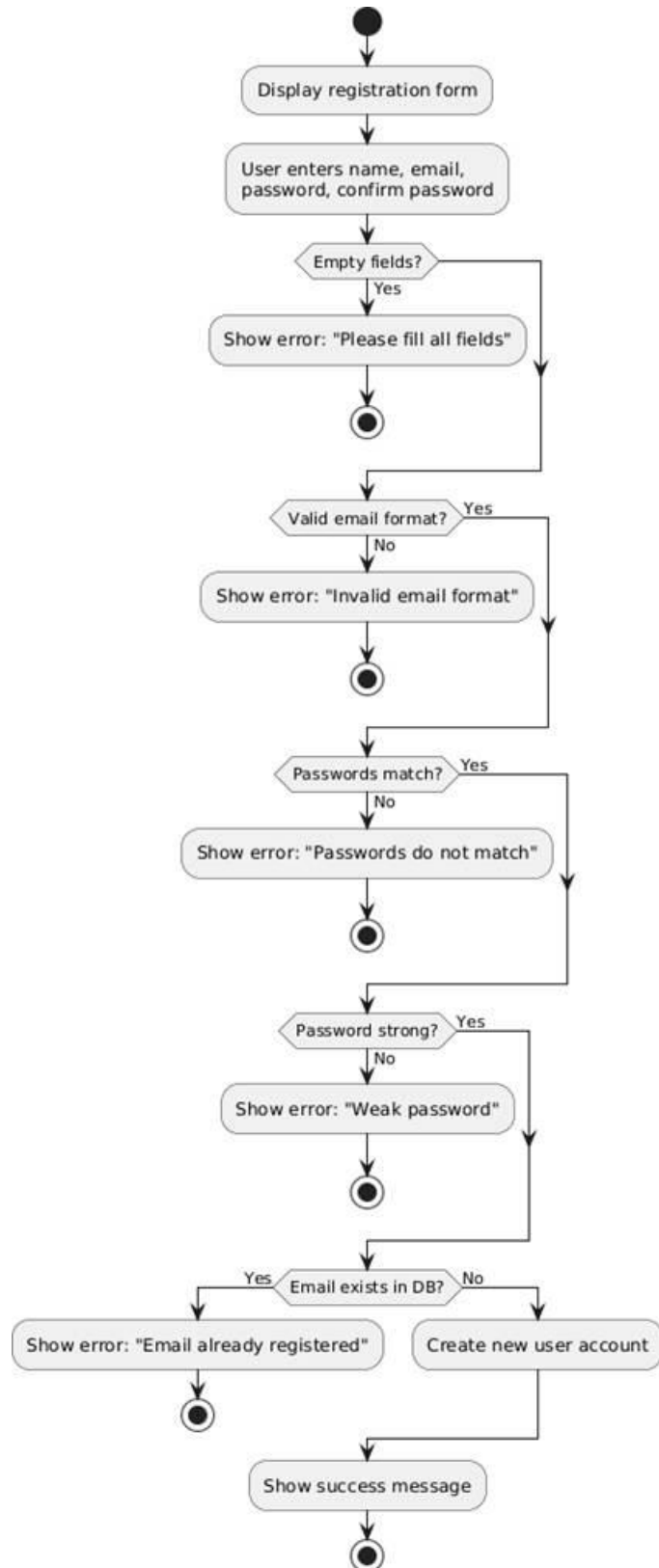


Figure 2.3.2: Activity Diagram for <Register Account>

2.4.4 US004: User Story <Customer> Sprint 2 – Login Module

<Login Account> is a task that can be performed by a customer in <Login and Register Account>. Table 2.4 describes how customers can enter credentials, validate inputs, and access their dashboard. The sequence diagram in Figure 2.4.1 illustrates the interactions between the customer, login interface, authentication controller, user service, password verifier, and logger. Figure 2.4.2 shows the workflow, decision points for input validation, account verification, password check, and exception handling for incorrect credentials.

Table 2.4: User Story Description for <Register and Login Account>

User story: Login Account
ID: US004
User Story Description As a customer, I want to log in to my account securely So that I can manage my bookings.
Flow of events: 1. Customer opens the login page. 2. System displays the login form. 3. Customer enters email and password. 4. Customer submits the login form. 5. System validates that fields are not empty. 6. System validates email format. 7. System checks if the email exists in the database. 8. System verifies the password. 9. System logs the user in. 10. System redirects the user to the dashboard.
Alternative flow: Alternative flow - Empty field 1. System displays: "Please fill in all fields." Alternative flow - Invalid Email Format 1. System displays: "Invalid email format." Alternative flow - Email Not Found 1. System displays: "Account does not exist." Alternative flow - Wrong Password 1. System displays: "Incorrect password."

Acceptance Criteria

Postcondition:

- Customer is authenticated and redirected to the dashboard/homepage.

Precondition:

- Customer must already have a registered account.

other conditions: none

Exception flow:

If the system detects empty email or password. The system displays:
"Please fill in all fields."

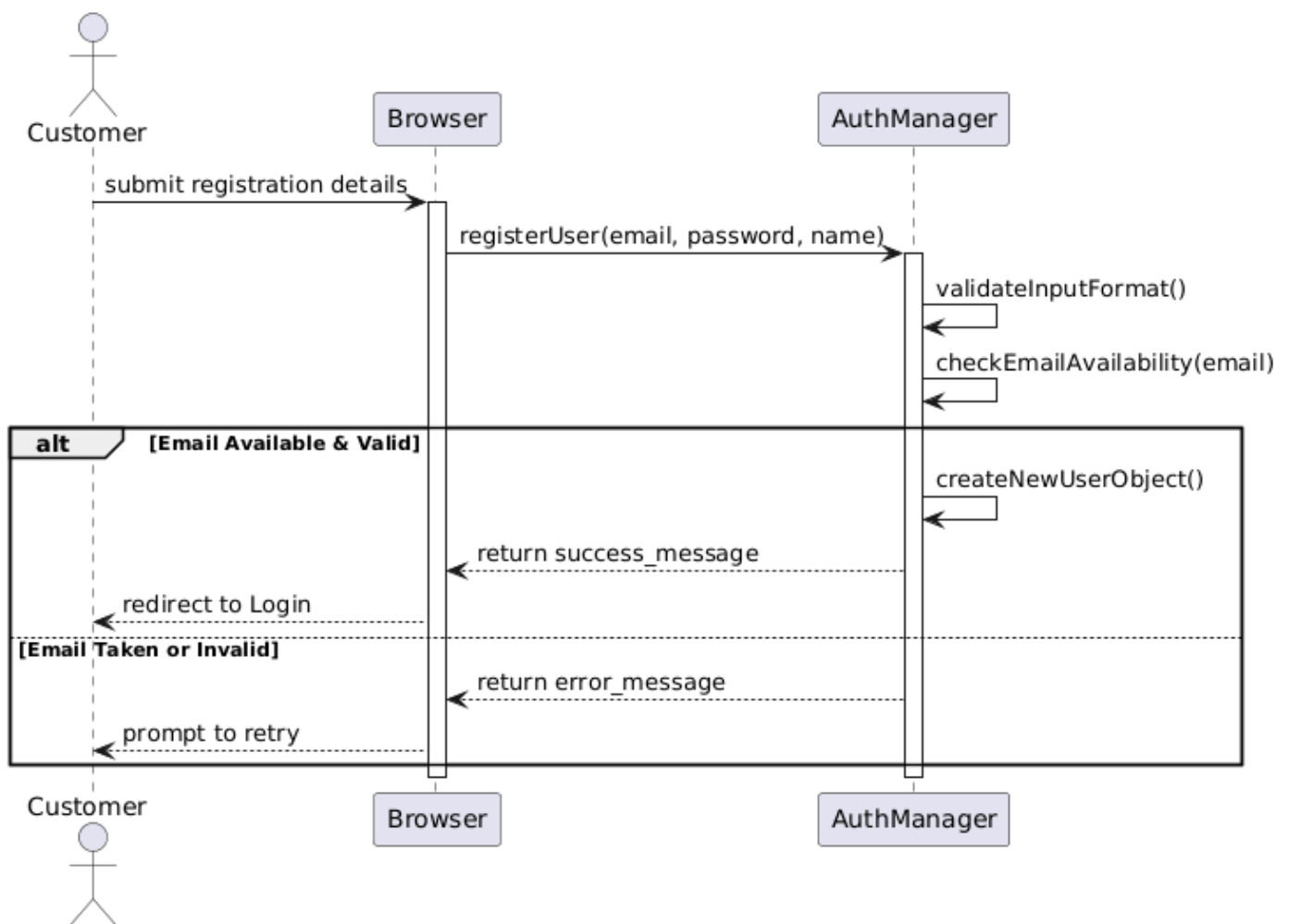


Figure 2.4.1: Sequence Diagram for < Login Account >

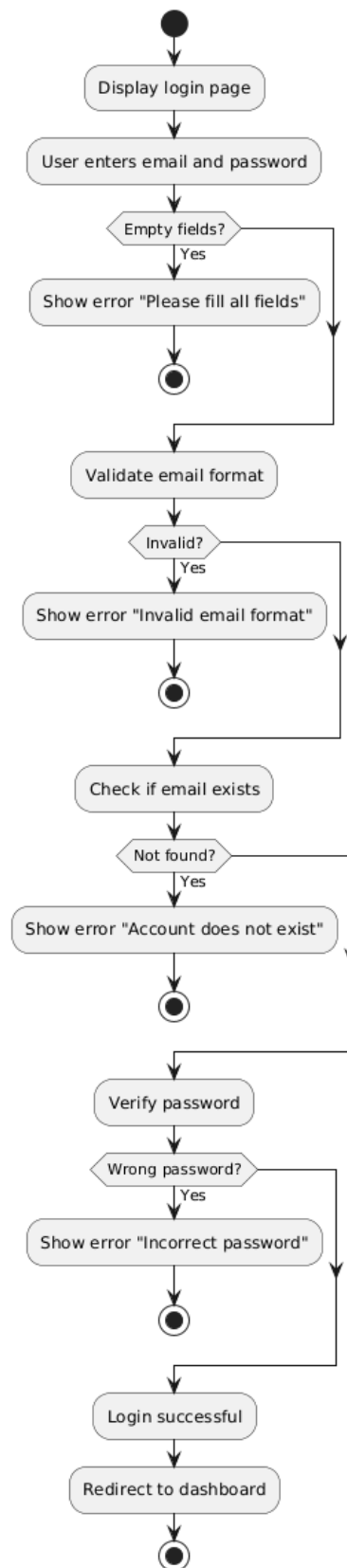


Figure 2.4.2: Activity Diagram for <Login Account>

2.4.5 US005: User Story <Customer> Sprint 2 – Login Module

<Book Vehicle> is a task that can be performed by a customer. Table 2.5 describes how customers can request their past bookings and view details. The sequence diagram in Figure 2.5.1 illustrates the interactions between the customer, dashboard interface, booking controller, booking service, booking repository, and logger. Figure 2.5.2 shows the workflow, including decision points for session validation, retrieving booking records, and handling exceptions when no bookings are found.

Table 2.5: User Story Description for <Login Module>

User story: View Booking History
ID: US005
User Story Description As a customer, I want to view my booking history So that I can review my past and upcoming rentals.
Flow of events: 1. Customer clicks View Booking History. 2. System receives the request. 3. System verifies that the user is authenticated. 4. System retrieves booking records from the database. 5. System displays the booking history page to the user.
Alternative flow: Alternative flow - User does not log in 1. System redirects to login page 2. System display "Please login to continue" Alternative flow - No Booking Records Found 1. System displays: "You have no bookings yet."
Acceptance Criteria Postcondition: - System displays a list of all bookings belonging to the user. Precondition: - Customer must be logged in. - Booking records must exist in the system other conditions: none
Exception flow: If no booking records found, the system displays "You have no bookings yet."

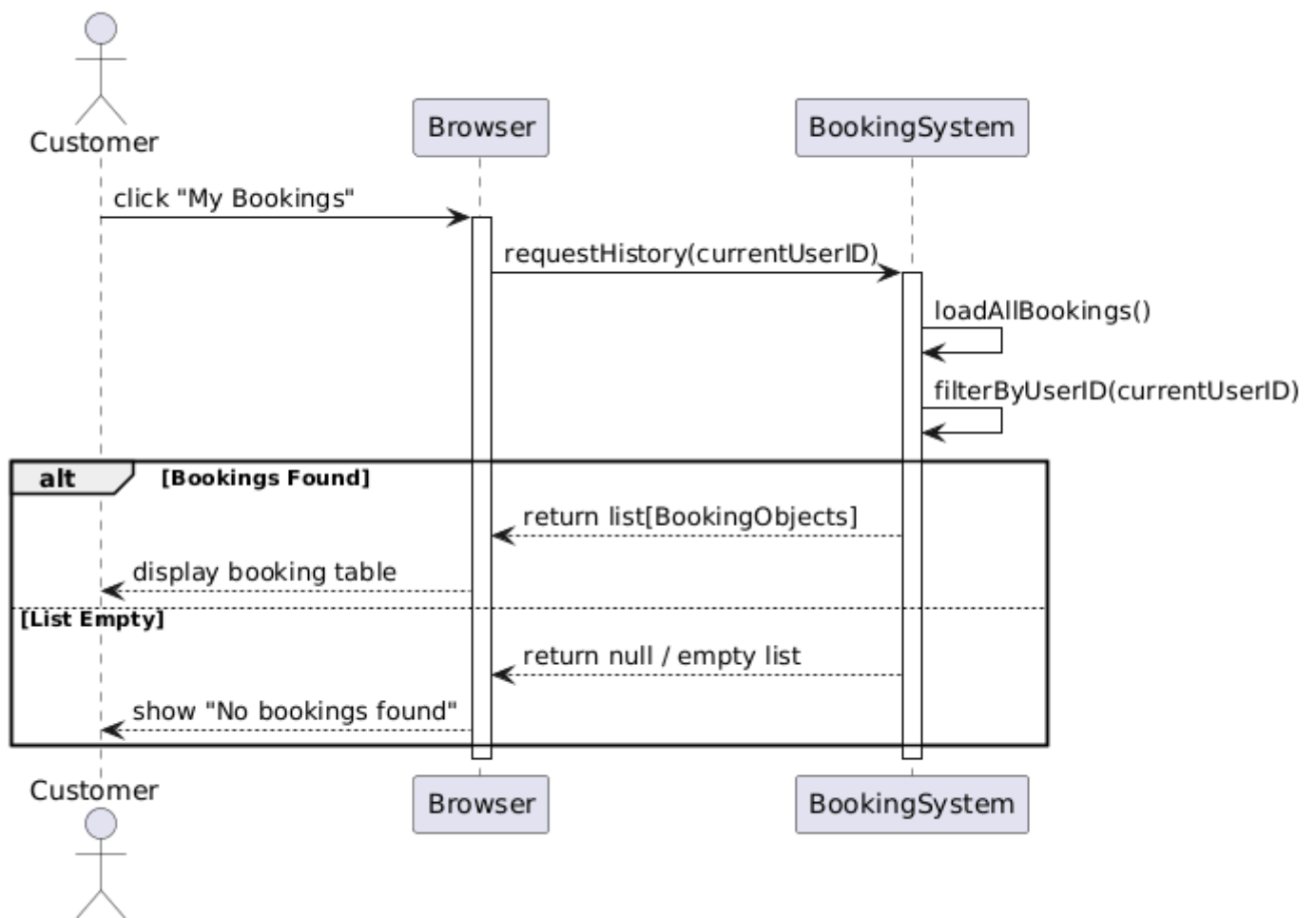


Figure 2.5.1: Sequence Diagram for <View Booking History>

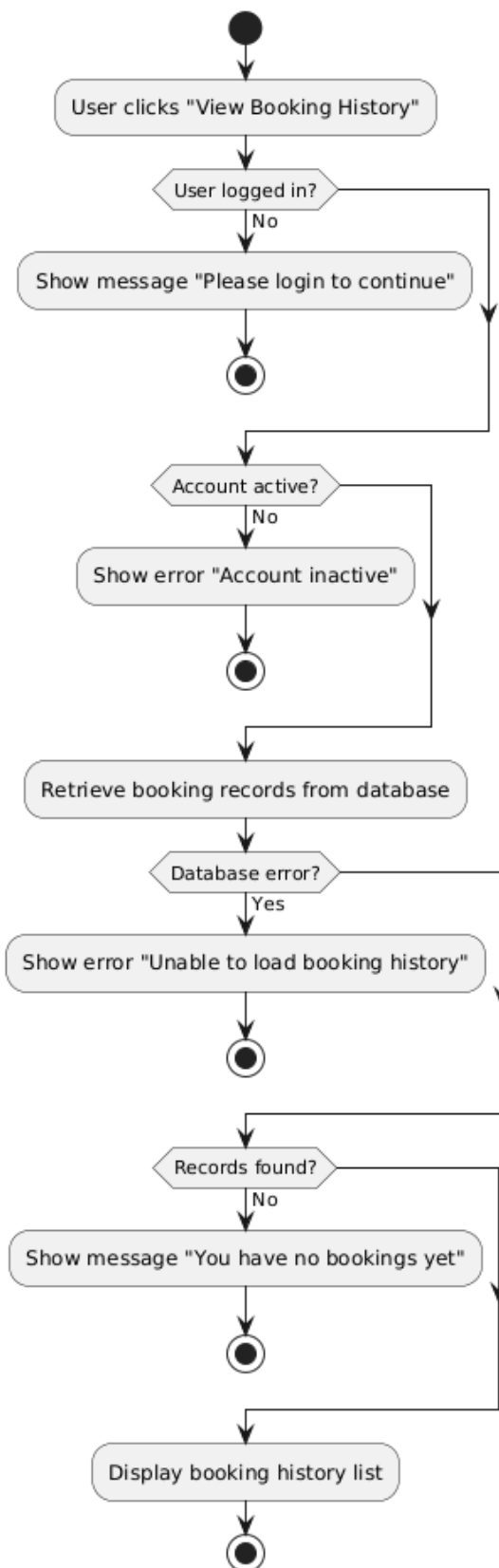


Figure 2.5.2: Activity Diagram for <View Booking History>

2.4.6 US006: User Story <Customer> Sprint 3 – Vehicle Booking Module

The <Book Vehicle> is in customer module that is designed for the customer to cancel their bookings if they change their mind. Table 2.6 describes how a customer can withdraw an upcoming car booking. Figure 2.6.1 is the sequence diagram that the interaction between the customer and the system when a cancellation request is initiated. It illustrates how the system processes the request, checks cancellation rules, and responds by either approving the cancellation or displaying a rejection message while Figure 2.6.2 highlights the decision points in determining whether a cancellation is allowed.

Table 2.6: User Story Description for <Book Vehicle>

User story: Cancel Booking
ID: US006
User Story Description As a customer, I want to cancel my booking if my plans change So that I do not occupy a vehicle I would not use and let staff process my cancellation.
Flow of events: 1. Customer logs into the system. 2. Customer opens the Booking History page. 3. System shows all bookings. 4. Customer selects a booking to cancel. 5. Customer clicks "Request Cancellation". 6. System asks for confirmation. 7. Customer confirms cancellation. 8. System updates booking status to "Pending Cancellation Approval". 9. System notifies staff on the cancellation request. 10. Customer waits for the staff to approve cancellation and credit refund.
Alternative flow: Alternative flow – Too late cancellation is not allowed: 1. System attempts cancellation near pickup time. 2. System denies cancellation request and displays policy message.
Acceptance Criteria Postcondition: • Booking cancellation is wait for staff approval. Precondition • Customer must be logged in to an account to cancel booking. Other conditions • Cancellation policy rules apply, no cancellation 12 hours before pickup.
Exception flow: • If system error during cancellation, the system displays an error message • If network or database failure during cancellation, the system displays an error message for failure submission of cancellation request.

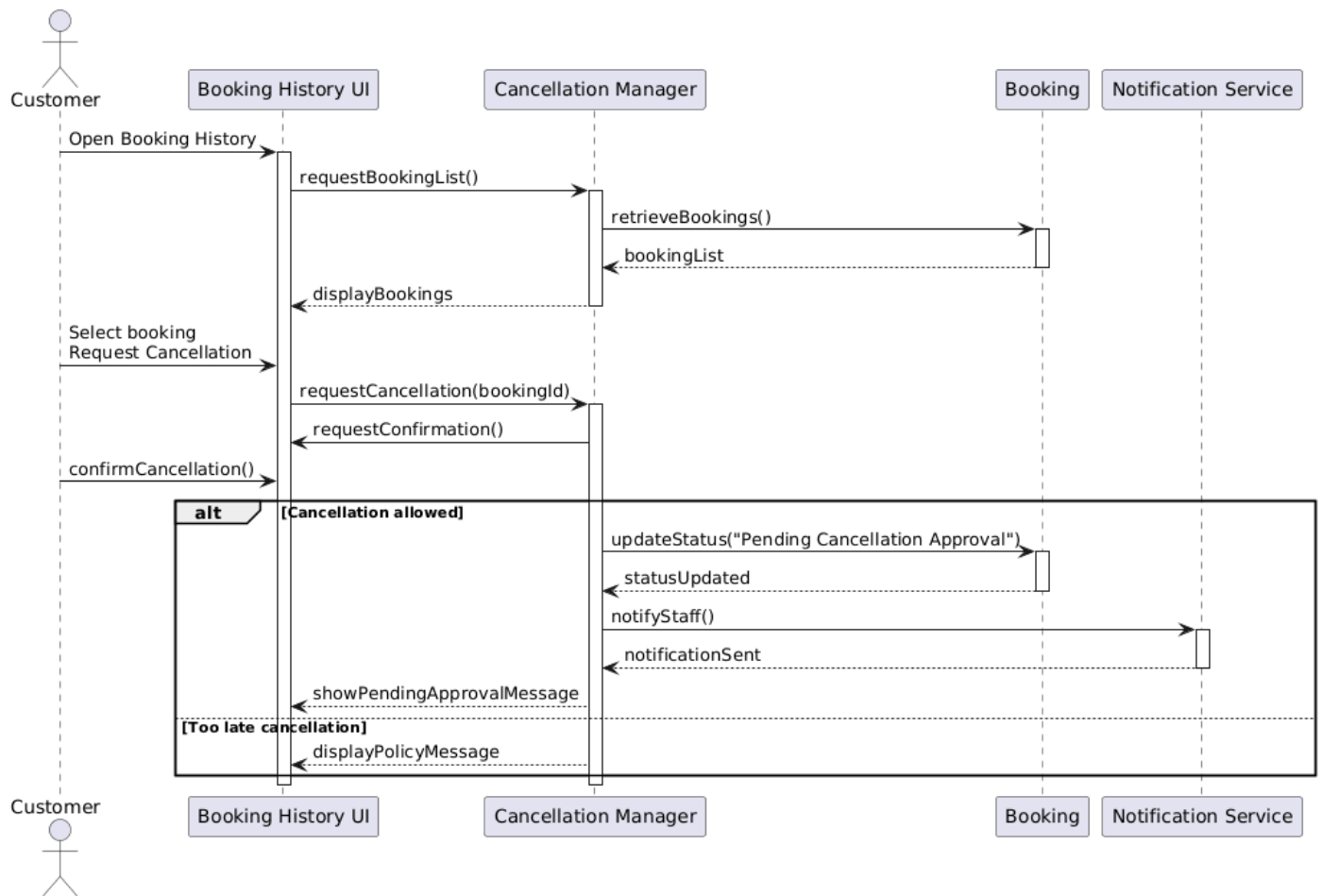


Figure 2.6.1: Sequence Diagram for <Cancel Booking>

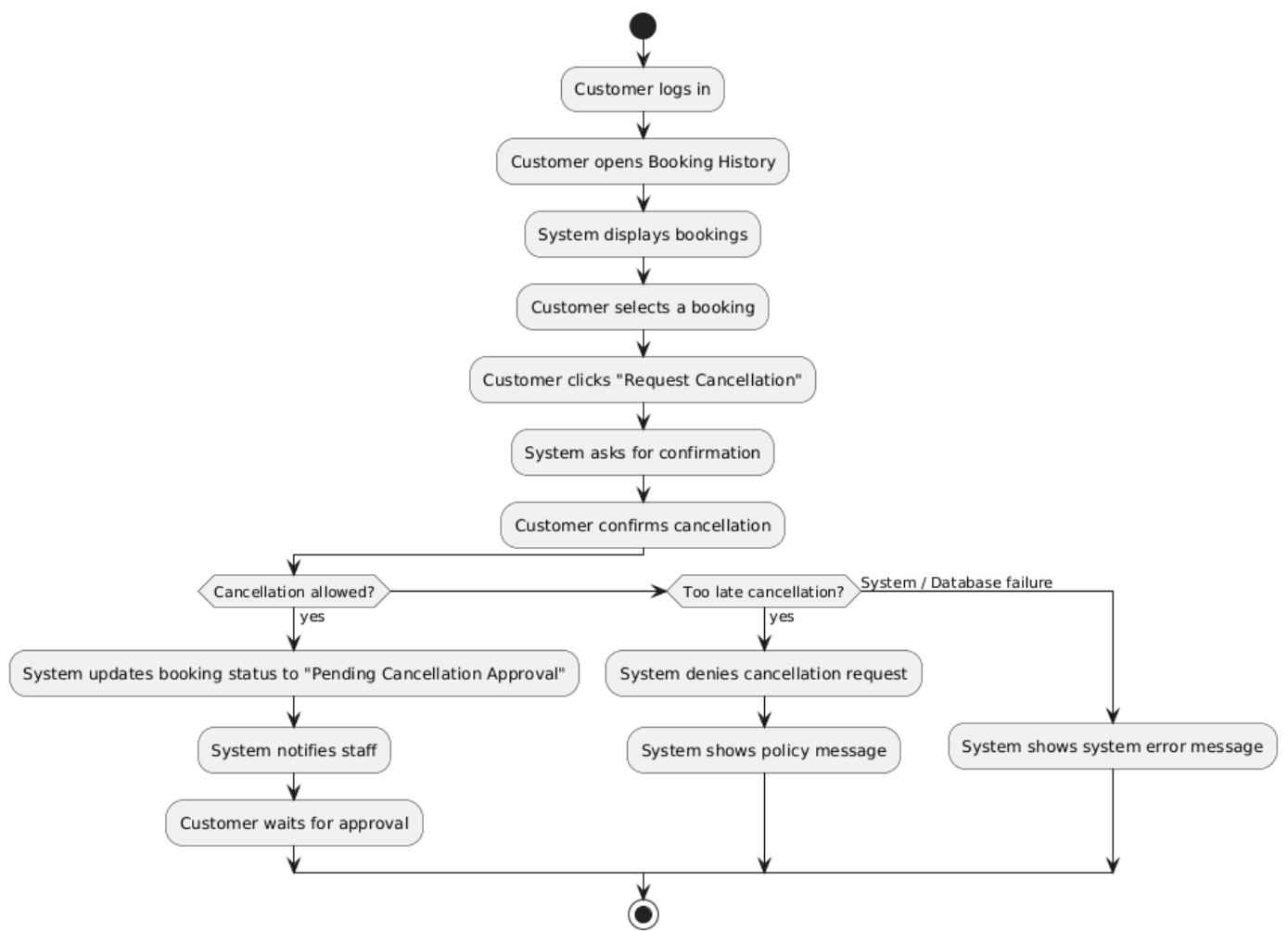


Figure 2.6.2: Activity Diagram for <Cancel Booking>

2.4.7 US007: User Story <Administrator> Sprint 5 - Admin Module

The <Generate Report> is an action can be taken by admin in the reporting module. In Table 2.7, it describes the <Generate Report> use case allows the administrator to generate and review rental and booking reports. The sequence diagram highlights the interaction between the administrator and the system when a report is requested. It shows how the system retrieves relevant booking or rental data and presents the generated report to the administrator. as shown in Figure 2.7.1. In Figure 2.7.2, it shows the activity diagram of flow of actions, including select report types, filter action, generating reports and export the reports.

Table 2.7: User Story Description for <Generate Report>

User story: Generate Report
ID: US007
User Story Description As an administrator, I want to generate rental and booking reports, So that I can review system performance, rental trends, and peak session.
Flow of events: 1. Admin navigates to Admin Dashboard 2. Admin expands Reports submenu in sidebar 3. Admin selects a report type 4. Admin select filtering criteria. 5. System displays the selected report main page. 6. System processes the data. 7. System displays the generated report with tables or charts. 8. Admin select one of export option. 9. System generates the report in the selected format. 10. System downloads or opens the exported report.
Alternative flow: Alternative flow – No Export Selected 1. Admin views the report without exporting it. 2. Flow ends.
Acceptance Criteria Postcondition - Admin is able to view generated rental statistic Precondition - Admin user is authenticated - Admin has administrator role
Exception flow: - If no data found, the system display an error message - If report generation fails, the system displays error message and suggest retry.

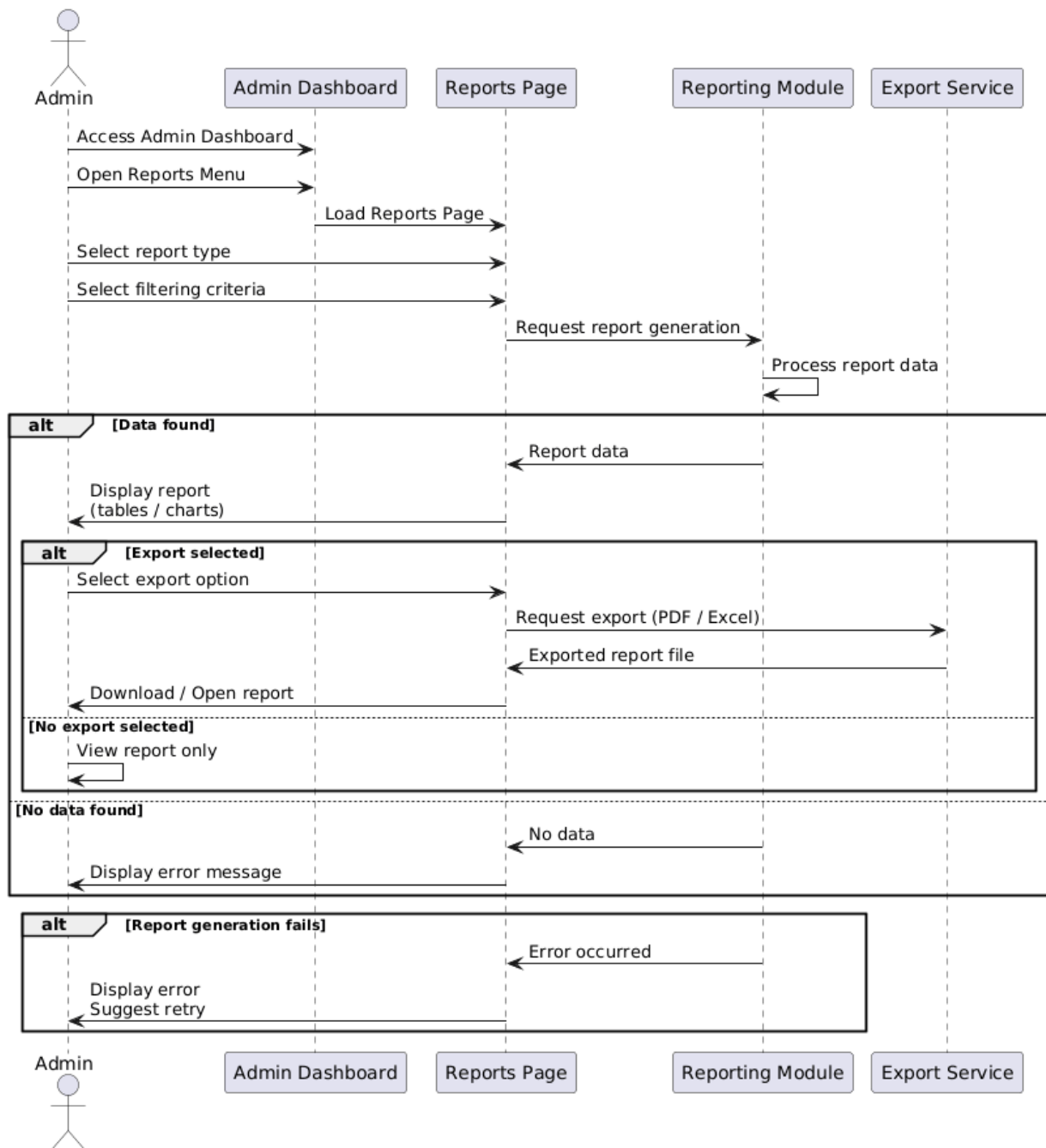


Figure 2.7.1: Sequence Diagram for <Generate Report>

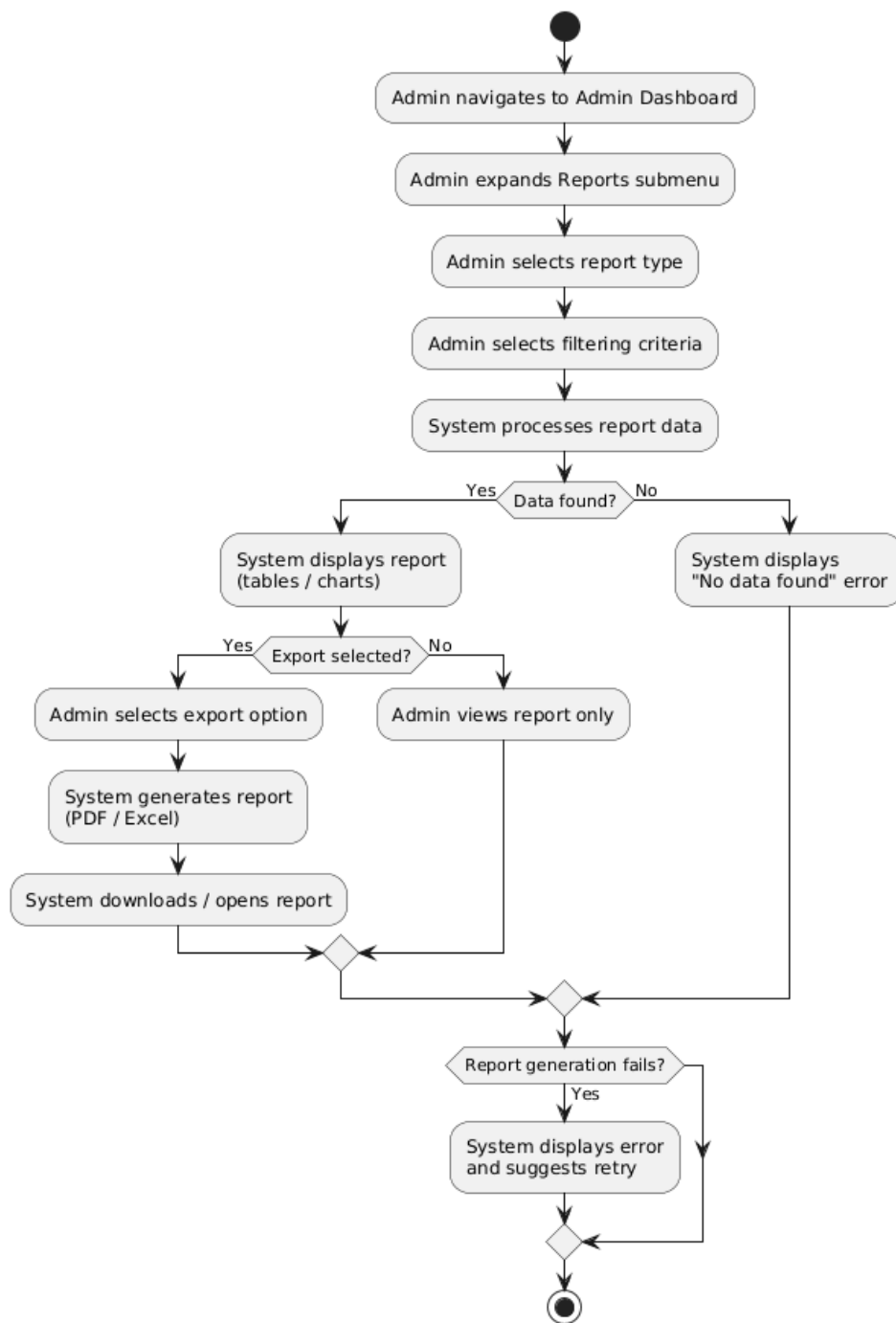


Figure 2.7.2: Activity Diagram for <Generate Report>

2.4.8 US008: User Story <Staff> Sprint 4 – Payment Module

Figure 2.8.1 sequence diagram models the dynamic behavior of the **Staff** member verifying a payment. Figure 2.8.2: Activity Diagram for <Verify Payment> illustrates the sequential workflow for User Story US008, beginning with the staff member logging into the system and navigating to the payment verification page where the system attempts to fetch the "Awaiting Verification" list. A decision node determines the next step based on whether the list is empty; if items exist, the staff selects a booking to inspect the proof of payment image, leading to a validation decision point where the staff either approves the receipt, causing the system to update the status to "Confirmed" and notify the customer, or rejects it by inputting a reason, which updates the status to "Payment Rejected" and alerts the customer to re-upload, with both paths eventually merging to conclude the activity.

Table 2.8: User Story Description for <Process Payment>

User story: Verify Payment
ID: US008
User Story Description As a staff member, I want to verify incoming payments So that bookings are confirmed only when payment is valid
Flow of events: 1. Staff logs into the system. 2. Staff navigate the "Payment Verification" or "Booking" list. 3. System displays bookings with status "Awaiting Payment Verification". 4. Staff selects a booking to review. 5. System displays the uploaded payment receipt image. 6. Staff compare the receipt with the company bank records. 7. Staff clicks "Approve Payment". 8. System updates booking status to "Confirmed/Paid". 9. System sends notification to the customer.
Alternative flow: Alternative flow -Rejection Flow 1. Staff determines the receipt is invalid or unclear. 2. Staff clicks "Reject Payment". 3. Staff enters a reason for rejection (e.g., "Image blurry", "Wrong amount"). 4. System updates booking status to "Payment Rejected". 5. System notifies the customer to re-upload the receipt.

Acceptance Criteria

Postcondition

- Booking status is updated to "Confirmed" or "Rejected" based on staff action.

Precondition

- Customer must have uploaded a receipt

Staff must be logged in.

other conditions

- The system handles manual verification as no payment gateway is integrated.

Exception flow:

If the receipt image fails to load, the system displays "Error loading image" and allows the staff to refresh the verification page.

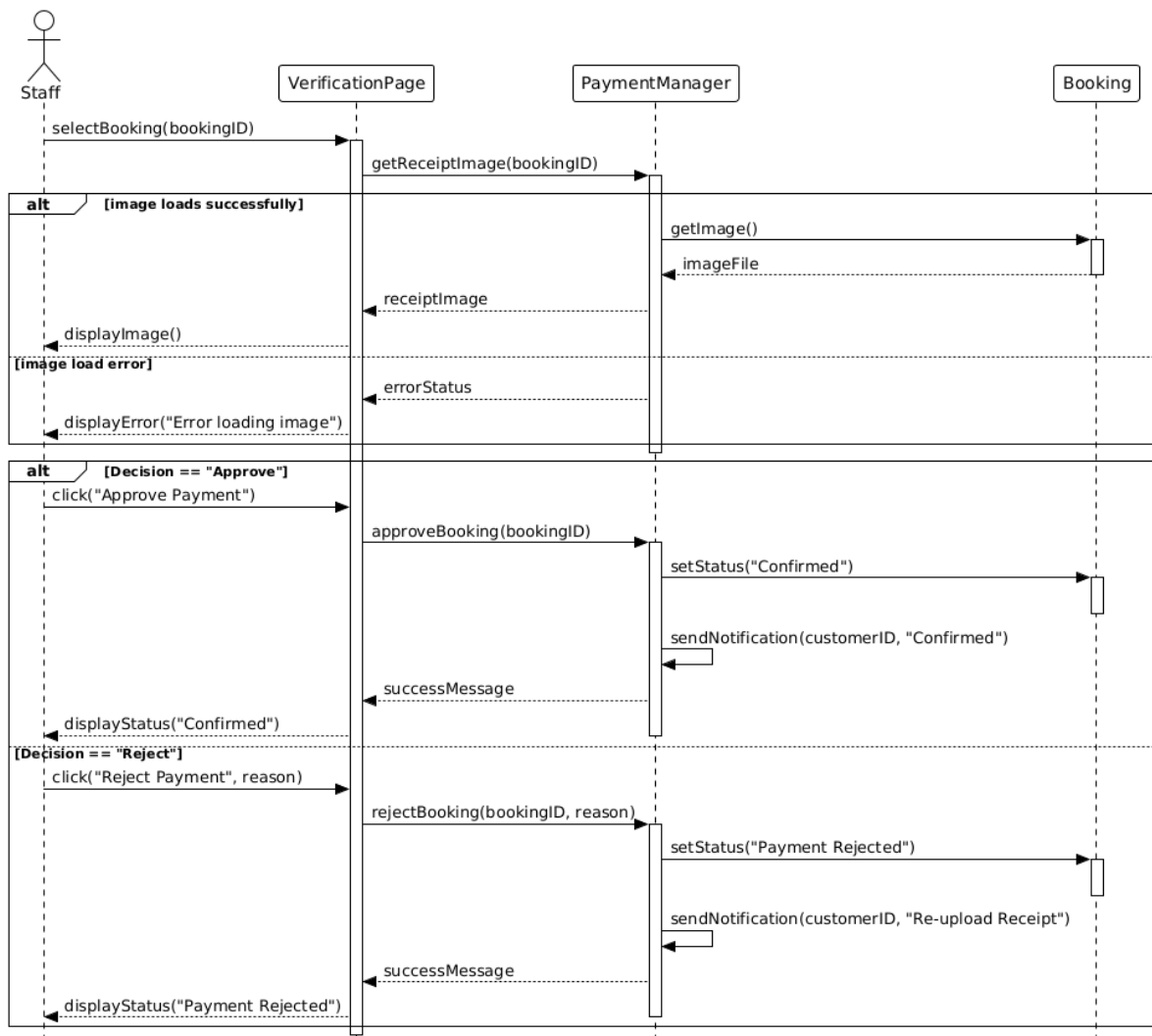


Figure 2.8.1: Sequence Diagram for <Verify Paymentt>

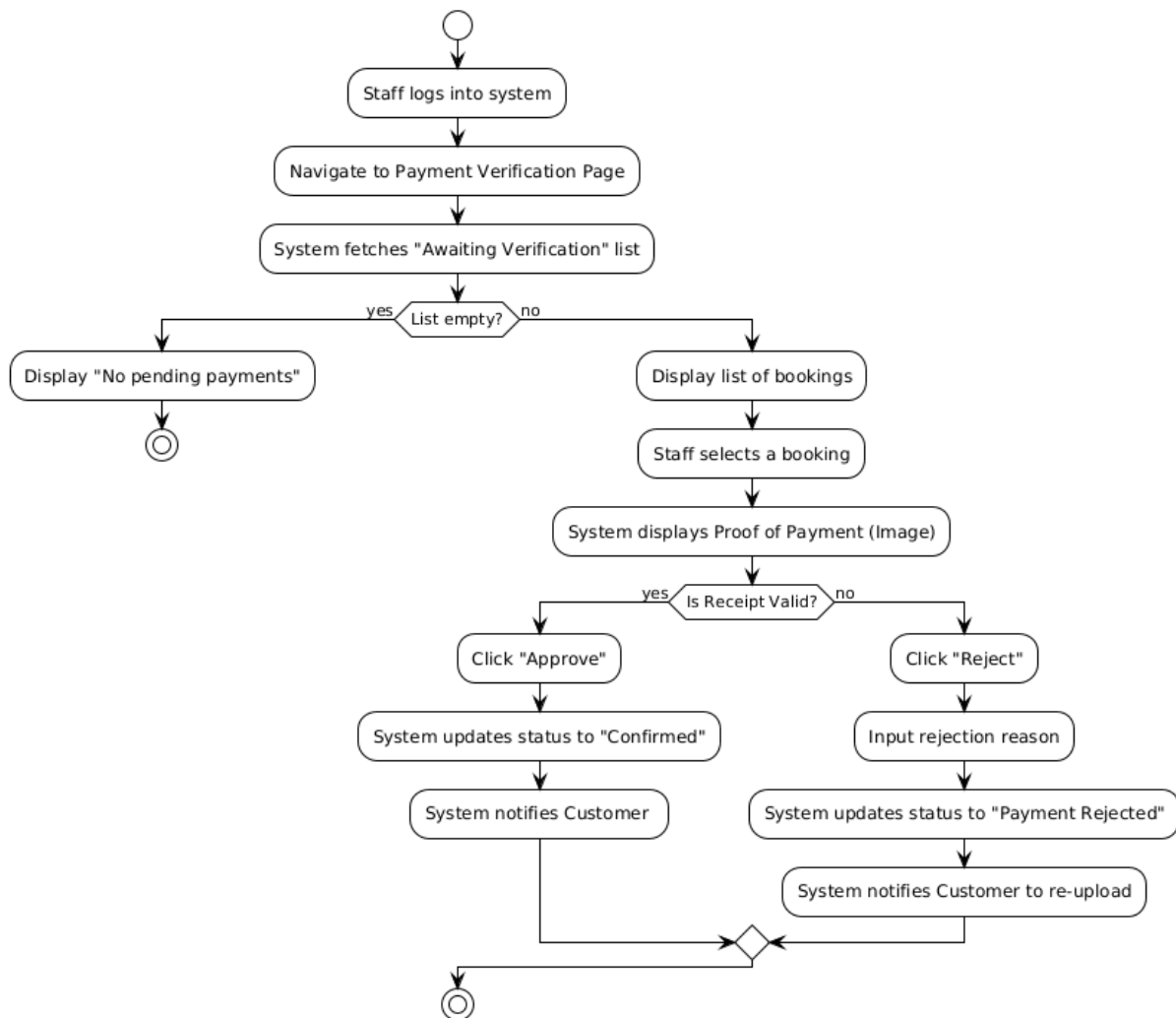


Figure 2.8.2: Activity Diagram for <Verify Payment>

2.4.9 US009: User Story <Customer> Sprint 4 – Payment Module

Figure 2.9.1 sequence diagram illustrates the interactions required for a customer to make a payment by uploading a receipt. It covers two main phases: calculating the required deposit amount (Alternative Flow) and processing the payment submission (Main Flow with Exception).

Figure 2.9.2: Activity Diagram for <Make Payment> illustrates the payment submission workflow, starting with the customer viewing the booking summary which triggers the system to calculate the required deposit based on whether the rental duration is less than 15 days (RM50) or longer (Total Rental Price). Following the display of the total amount and bank details, the customer performs a manual transfer and uploads a receipt image, leading to a validation decision where a valid format causes the system to save the receipt, update the status to "Awaiting Verification," and show a success message, while an invalid format triggers an error display and prompts a re-upload before the process concludes.

Table 2.9: User Story Description for <Process Payment>

User story: Make Payment
ID: US009
User Story Description As a customer, I want to make payments for my booking by uploading a receipt, So that my reservation is confirmed.
Flow of events: 1. Customer logs into the system. 2. Customer navigates to "My Bookings" or is redirected after booking. 3. System displays the booking summary and total amount required (Deposit + Rental) . 4. System displays Hasta Travel's bank account details. 5. Customer performs a manual bank transfer outside the system. 6. Customer clicks "Upload Receipt" and selects the proof of payment image 7. Customer clicks "Submit Payment". 8. System saves the image and updates the booking status to "pending "
Alternative flow: Alternative flow -Deposit Handling 1. If the rental duration is less than 15 days, the system requests a fixed RM50 deposit . 2. If the rental duration is more than 15 days, the system requests a deposit equal to the full rental price

Acceptance Criteria

Postcondition

- Receipt image is stored in the database.Booking status changes to "Awaiting Verification".

Precondition

- Booking must be created and in "Awaiting Payment" status.
- other conditions
- Only manual upload is supported; no e-wallet or QR integration .

Exception flow:

If the uploaded file is not a valid image format (JPG/PNG), the system displays "Invalid file format. Please upload an image."

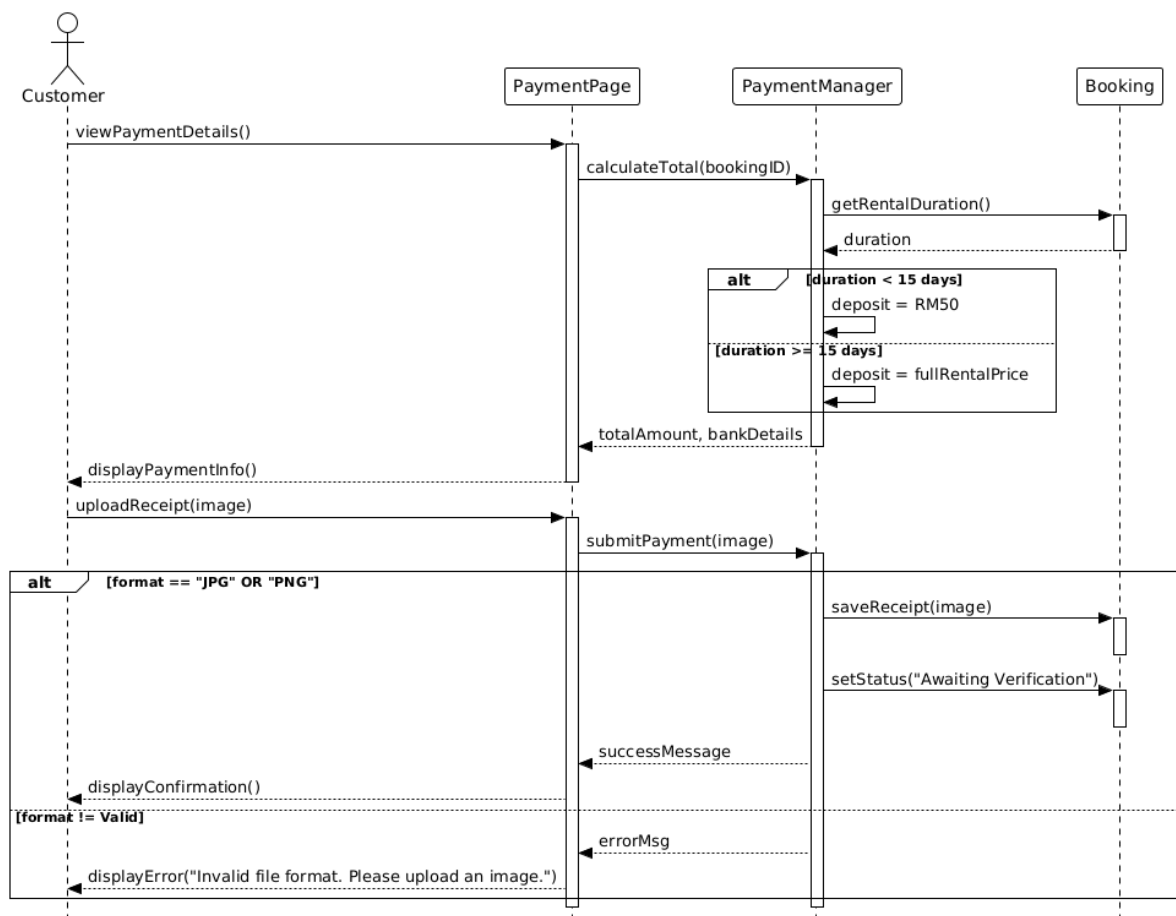


Figure 2.9.1: Sequence Diagram for <Make Payment>

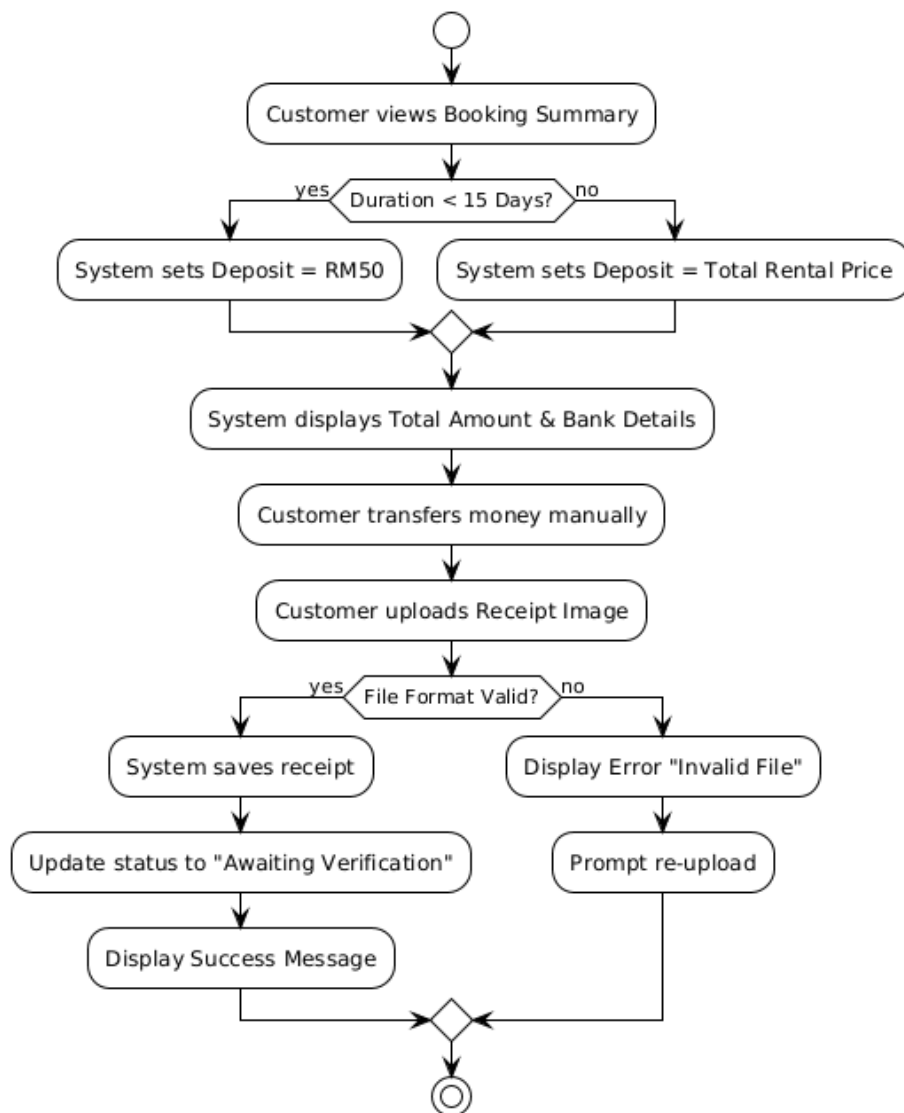


Figure 2.9.2: Activity Diagram for <Make Payment>

2.4.10 US0010: User Story <Customer> Sprint 4 – Payment Module

Figure 2.10.1 sequence diagram models the interactions between the customer(actor) and the system objects during the execution of the " Generate Invoice " scenario. It arranges these interactions in a time sequence to show how the system responds to the user's request. Figure 2.10.2: Activity Diagram for < Generate Invoice > illustrates the receipt retrieval workflow using swimlanes to partition actions between the customer and the System. The process initiates with the customer selecting a booking record, prompting the system to verify payment status; if unverified, the flow terminates with a status message, whereas a verified status allows the customer to request the receipt, triggering the system to generate a PDF. A subsequent decision node handles potential generation errors by displaying a failure message, while successful generation presents the receipt to the customer for viewing, with an optional path to download or print the document before the activity concludes

Table 2.10: User Story Description for <Process Payment>

User story: Generate Invoice
ID: US010
User Story Description As a staff member or customer, I want to generate, view, and download the invoice or receipt for a verified booking, So that customers receive official payment records and the system maintains accurate accounting documentation.
Flow of events: 1. Staff successfully verifies a customer payment (triggered from US016). 2. The system automatically generates a digital invoice/receipt containing: - Booking ID - Customer name - Vehicle details - Rental amount - Deposit paid 3. The system links the generated receipt to the corresponding booking record. 4. The system sends the receipt to the customer via email. 5. Staff or customer navigates to Booking Details or Booking History. 6. The user selects a confirmed booking. 7. The user clicks "View Receipt". 8. The system displays the receipt in PDF format. 9. The user may download or print the receipt.
Alternative flow: Alternative Flow A – Manual Generation by Staff 1 Staff navigates to a completed booking. 2 Staff clicks "Generate Invoice". 3 The system generates the receipt and links it to the booking. 4 The receipt becomes available for viewing and download. Alternative Flow B – Email Retrieval 1 The system emails the receipt automatically after payment verification. 2 The customer opens their email. 3 The customer accesses the receipt via the email attachment or link.

Acceptance Criteria

Precondition

- The booking status must be "Confirmed".
- The payment must be fully verified (triggered from US016).
- The customer must be logged in to access booking history..

Postcondition

- A digital invoice/receipt is generated and linked to the booking.
- The receipt is accessible to both staff and customer.
- The receipt is available in PDF format for viewing or download.

Exception flow:

- If a user attempts to view a receipt for a booking that is not yet verified, the system displays: "Receipt not available. Payment is currently being verified."
- If PDF generation fails due to a server error, the system displays: "Unable to generate receipt. Please try again later."

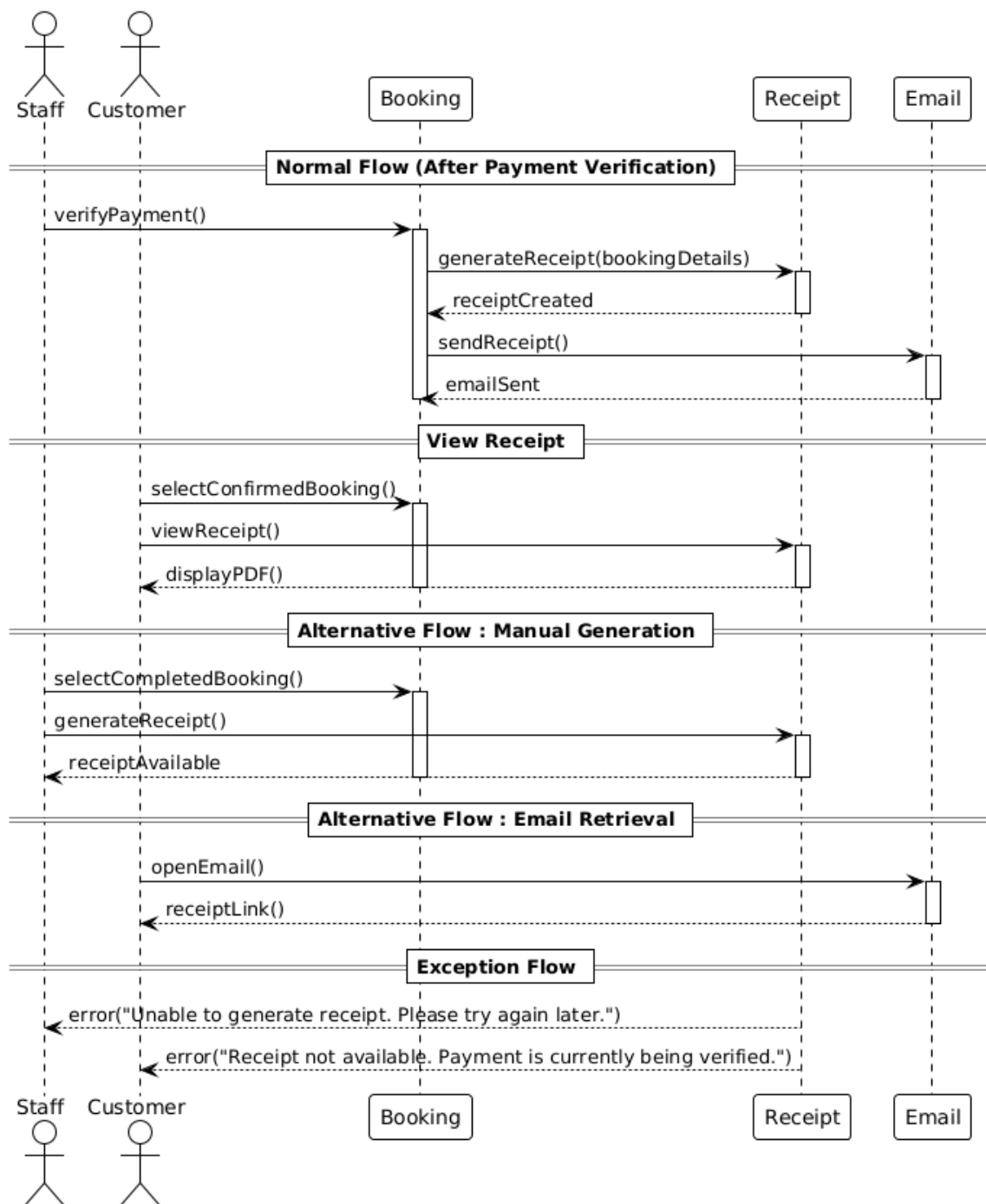


Figure 2.10.1: Sequence Diagram for <Generate Invoice>

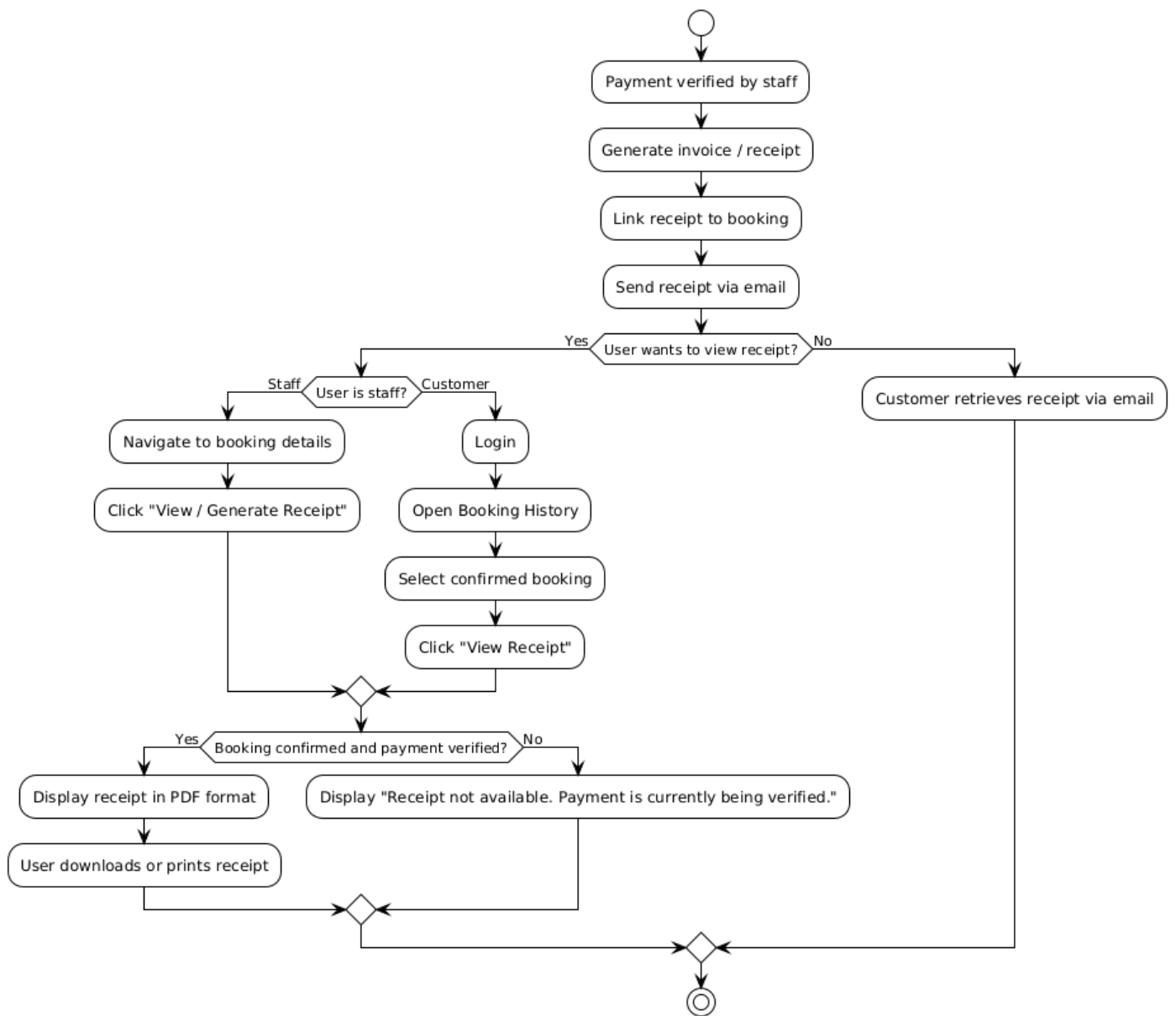


Figure 2.10.2: Activity Diagram for <Generate Invoice>

2.5 Performance and Other Requirements

Non-functional requirements describe how the Hasta Car Rental system should operate rather than what it should do. They ensure the system's performance, usability, and quality meet stakeholder expectations. These requirements are categorized into Performance Requirements and Software System Attributes to align with the ISO/IEC/IEEE 29148 standard.

Performance Requirement

Performance requirements define the system's capability to user requests and handle data efficiently. These requirements include following:

- Response Time: The system shall load the homepage within 3 seconds
- Throughput: The system shall process payment verification updates within 2 seconds after approval
- Capacity: The system shall support a minimum of 10,000 registered users.
- Availability: Users shall be able to access booking and payment features 24/7.

Software System Attributes Requirement

- Software system attributes define the overall qualities or characteristics of the software. These attributes are the foundation on which the software is built, and they include the following:
- Usability: Users can easily register, log in, book cars, view booking history, and cancel reservations with minimal effort.
- Reliability: Booking, cancellation, and payment processes are tested to ensure accuracy and dependability.
- Maintainability: The system is built using modular design principles, making it easy to update, repair, or enhance individual components without affecting the rest of the system.
- Portability: Hasta is designed to operate on multiple platforms, including web browsers and mobile devices.
- Compatibility: The system is compatible with multiple web browsers such as Chrome, Firefox, and Edge and mobile operating systems such as Android and iOS.
- Safety: Data operations, financial transactions, and system actions are controlled to prevent accidental data leak.
- Legal and Regulatory: Hasta complies with relevant laws, regulations, and industry standards regarding data privacy, user protection, and secure financial transactions.
- Environmental: Efficient coding and database management reduce resource usage while maintaining performance.

2.6 Design Constraints

Design Constraints

The development of the Hasta Car Booking System is subject to several organizational, environmental, technical, and regulatory constraints that guide how the system should be designed, implemented, and operated. These constraints ensure that the system will conform to institutional standards, will operate reliably within the environment in which it is placed, and will meet the requirements imposed externally from security, hardware, and software perspectives.

Environmental Constraints

The system will be deployed and accessed within typical indoor environments such as offices, customer accommodations, or campus facilities. Therefore, it must operate reliably under normal indoor conditions. Constraint: The system shall be able to operate reliably in temperatures between 0°C and 40°C and at normal indoor humidity and lighting.

Hardware Constraints

As a web-based system, the system will be accessed on user devices with varying hardware capabilities. It shall not require high-performance hardware to operate effectively. Constraint: The system shall operate smoothly on devices with at least 4GB of RAM, standard dual-core processors, and stable internet connectivity.

Software and Platform Compatibility Constraints

The system shall be compatible with common browsers and operating systems used by customers, staff, and supervisors. Constraint: The system shall support modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari, and shall operate on Windows, macOS, Android, and iOS platforms.

Security Constraints

The system deals with sensitive information including personal identification documents, booking records, and proof of payment. Stronger security measures must be put into place to meet confidentiality and integrity requirements. Constraint: All sensitive user data shall be encrypted using AES-256 encryption, and the system shall implement secure authentication and role-based access control. Constraint: The system shall follow secure development best practices to protect against common vulnerabilities like SQL injection, XSS, and unauthorized access.

Organizational and Policy Constraints

The system shall meet Hasta Travel's operational workflow and university policies on data protection and record maintenance. Constraint: The system shall store booking, transaction, and document records in line with the organization's policies on data retention. Constraint: The workflow for the approval of the booking, vehicle management, and reporting shall follow the procedures outlined by the administrative staff and supervisors.

Legal and Regulatory Constraints

The system shall ensure that personal data are processed in accordance with relevant Malaysian data protection guidelines, for example, PDPA. Constraint: The system shall ensure that the users' personal information is kept confidential and accessed only by authorized personnel.

Performance Constraints

The system is anticipated to handle multiple users simultaneously at any given time in the case of peak bookings. Constraint: The system shall be able to support at least 200 concurrent users with a maximum response time of less than 3 seconds for standard operations. Constraint: The system shall update real-time vehicle availability with minimum delay.

3. Architectural Rationale

The architectural style that is adopted into the Hasta Car Booking System is Layered Architecture, which organizes the layers into multiple layers. Each layer serves a purpose in the system, starting with the User Interface layer which handles all interface and user interactions. The next layer is the Management layer, which is responsible for authentication and authorization. After that is the Core Business Logic layer which contains essential processing rules, workflows, computations and system utilities. Lastly, the System Support layer includes the operating system and database management system.

3.1 Architecture Style and Rationale

Layered Architecture is selected for the Hasta Car Rental System because it provides a clear separation of responsibilities across the system, making it easier to manage, extend, and maintain. By organizing the system into the User Interface Layer, Management Layer, Core Business Logic/Application Functionality Layer, and System Support Layer, each part of the system can operate independently without affecting the others. This modular structure enhances maintainability by allowing developers to update or modify one layer without disrupting the entire system. It also improves security by isolating authentication and authorization within the Management Layer, ensuring controlled access to protected resources. Additionally, the architecture supports scalability and future expansion, as new features can be added within the appropriate layer without requiring major structural changes. The separation also simplifies testing, since each layer can be tested individually, and supports team collaboration by enabling different development teams to work on different layers simultaneously.

4. Architectural Views

Layered Architecture

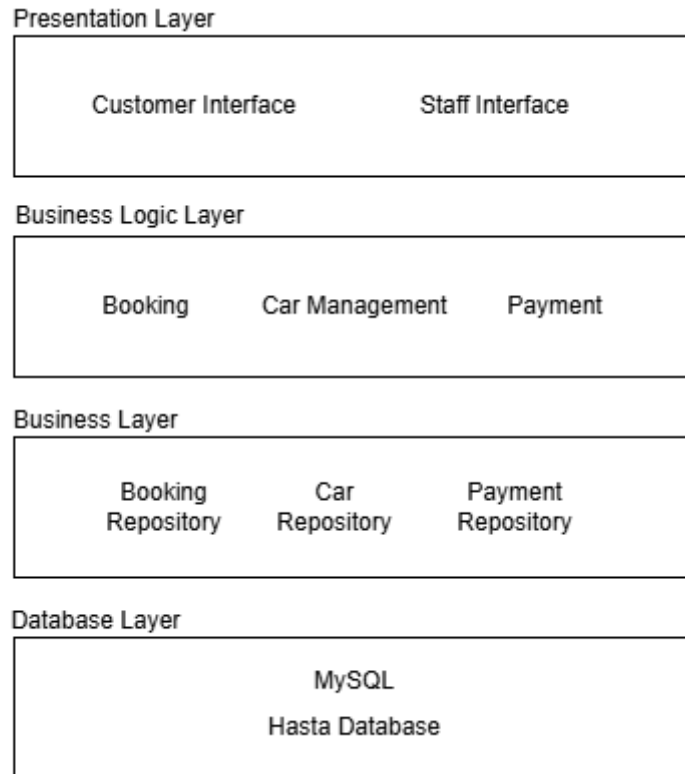


Figure 4.2 – Architectural Views

The Hasta Travel Car Rental System adopts a Layered Architecture based on Philippe Kruchten's 4+1 View Model, which organizes the system into complementary architectural perspectives to enhance clarity, modularity, and maintainability. The Use Case (Scenario) View identifies key system interactions and ensures all architectural decisions align with user needs. The Logical View represents the main abstractions, such as classes and relationships, that support functional requirements. The Process View describes the system's dynamic behaviour, communication flows, and runtime interactions. The Development (Implementation) View shows how the software is structured into modules and components during development, emphasizing maintainability and code organization. Finally, the Physical (Deployment) View illustrates how system components are deployed across hardware infrastructure. Together, these views provide a comprehensive understanding of how the system behaves, is structured, implemented, and deployed.

Figure 4.4 shows the Hasta Car Rental System architecture based on the Model-View-Controller (MVC) pattern. The system is divided into three main parts to keep it organized and easy to maintain. The View handles all user interactions and displays information on the screen. The Controller receives input from the View, processes actions, applies rules, and decides what should happen next. The Model stores and manages all system data, including bookings, vehicles, and user accounts.

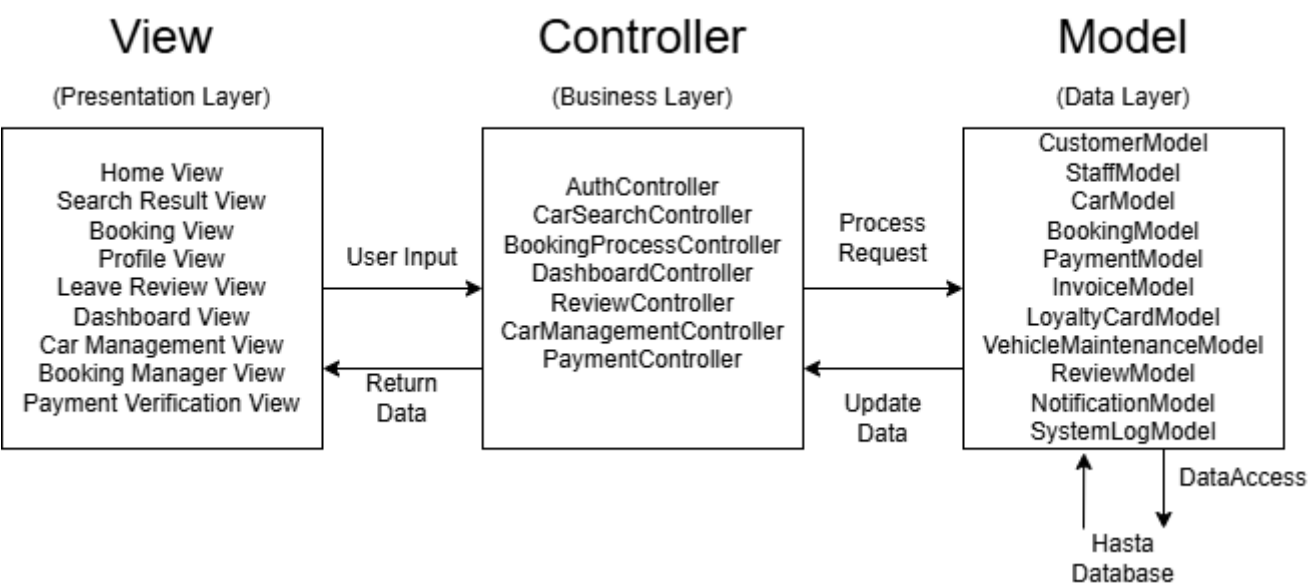


Figure 4.4 - Model-View-Controller (MVC)

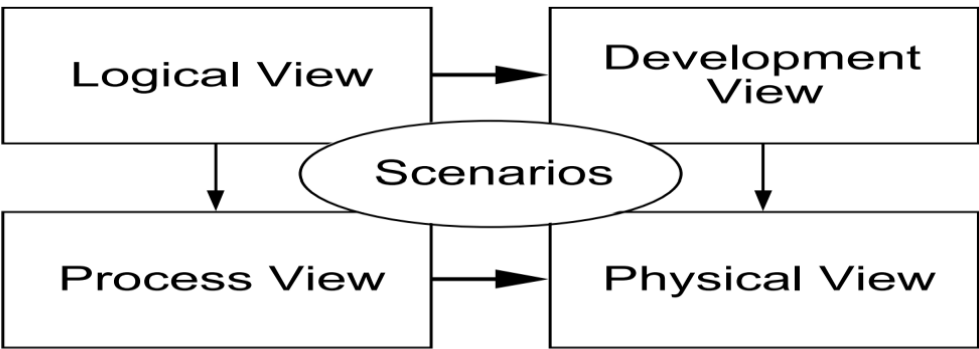


Figure 4.3 – Architectural Views

Table 4.1: Description of Architecture view

Architectural Views	Description	Diagrams used
Use Case View	The Use Case View highlights the key interactions between users and the Hasta Travel Car Rental System, showing how customers, staff, and administrators perform essential tasks such as browsing vehicles, managing bookings, processing payments, and overseeing system operations. These core scenarios ensure that all other architectural views align with real system usage and functional requirements.	Use Case Diagram
Logical View	The Logical View identifies the main abstractions and object classes within the car rental system, including users, vehicles, bookings, payments, and cancellations. It illustrates how these entities relate to one another and forms the conceptual structure needed to support the system's core functionalities, such as booking management and payment processing.	Class Diagram
Process	The Process View describes the dynamic behavior of the Hasta Travel Car Rental System during runtime. It focuses on how components interact—for example, how booking requests flow from the customer interface to the booking service, how cancellations trigger updates, and how payment transactions are validated and recorded. It explains system processes, message flows, and operational sequencing.	Sequence Diagram, Activity Diagram
Development (or Implementation) View	The Development View shows how the system's software is decomposed into modules for implementation, such as the Authentication Service, Booking Service, Payment Service, Vehicle Management Service, and their corresponding repositories. This view emphasizes maintainability and modular design, ensuring that system components can be independently developed and updated.	Component Diagram Package Diagram

Physical (or Deployment) View	The Physical View illustrates how the car rental system is deployed on physical hardware, showing how the web application, business logic, a It typically includes a web server hosting the application interface, an application server managing business logic, and a database server hosting the HastaTravelDB, enabling communication across a networked deployment environment. components are distributed across servers.	Deployment Diagram
-------------------------------	---	--------------------

4.1 Use Case View

The Use Case Design Figure 4.1 below illustrates the design-level interactions between actors and the system. It represents how different user roles interact with specific system functionalities.

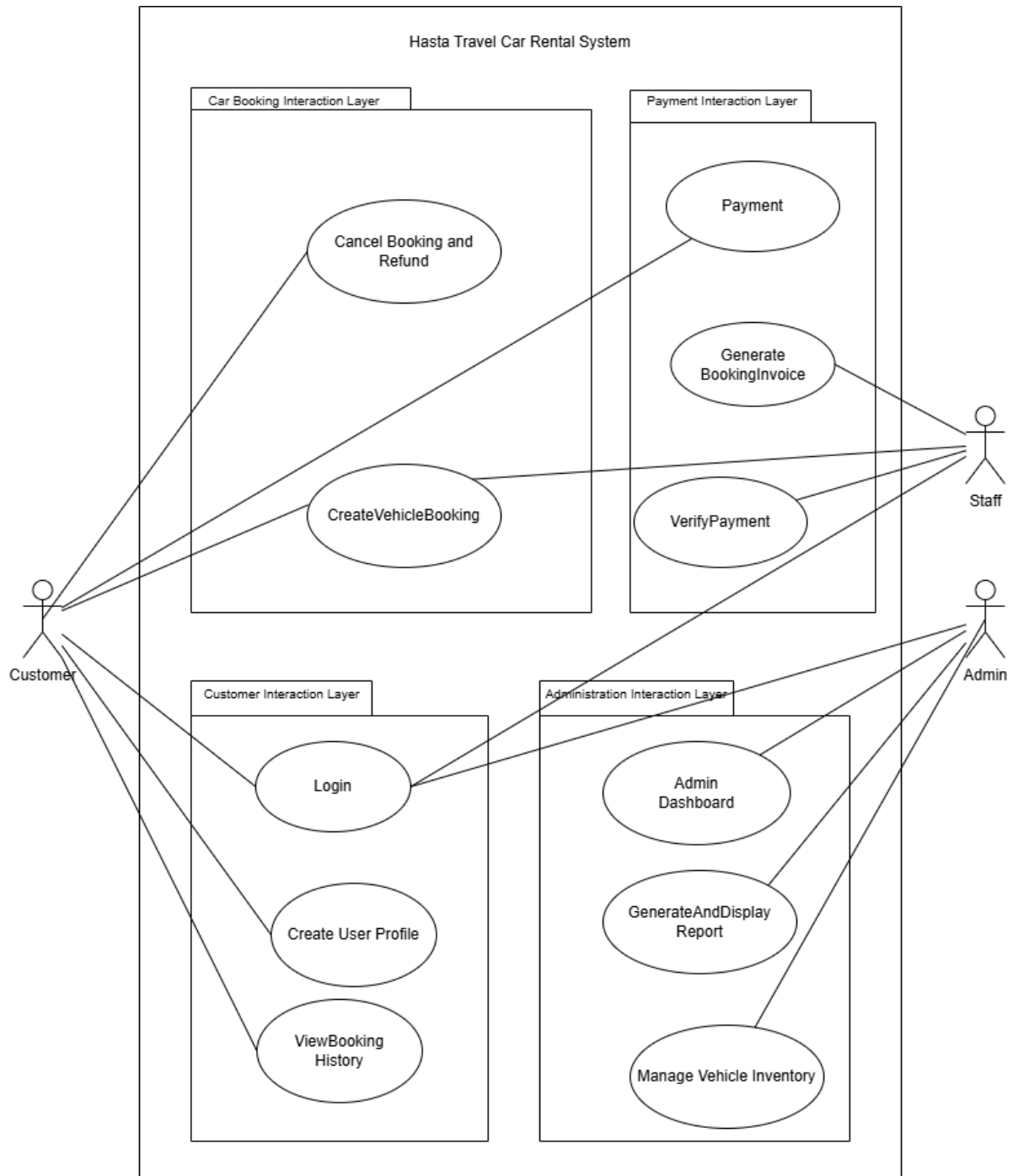


Figure 4.1: Use Case Design for <Hasta Travel Car Rental>

4.2 Implementation View

The component diagram illustrates the three-layer architecture of the Hasta Travel Car Rental System, consisting of the User Interface Layer, the Domain Layer, and the Data Access Layer. The User Interface Layer includes the Customer Web UI, Staff Web UI, and Admin Web UI, which send requests to the Domain Layer where business logic is handled by components such as the Authentication Service, Booking Service, Cancellation Service, and Payment Service. These domain components enforce system rules and coordinate operations before interacting with the Data Access Layer, which contains repositories responsible for retrieving and storing information related to users, bookings, and payments. All repositories communicate directly with the centralized HastaTravelDB, where the system's persistent data is stored. The flow of arrows shows how requests propagate downward through the layers for processing and return upward with completed responses, demonstrating a clear separation of concerns and maintainable system structure.

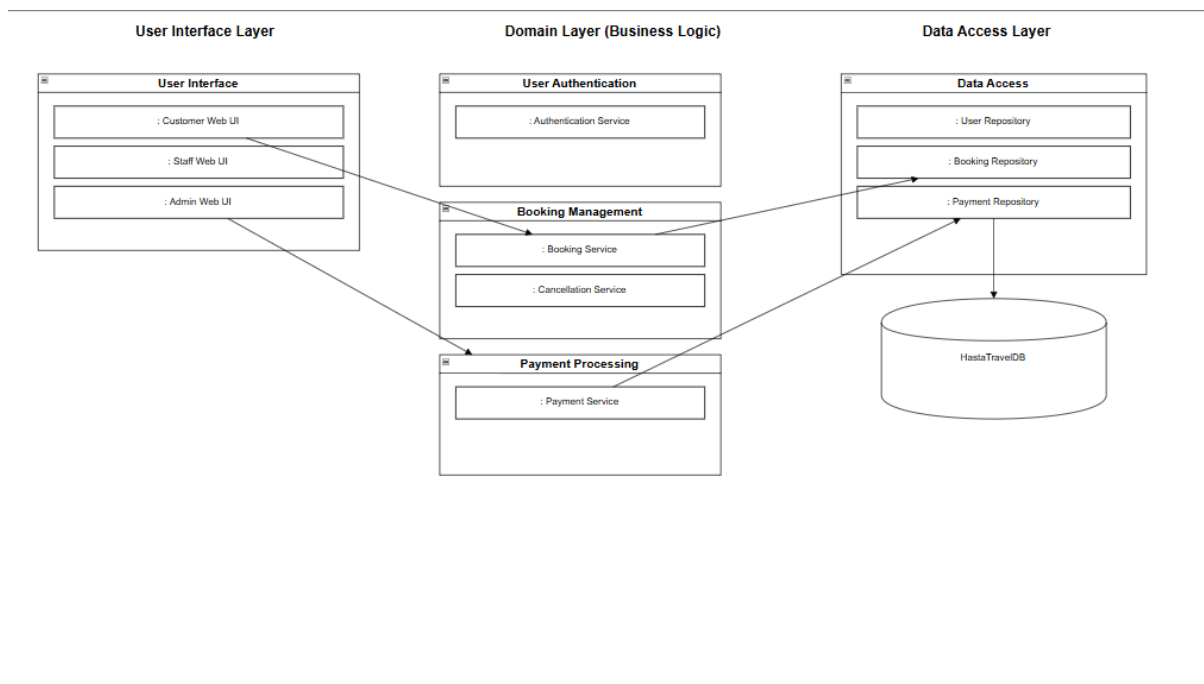


Figure 4.2.1: Component Diagram of <Hasta Travel Car Rental System>

The package diagram in the logical view shows how the system is organized into packages or subsystems as shown in Figure 4.2.2. It illustrates the relationships and dependencies between these packages, helping to manage complexity. This diagram provides a high-level view of the system's structure and modular organization.

Hasta Travel Car Rental System Package Diagram

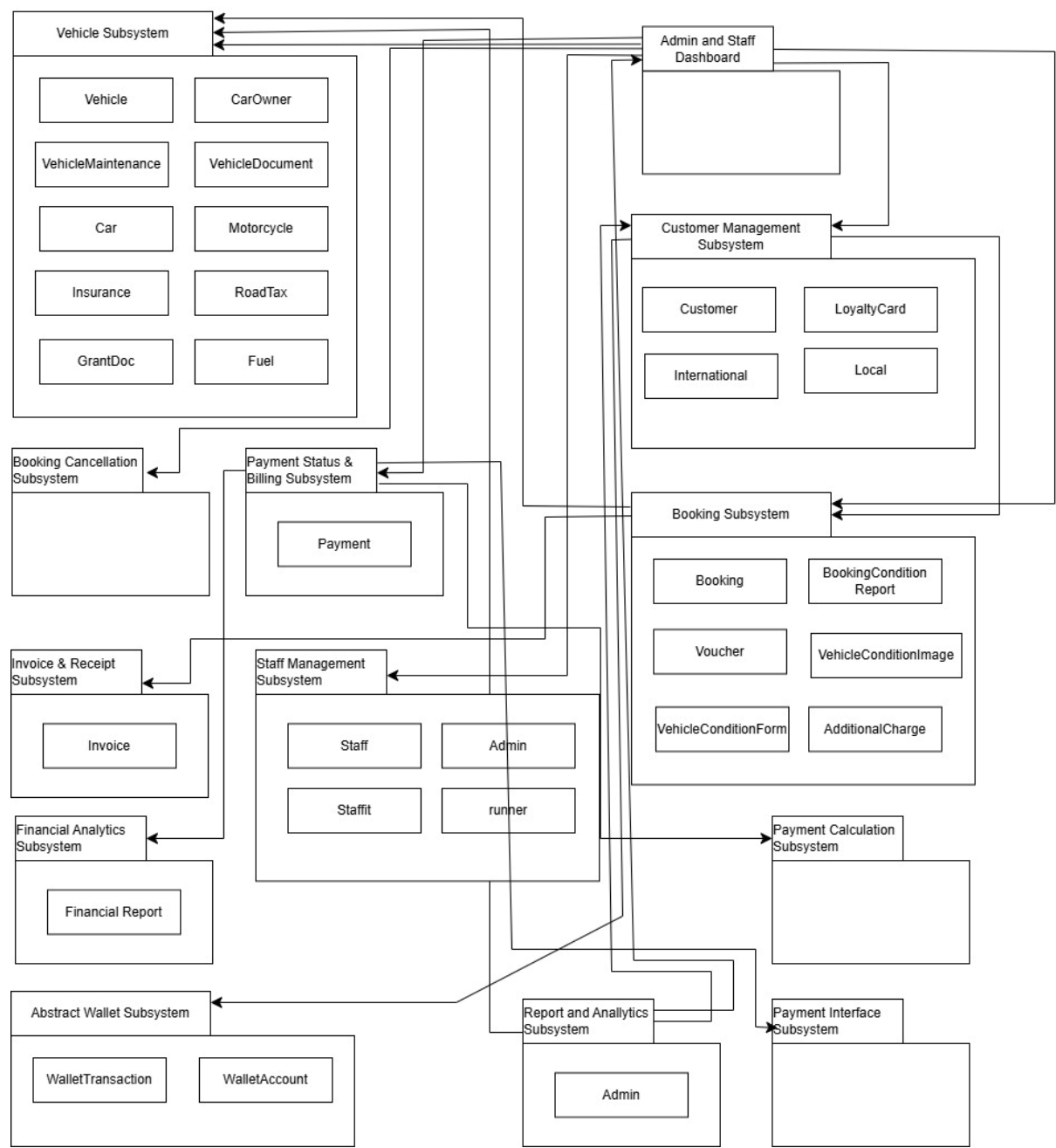


Figure 4.2.2: Package Diagram for <Hasta Travel Car Rental System>

The part below show the package diagram for each subsystem included in Figure 4.2.2. Figures 4.2.3 to 4.2.17 present the package diagrams for the various subsystems of the system which describes the logical grouping of related classes. Figure 4.2.3 depicts the Vehicle Subsystem, Figure 4.2.4 illustrates the Booking Subsystem, Figure 4.2.5 represents the Customer Management Subsystem, Figure 4.2.6 shows the Staff Module Subsystem and Figure 4.2.7 illustrates the Admin Module Subsystem. Figure 4.2.8 represents the Reporting & Analytics Subsystem and Figure 4.2.9 shows the Booking Cancellation Subsystem. Figure 4.2.10 illustrates the Payment Calculation Subsystem whereas Figure 4.2.11 depicts the Payment Status & Billing Subsystem. Figure 4.2.12 presents the Invoice & Receipt Subsystem. Figure 4.2.13 represents the Abstract Wallet Subsystem. Figure 4.2.14 illustrates the Financial Analytics Subsystem and Figure 4.2.15 shows the Payment Interface Subsystem.

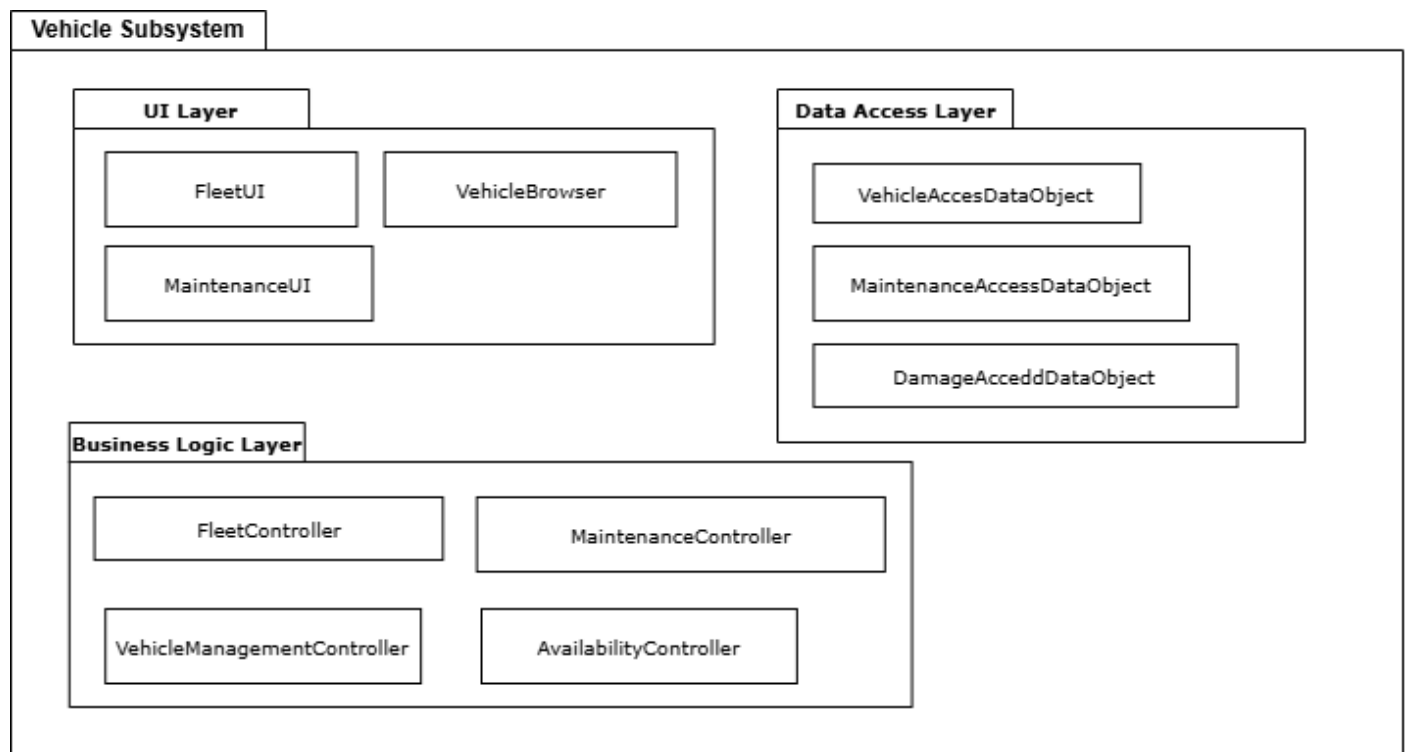


Figure 4.2.3: Package Diagram for <Vehicle> Subsystem

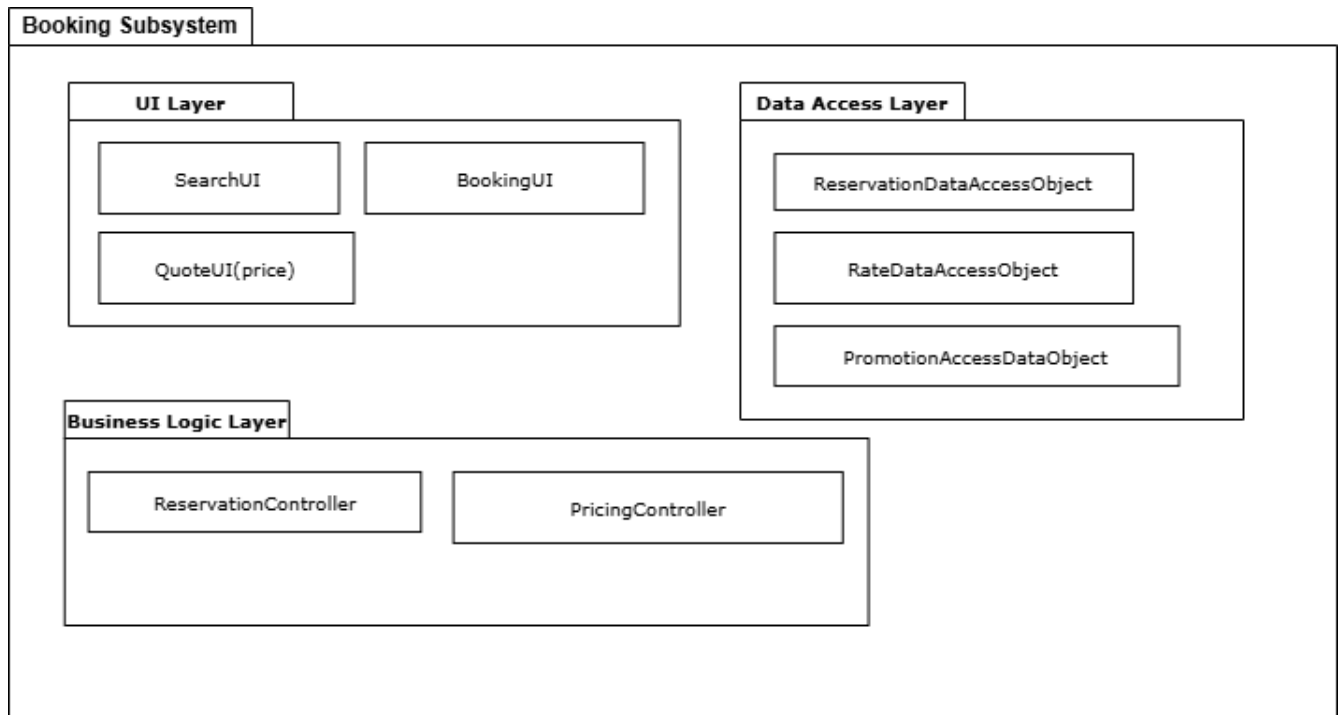


Figure 4.2.4: Package Diagram for <Booking> Subsystem

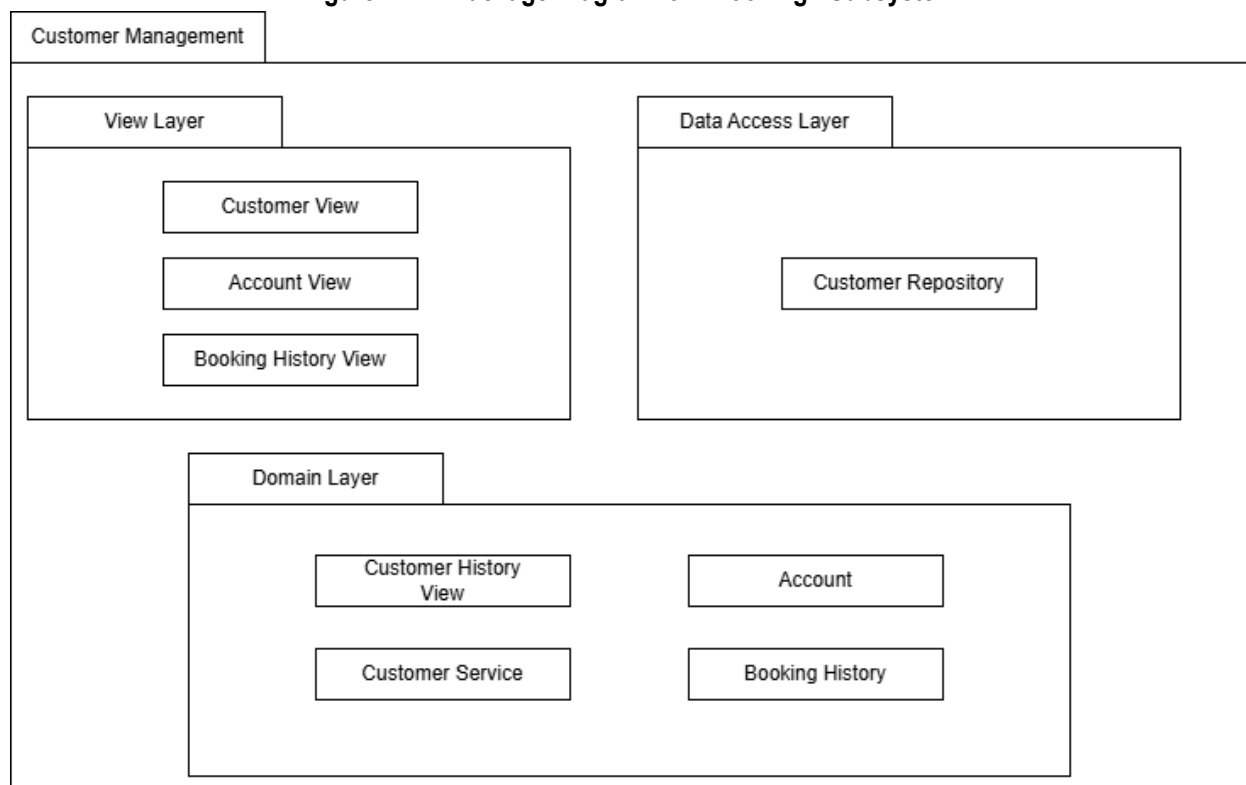


Figure 4.2.5: Package Diagram for <Customer Management> Subsystem

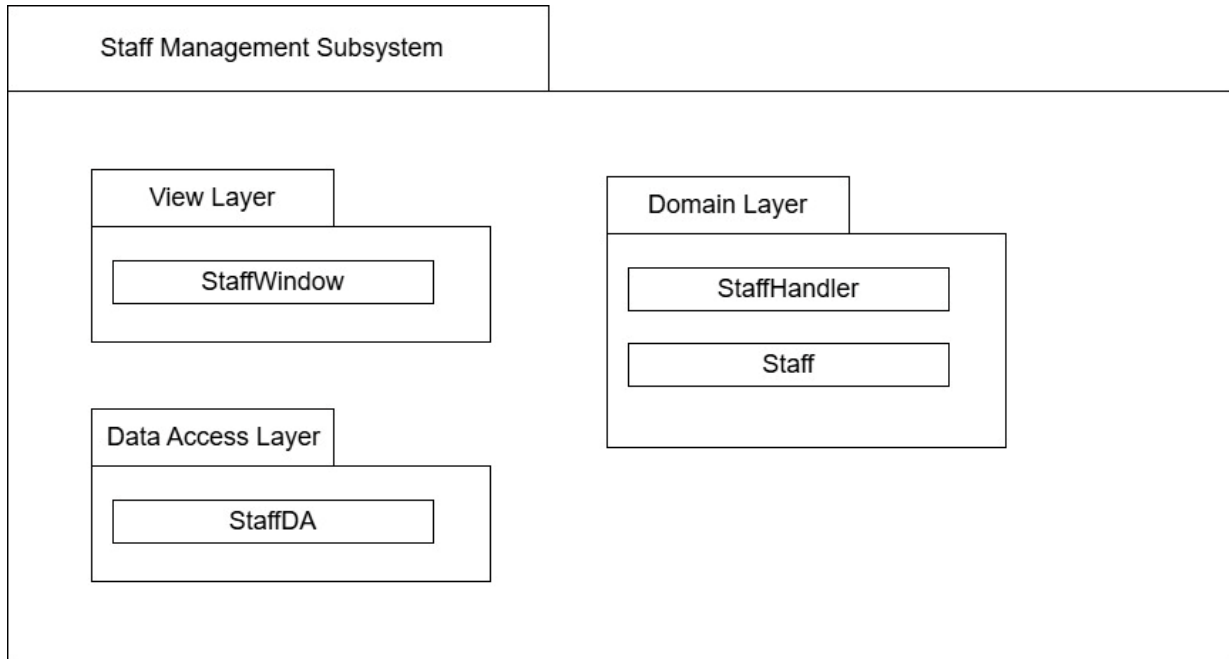


Figure 4.2.6: Package Diagram for <Staff Management> Subsystem

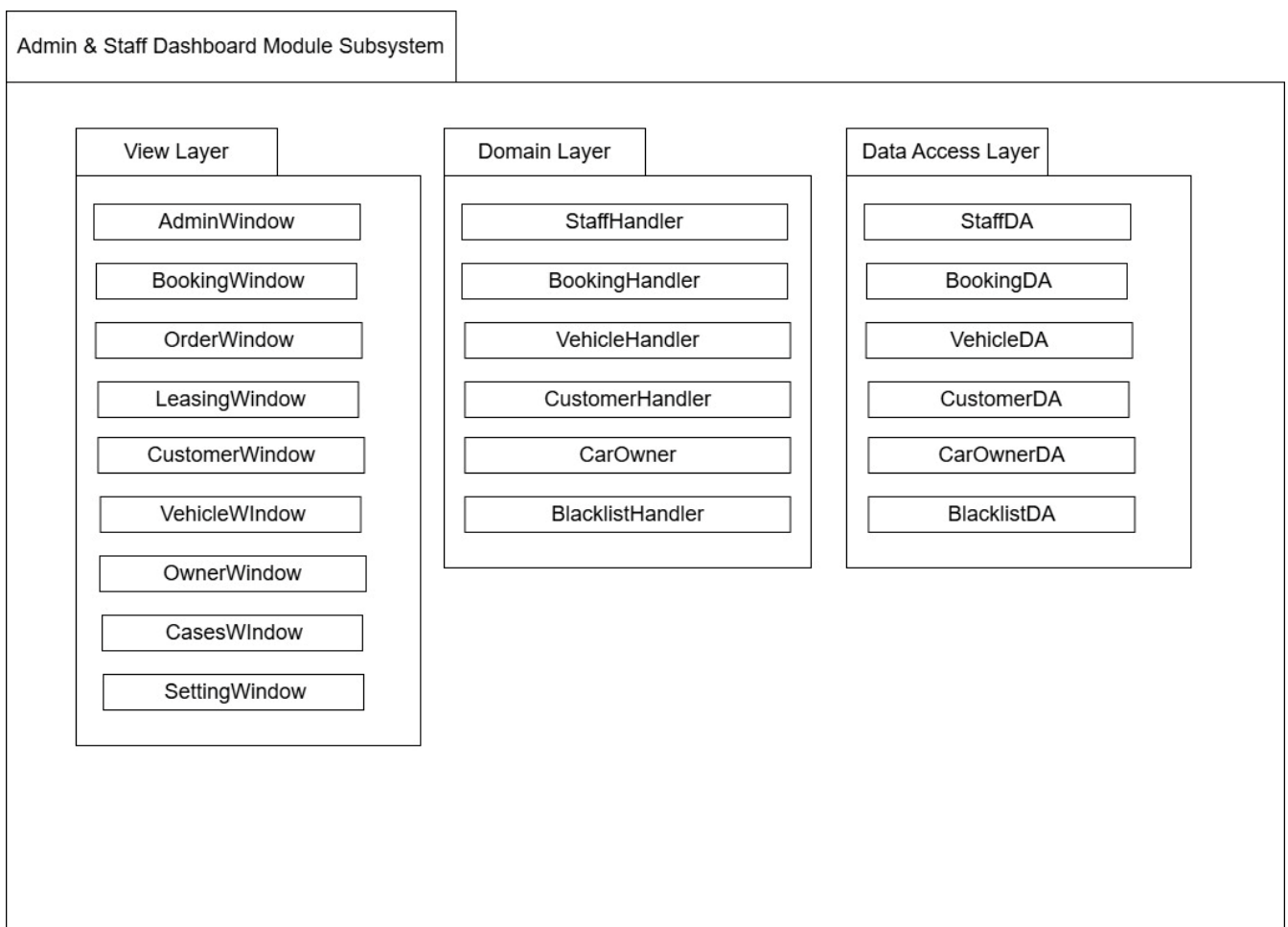


Figure 4.2.7: Package Diagram for <Admin and Staff Dashboard Module> Subsystem

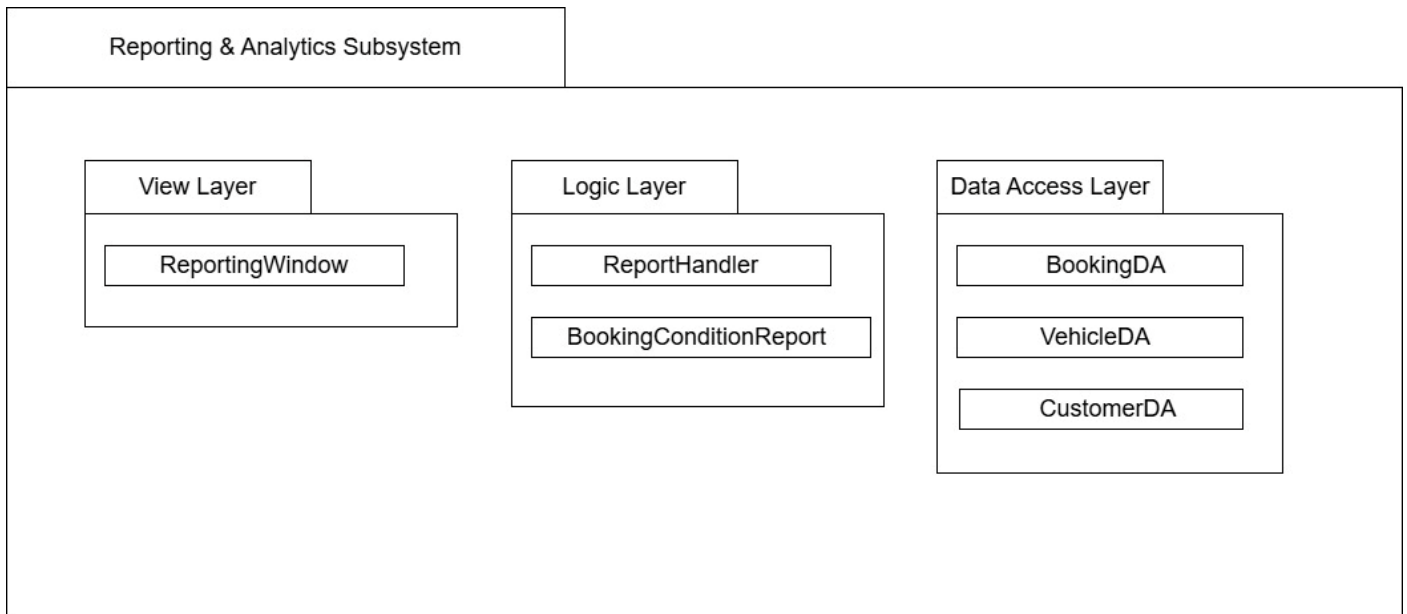


Figure 4.2.8: Package Diagram for <Reporting & Analytics> Subsystem

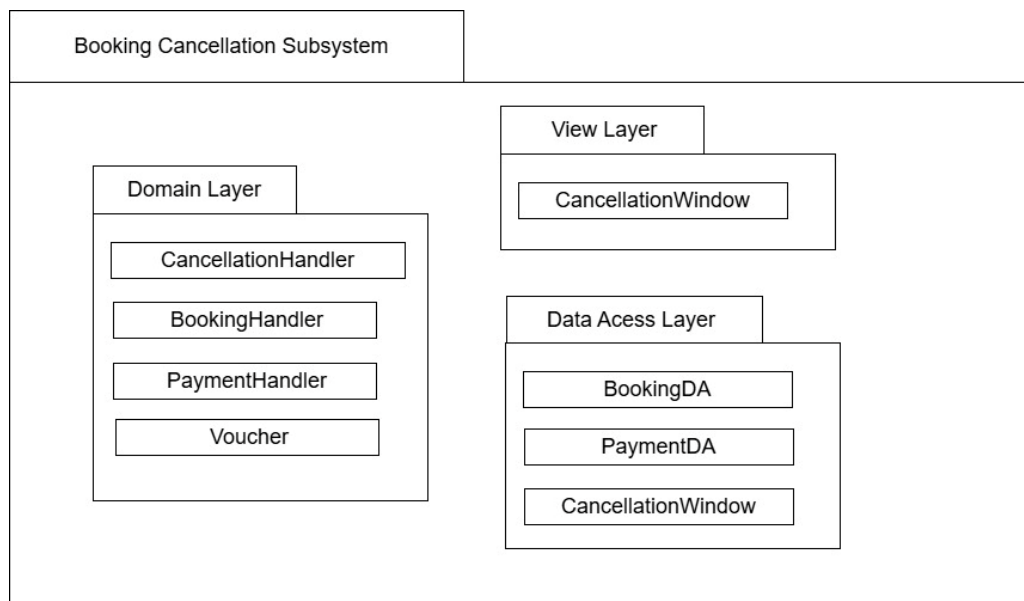


Figure 4.2.9: Package Diagram for <Booking Cancellation> Subsystem

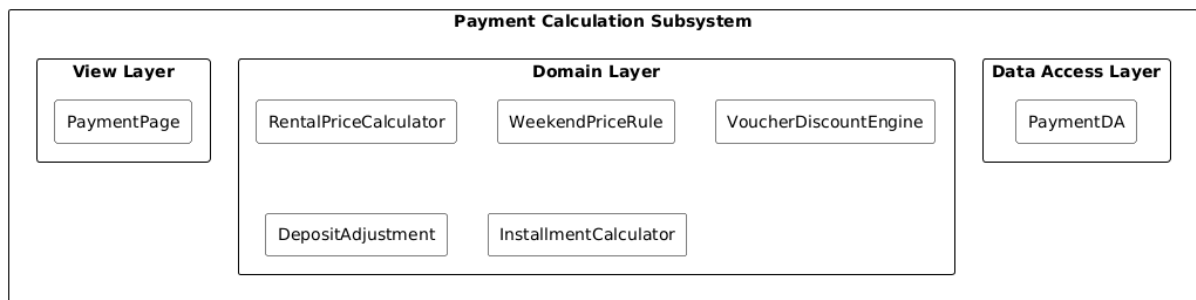


Figure 4.2.10: Package Diagram for <Payment Calculation> Subsystem

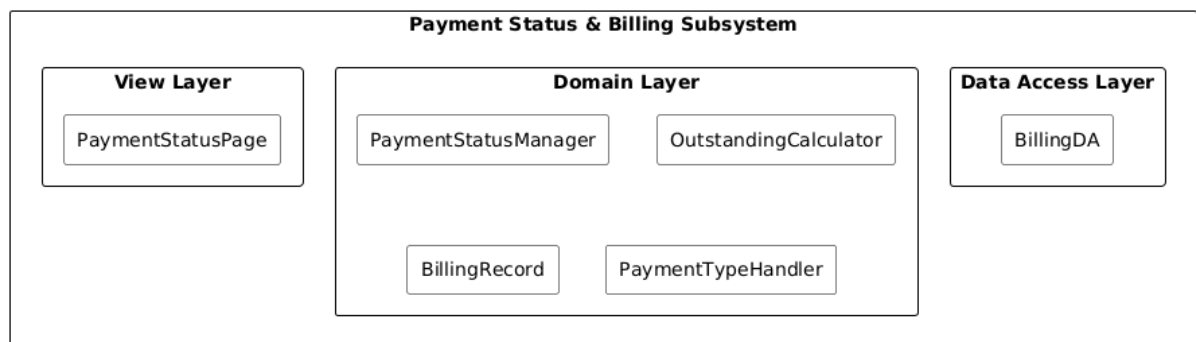


Figure 4.2.11: Package Diagram for <Payment Status & Billing> Subsystem

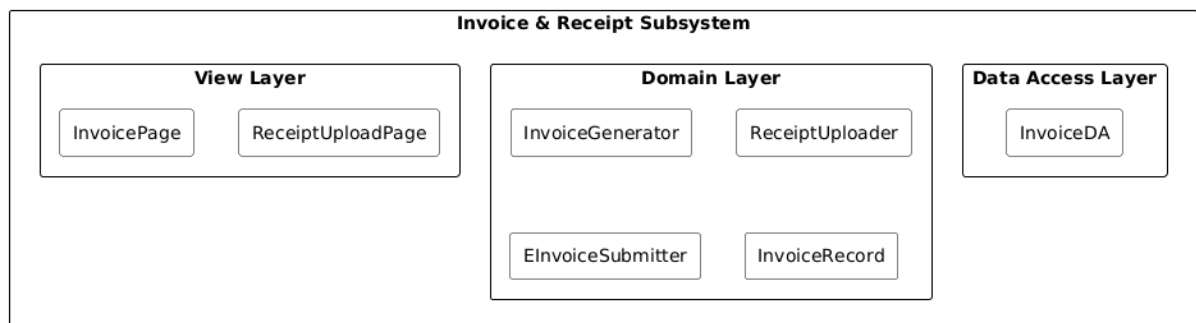


Figure 4.2.12: Package Diagram for <Invoice & Receipt> Subsystem

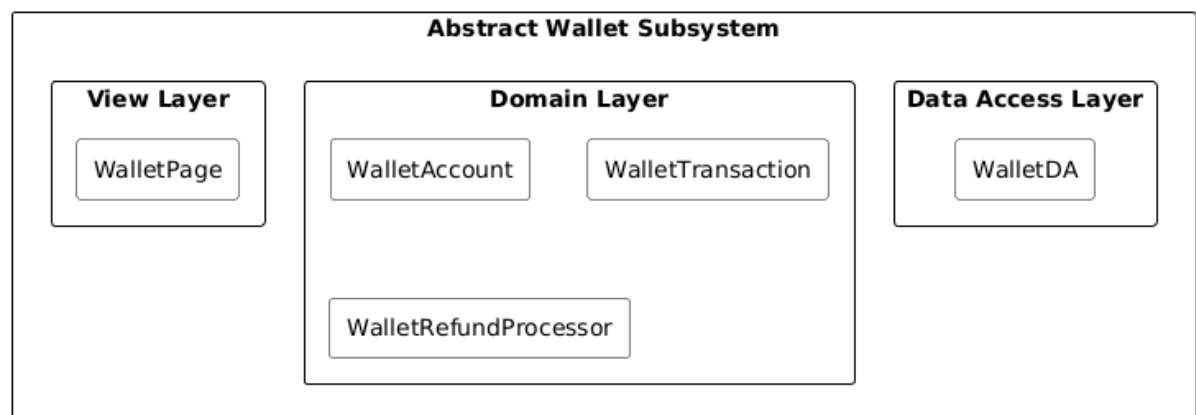


Figure 4.2.13: Package Diagram for <Abstract Wallet> Subsystem

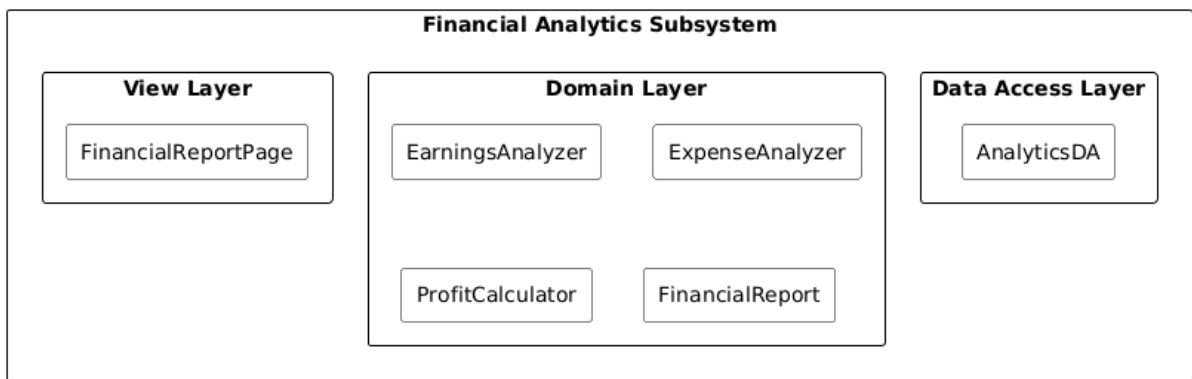


Figure 4.2.14: Package Diagram for < Financial Analytics> Subsystem

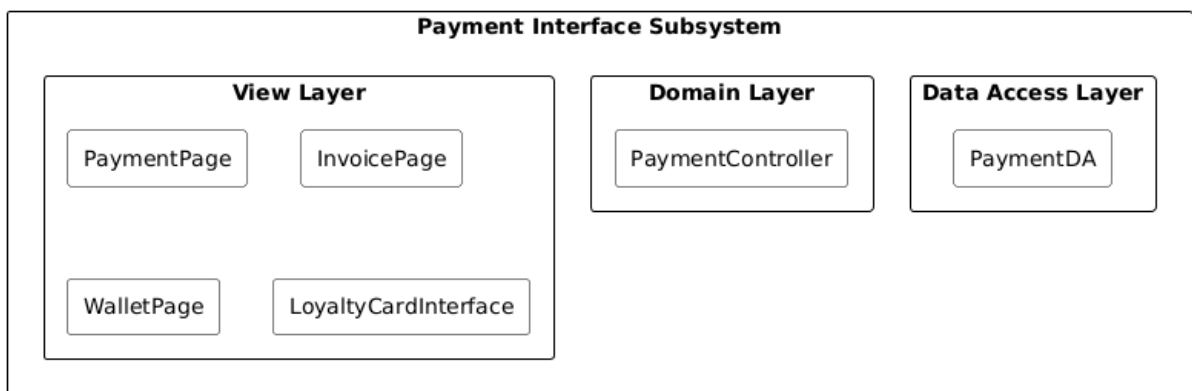


Figure 4.2.15: Package Diagram for < Payment Interface Subsystem > Subsystem

4.3 Logical View

This view presents the structural design of the Hasta Car Rental System from the perspective of the software designer. It focuses on the key abstractions of the system and shows how they are modeled as classes in an object-oriented design. The class diagrams illustrate the static structure of the system, including classes, their attributes, and the relationships between them.

Each subsystem is represented by one class diagram, which includes all relevant classes within that subsystem. These classes reflect the responsibilities identified during analysis and design, such as managing user information, and handling notification. The diagrams show how domain entities interact with supporting classes, allowing developers to clearly understand the organization and dependencies within each subsystem.

Together with the sequence diagrams, the class diagrams provide a complete view of both the system structure and behavior. This combined view helps ensure that the system design is clear, consistent, and aligned with the defined use cases, making it easier to implement, maintain, and extend the system in the future.

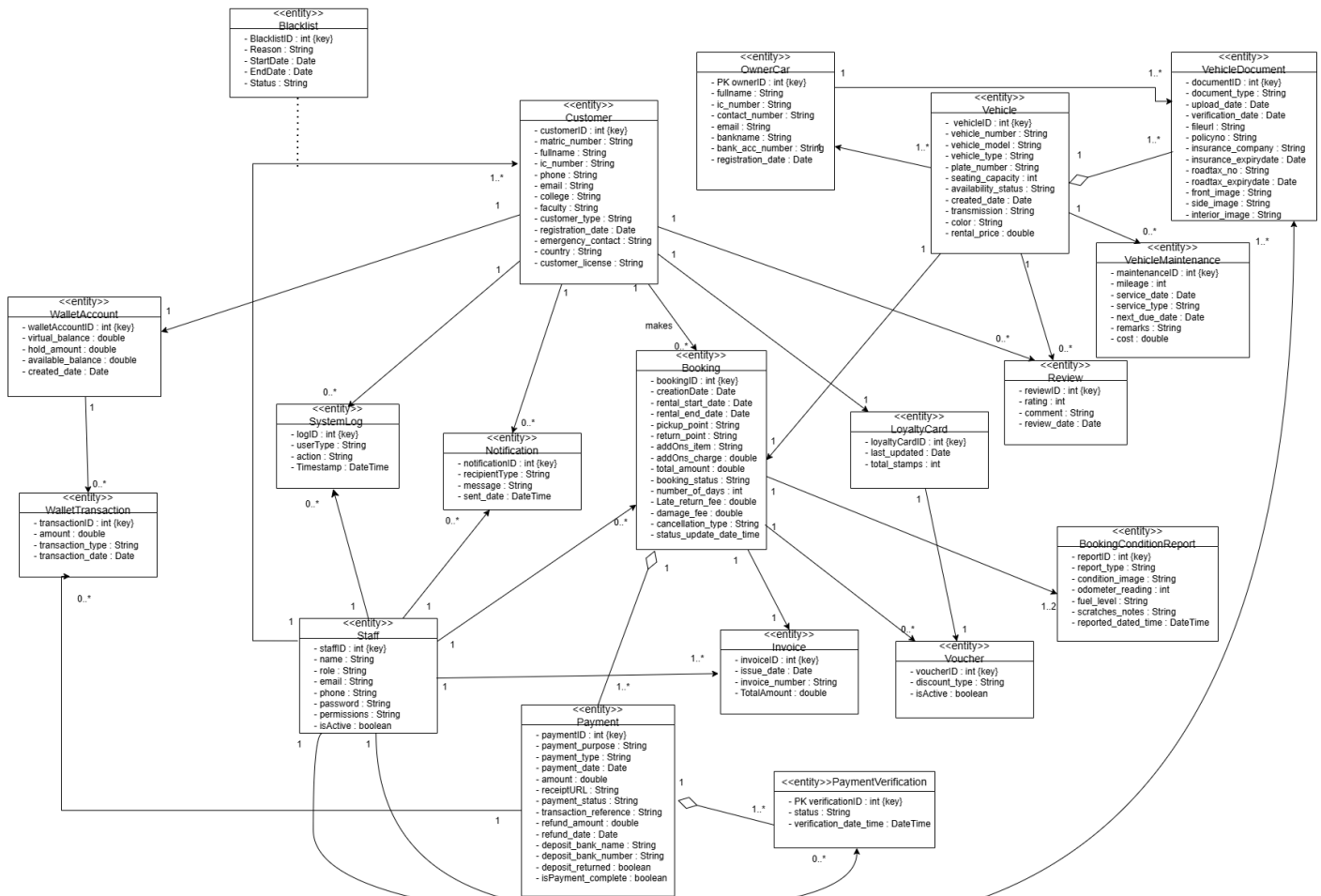


Figure 4.3: Class diagram for <Hasta Travel Car Rental System>

4.4 Process View

Process view describes the dynamic behavior of the system by showing how its components interact and execute during runtime. It focuses on the flow of control and data among processes across the view layer, controller, entity, and data access layer. Sequence diagrams are used to represent time-ordered interactions between system components, while activity diagrams describe the overall workflow and decision logic of the system.

Figure 4.4.1.1 is the sequence diagram for the Customer Management subsystem, which illustrates the detailed interactions between system components such as the user interface, controller, entity, and database during key user operations including registration, login, and viewing booking history. It shows how the system validates user inputs, verifies credentials, retrieves booking records, updates the database, and provides responses to the user based on the operation outcome. The activity diagram for the Customer Management subsystem in Figure 4.4.1.2 presents the internal workflow used to handle these user actions. It shows how system components process input validation, decision logic, database interactions, and alternative flows such as invalid login attempts or failed registration.

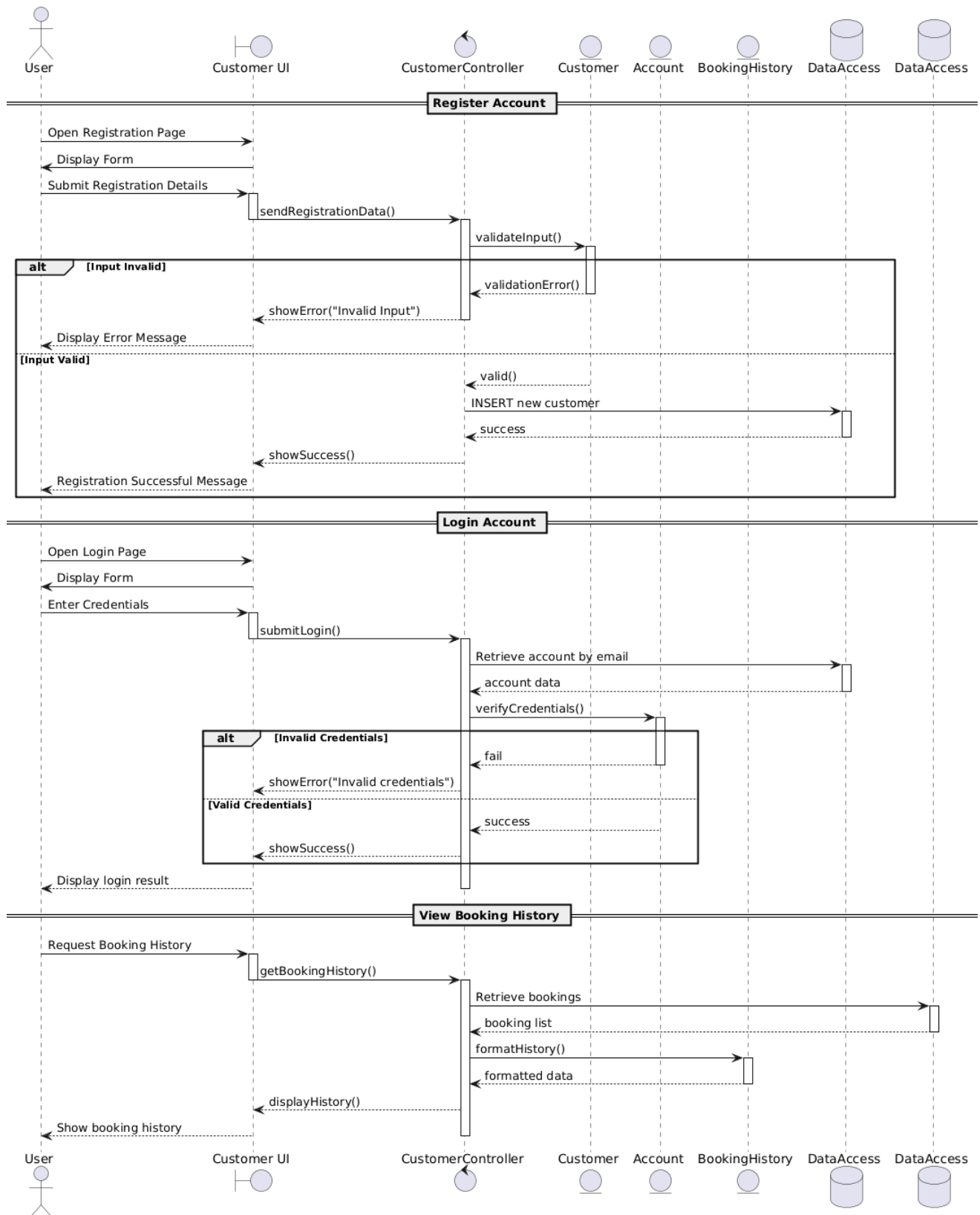


Figure 4.4.1.1: SD001 Sequence Diagram for <Customer Management Scenario>

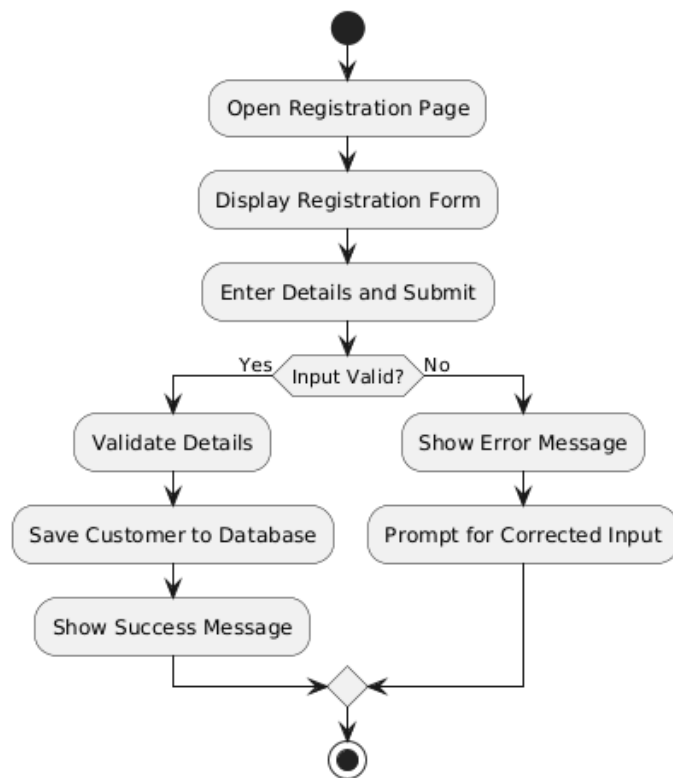


Figure 4.4.1.2: SD001 Activity Diagram for <Customer Management Scenario>

Figure 4.4.2.1 is the sequence diagram for the Create New Booking scenario, which illustrates the interactions between system components such as the user interface, controller, entity, and database during the booking process. It shows how the system validates booking details, checks vehicle availability, updates booking records in the database, and provides feedback to the user based on the booking outcome. The activity diagram for the Create New Booking scenario in Figure 4.4.2.2 presents the internal workflow used to process a new booking. It shows how system components handle validation, decision logic, database updates, and alternative flows such as booking rejection due to unavailable vehicles.

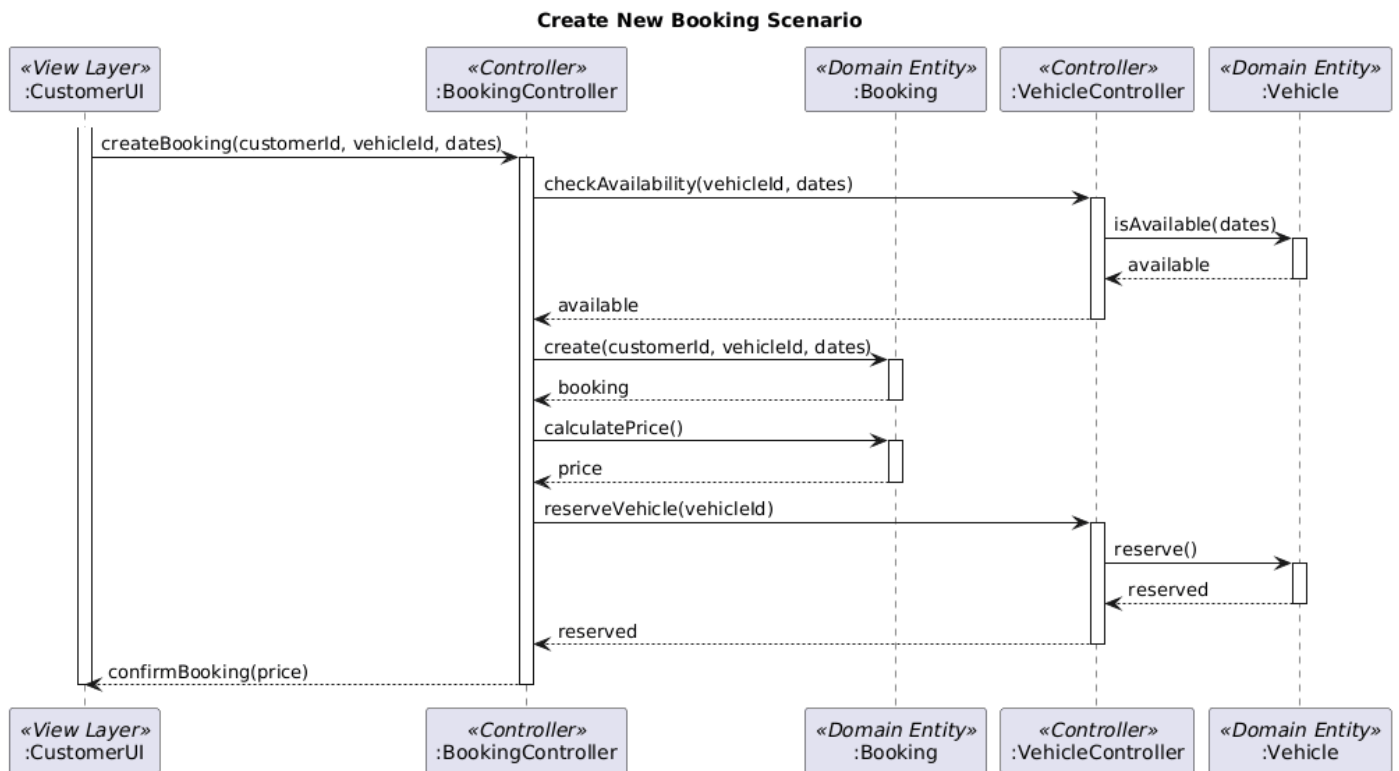


Figure 4.4.2.1: SD002 Sequence Diagram for <Create New Booking Scenario>

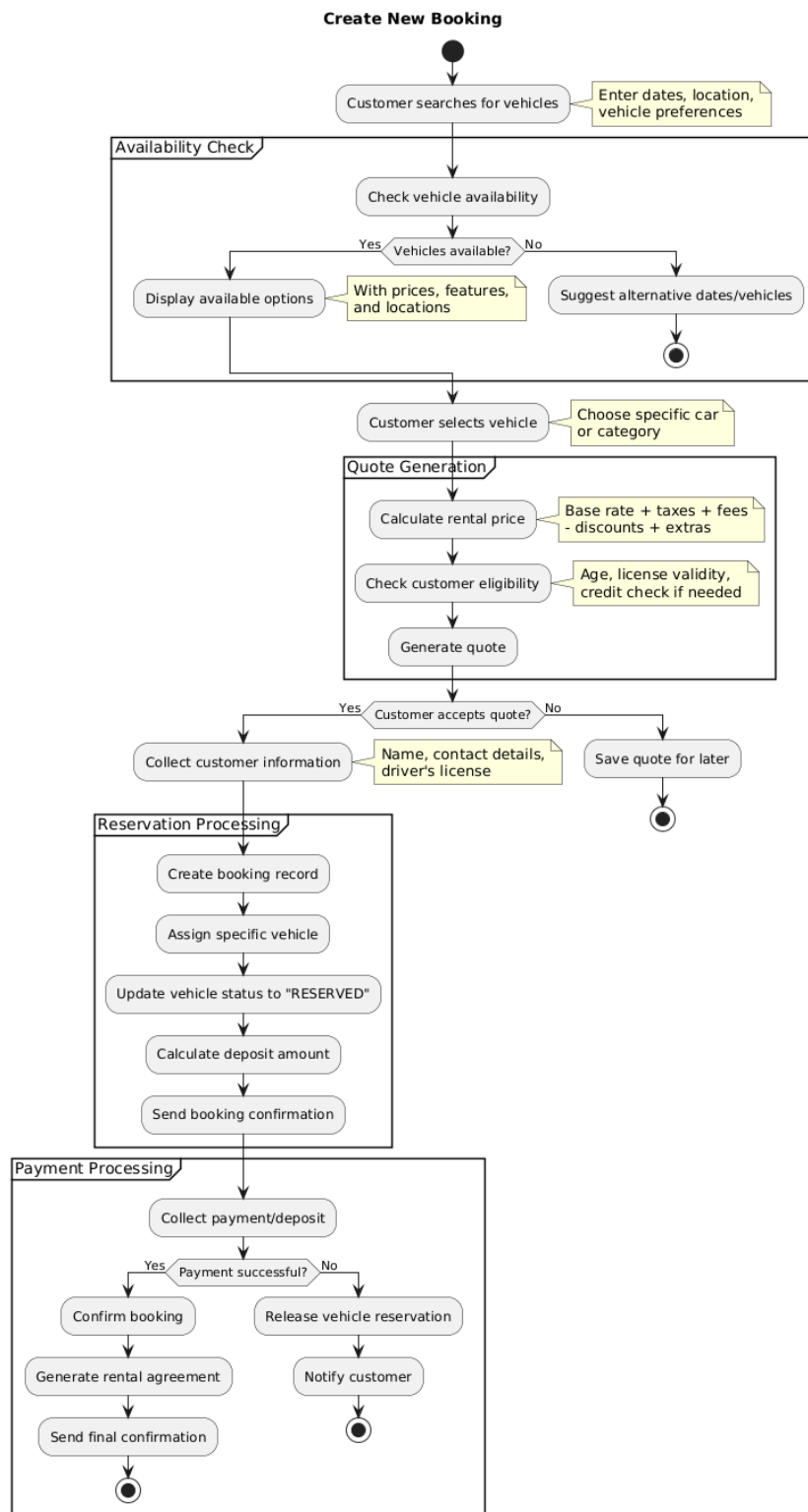


Figure 4.4.2.2: SD002 Activity Diagram for <Create New Booking Scenario>

Figure 4.4.3.1 is the sequence diagram for the Return Car scenario, which illustrates the detailed interactions between system components such as the user interface, controller, entity, and database during the car return process. It shows how the system records the return, updates booking status, calculates any additional charges if applicable, and provides feedback to the user. The activity diagram for the Return Car scenario in Figure 4.4.3.2 presents the internal workflow used to process car returns, including validation, decision logic, database updates, and alternative flows such as late return penalties or damage reports.

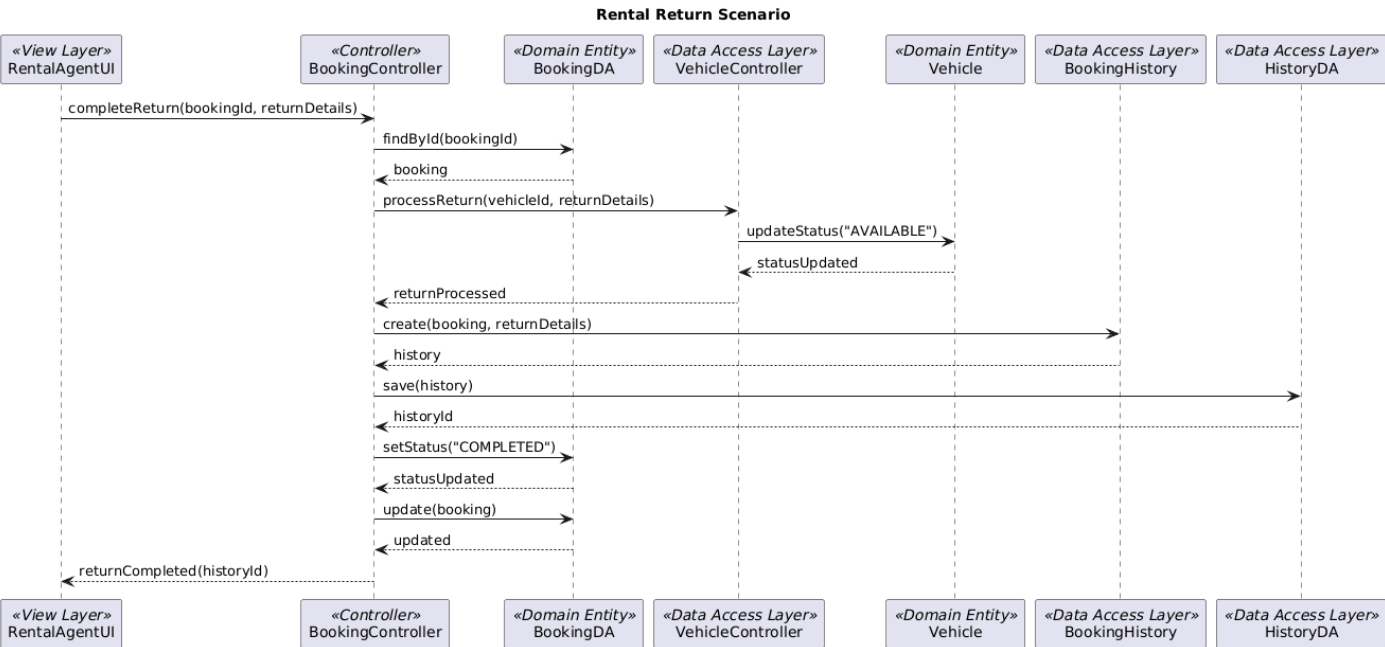


Figure 4.4.3.1: SD003 Sequence Diagram for <Return Car Scenario>

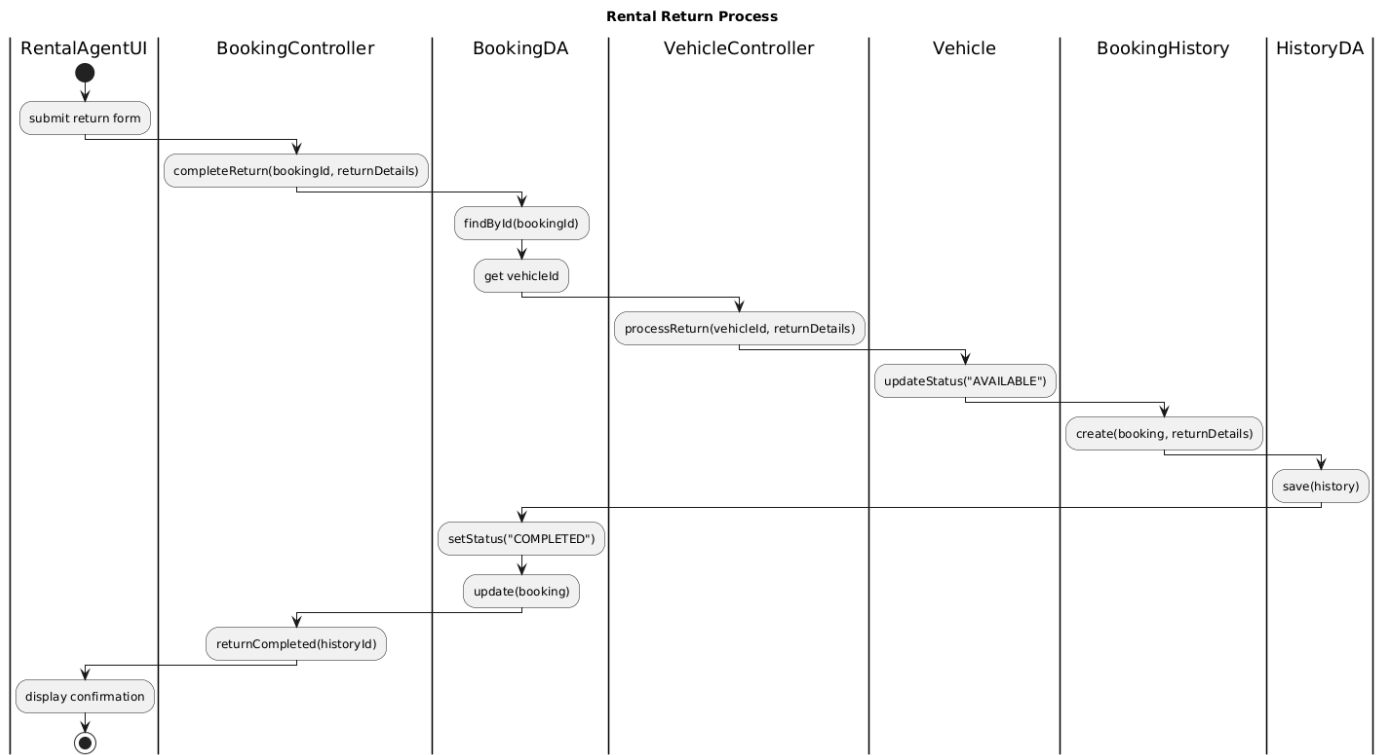


Figure 4.4.3.2: SD003 Activity Diagram for <Return Car Scenario>

Figure 4.4.4.1 is the sequence diagram for the Add New Vehicle scenario, which illustrates the interactions between system components such as the user interface, controller, entity, and database when adding a new vehicle to the system. It shows how the system validates vehicle details, updates the vehicle database, and provides confirmation to the user or staff. The activity diagram for the Add New Vehicle scenario in Figure 4.4.4.2 presents the internal workflow used to manage new vehicle entries, including validation, decision logic, database updates, and alternative flows such as input errors or duplicate entries.

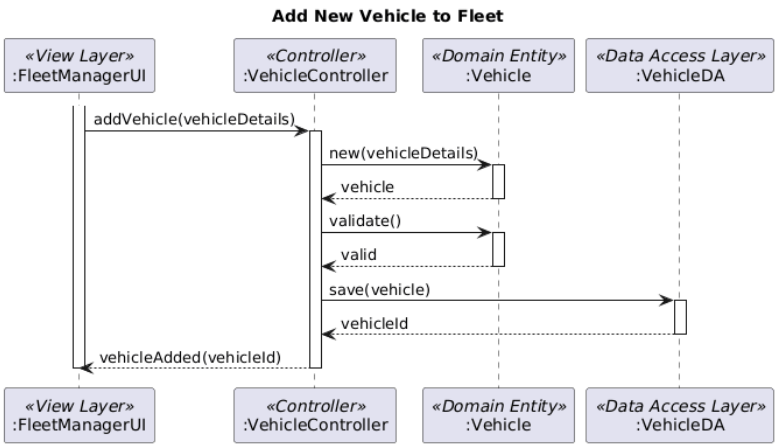


Figure 4.4.4.1: SD004 Sequence Diagram for <Add New Vehicle Scenario>

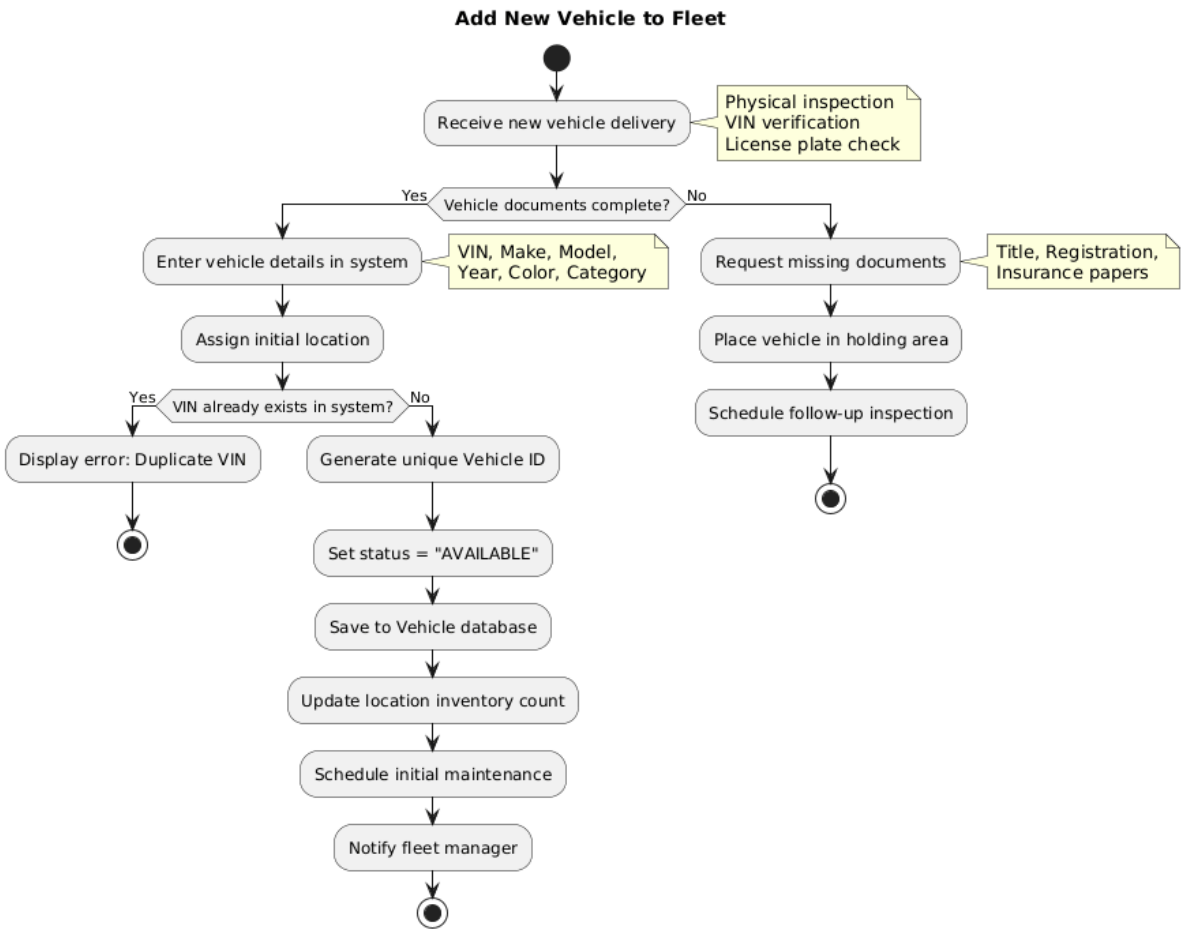


Figure 4.4.4.2: SD004 Activity Diagram for <Add New Vehicle Scenario>

Figure 4.4.5.1 is the sequence diagram for the Cancel Order scenario which illustrates the detailed interactions between system components such as the user interface, controller, service and database during the cancellation process. It shows how the system validates the request, updates the booking status and triggers notifications based on the cancellation outcome. The activity diagram for the Cancel Order scenario in Figure 4.4.5.2 presents the internal system workflow used to process a cancellation request. It shows how system components handle validation, decision logic, database updates and alternative flows such as cancellation rejection.

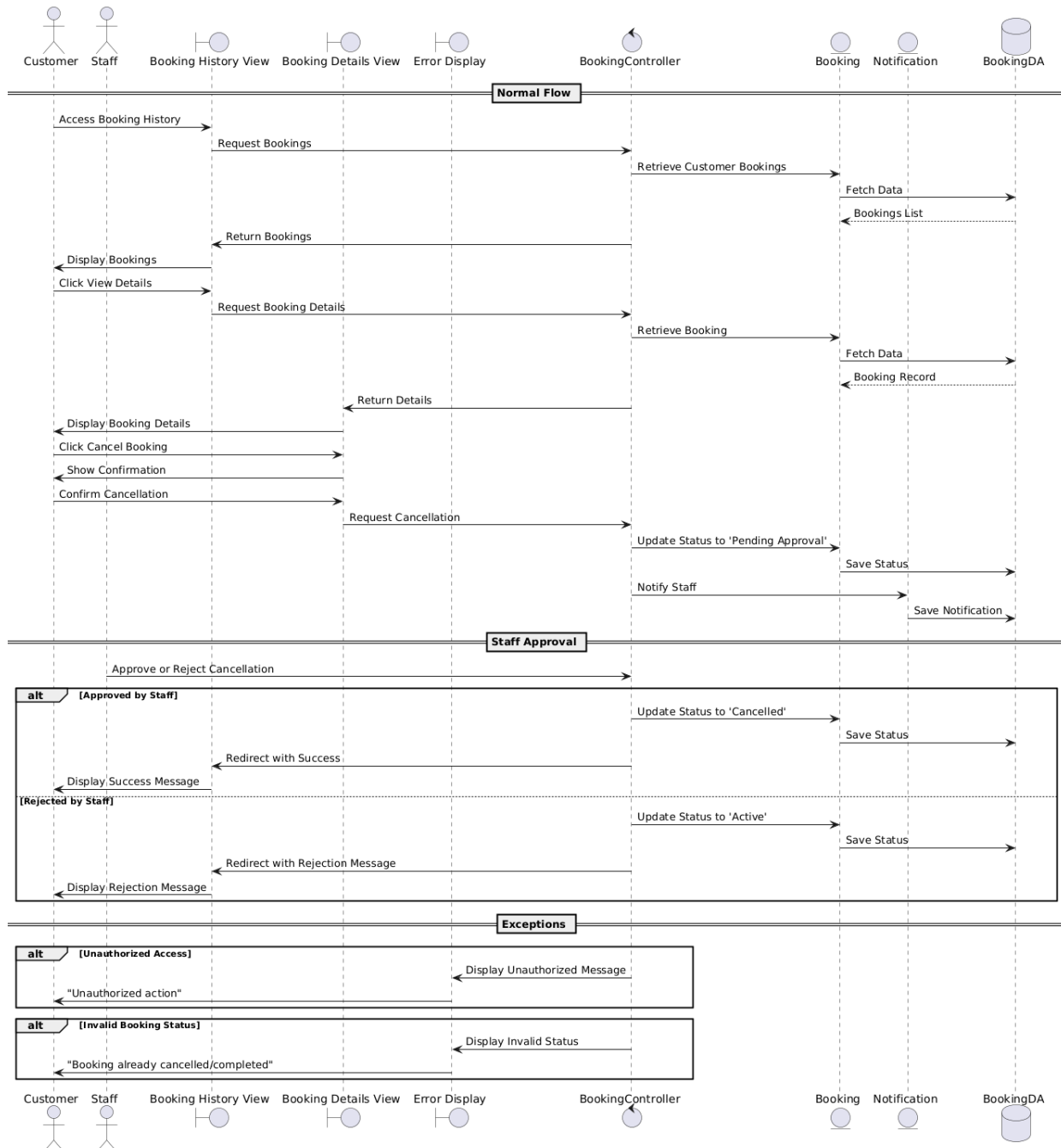


Figure 4.4.5.1: SD005 Sequence Diagram for <Cancel Order scenario>

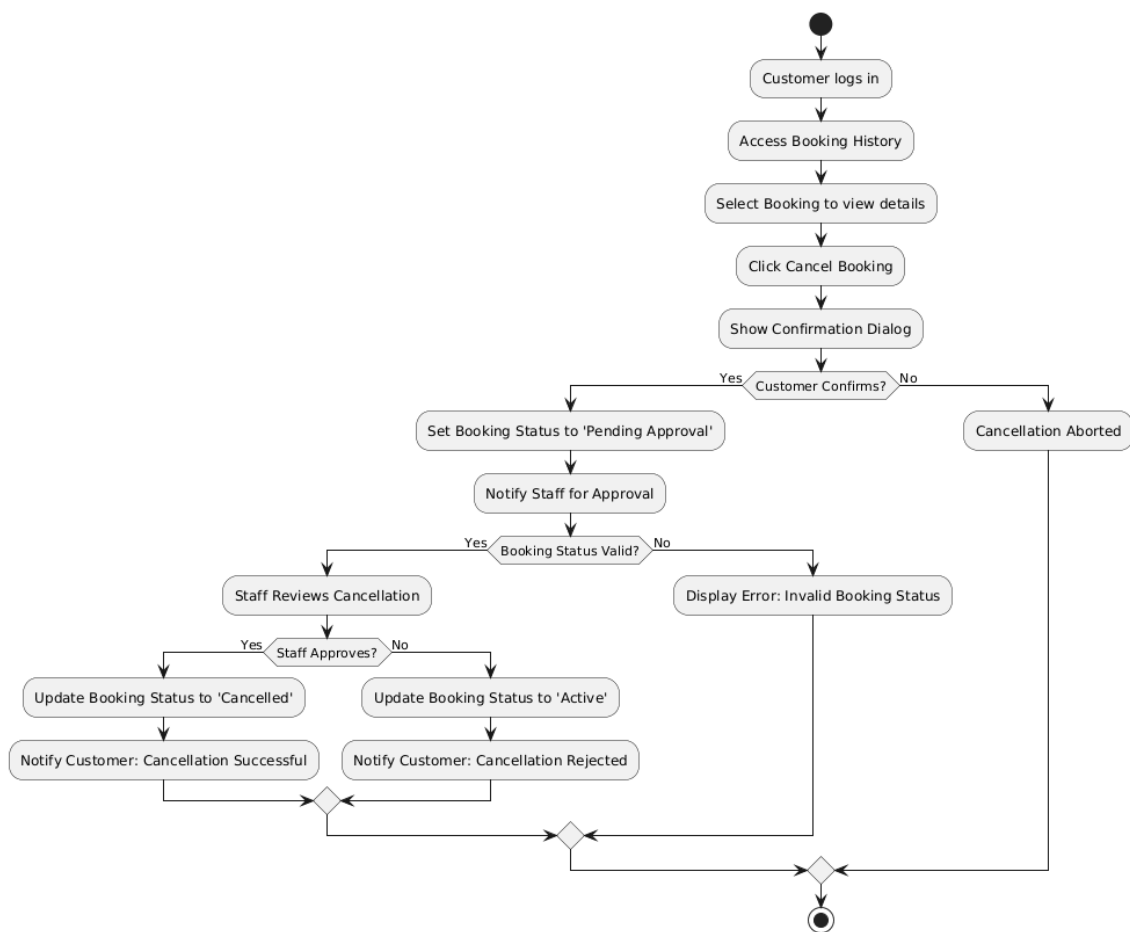


Figure 4.4.5.2: SD005 Analysis Diagram for <Cancel Order scenario>

Figure 4.4.6.1 shows the sequence diagram for the Generate Report scenario which describes the detailed interaction between the reporting user interface, report manager, report component, and database during report generation. It illustrates how the system processes the request, retrieves data, generates the report, and handles export operations. Figure 4.4.6.2 is the activity diagram for the Generate Report scenario showing the processing logic, decision points, and system actions such as data validation, report creation, and export handling.

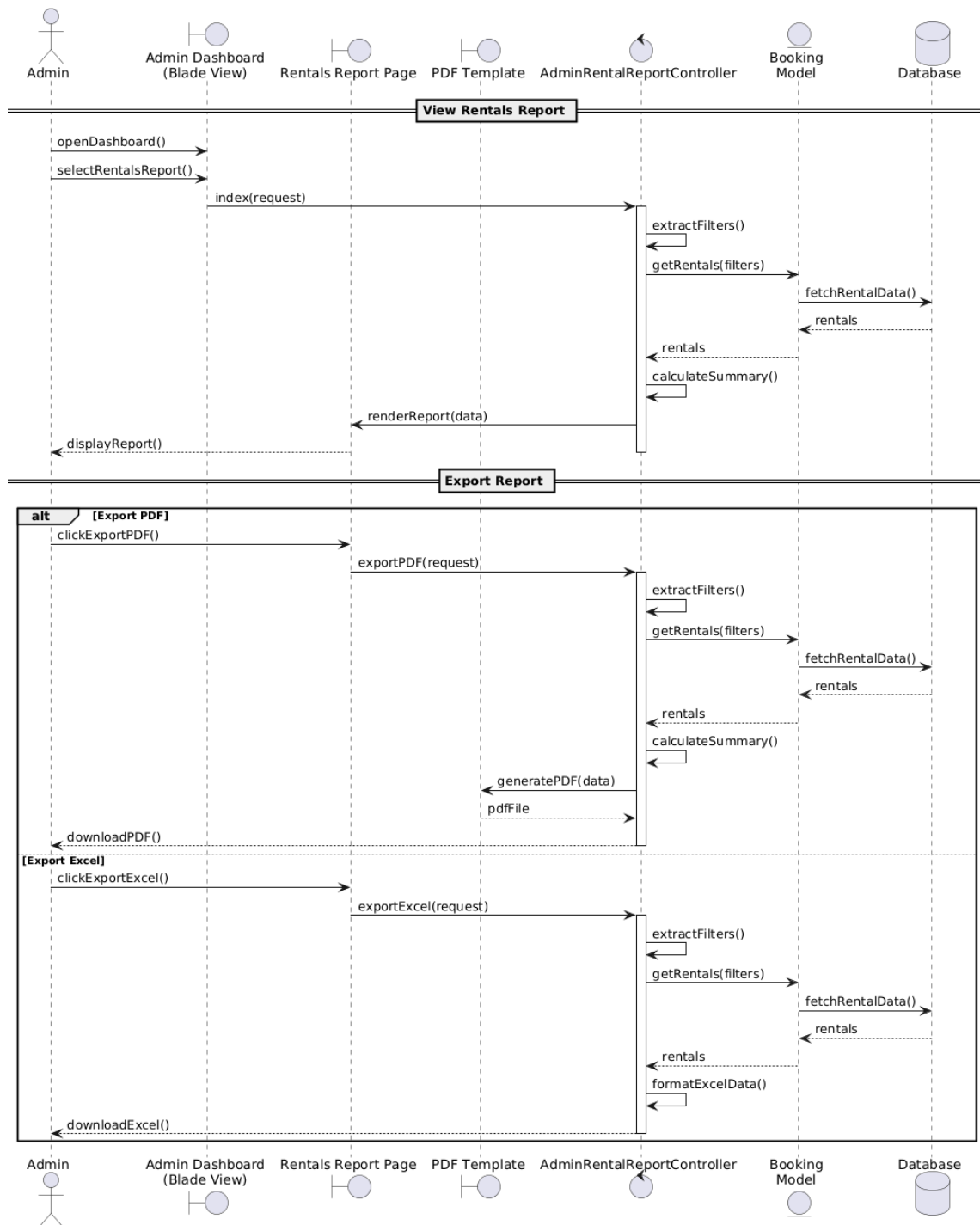


Figure 4.4.6.1: SD006 Sequence Diagram for <Generate Rental Report scenario>

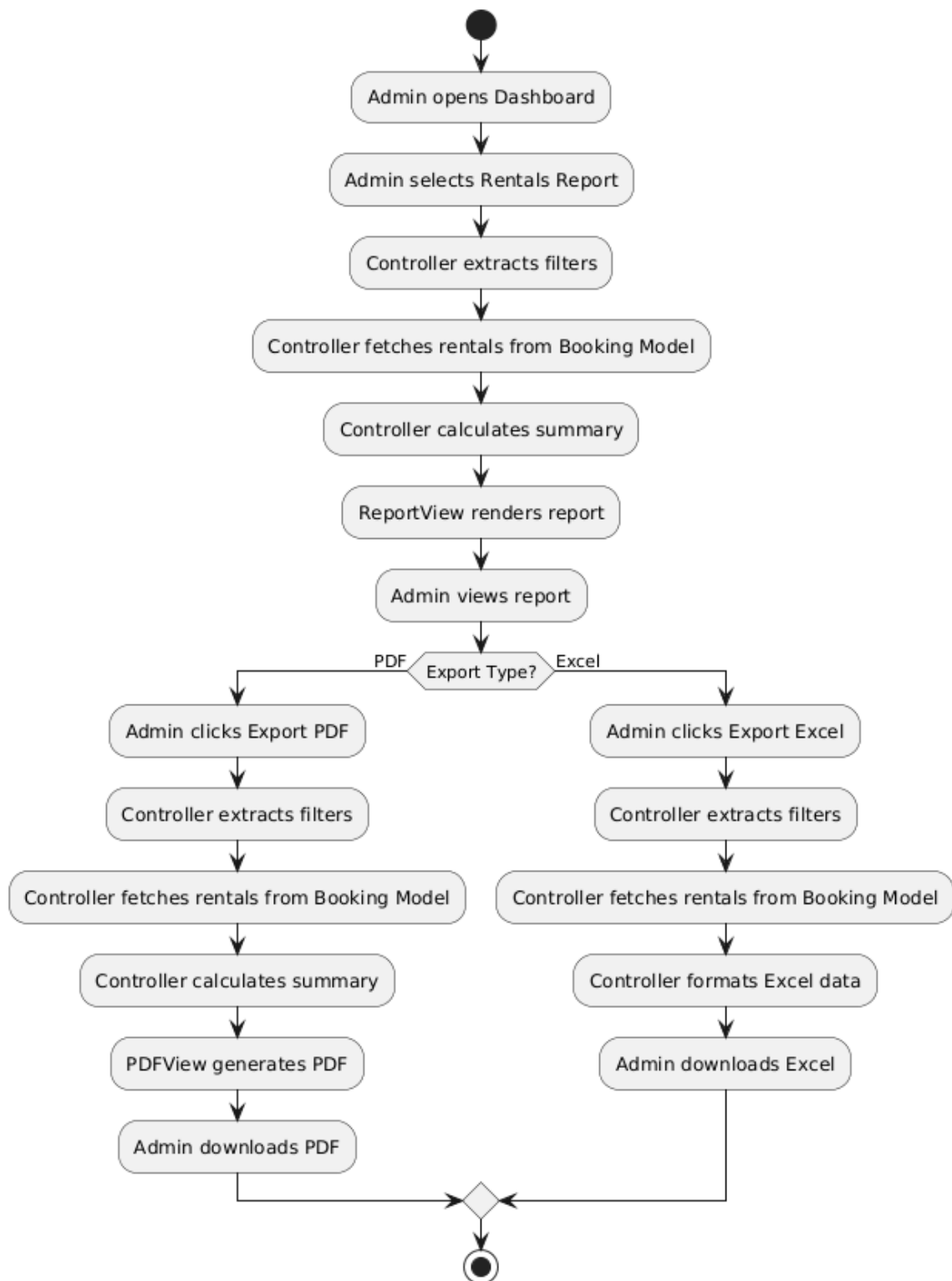


Figure 4.4.6.2: SD006 Activity Diagram for <Generate Rental Report scenario>

Figure 4.4.7.1 shows the sequence diagram for the *Payment* scenario, which describes the detailed interaction between the payment user interface, the Payment Controller, and the payment database when a user initiates a payment. It illustrates how the system captures the payment details, creates a payment record, and returns the payment result to the user. Figure 4.4.7.2 is also the activity diagram for the *Payment* scenario, showing the overall processing logic, the decision point for payment success or failure, and system actions such as updating the booking status, storing the transaction outcome, and displaying the corresponding success or failure message.

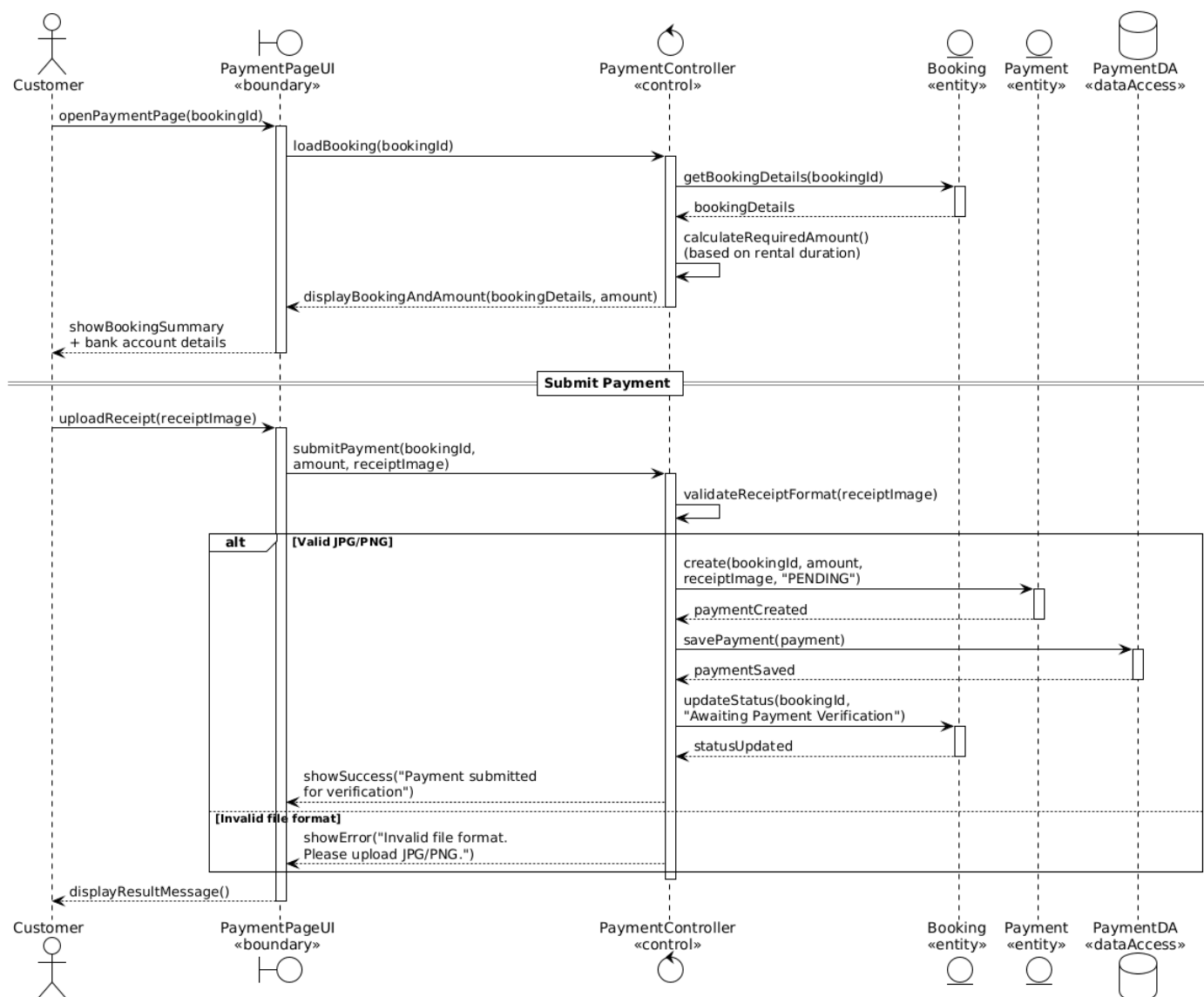


Figure 4.4.7.1: SD007 Sequence Diagram for <Payment scenario>

Activity Diagram - Payment

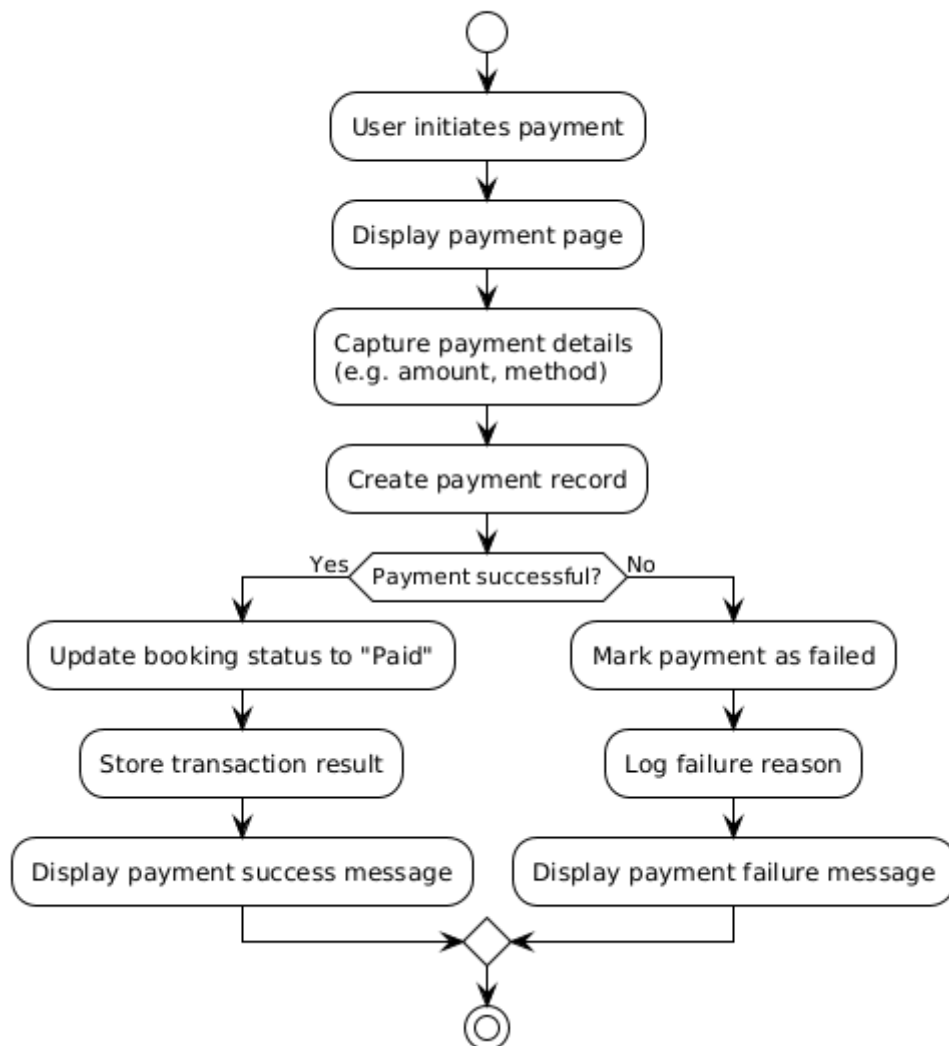


Figure 4.4.7.2: AD007 Activity Diagram for <Payment scenario>

Figure 4.4.8.1 shows the sequence diagram for the *Generate Booking Invoice* scenario, which describes the interaction between the invoice user interface, the Invoice Generator component, the Booking entity, and the invoice database during invoice generation. It illustrates how the system retrieves booking details, calculates the relevant charges, creates the invoice object, and persists it before returning the invoice for user viewing or download. Figure 4.4.8.2 is also the activity diagram for the *Generate Booking Invoice* scenario, showing the processing logic, including retrieving booking information, computing the charges, generating invoice data, saving it to the database, and finally presenting the invoice to the user.

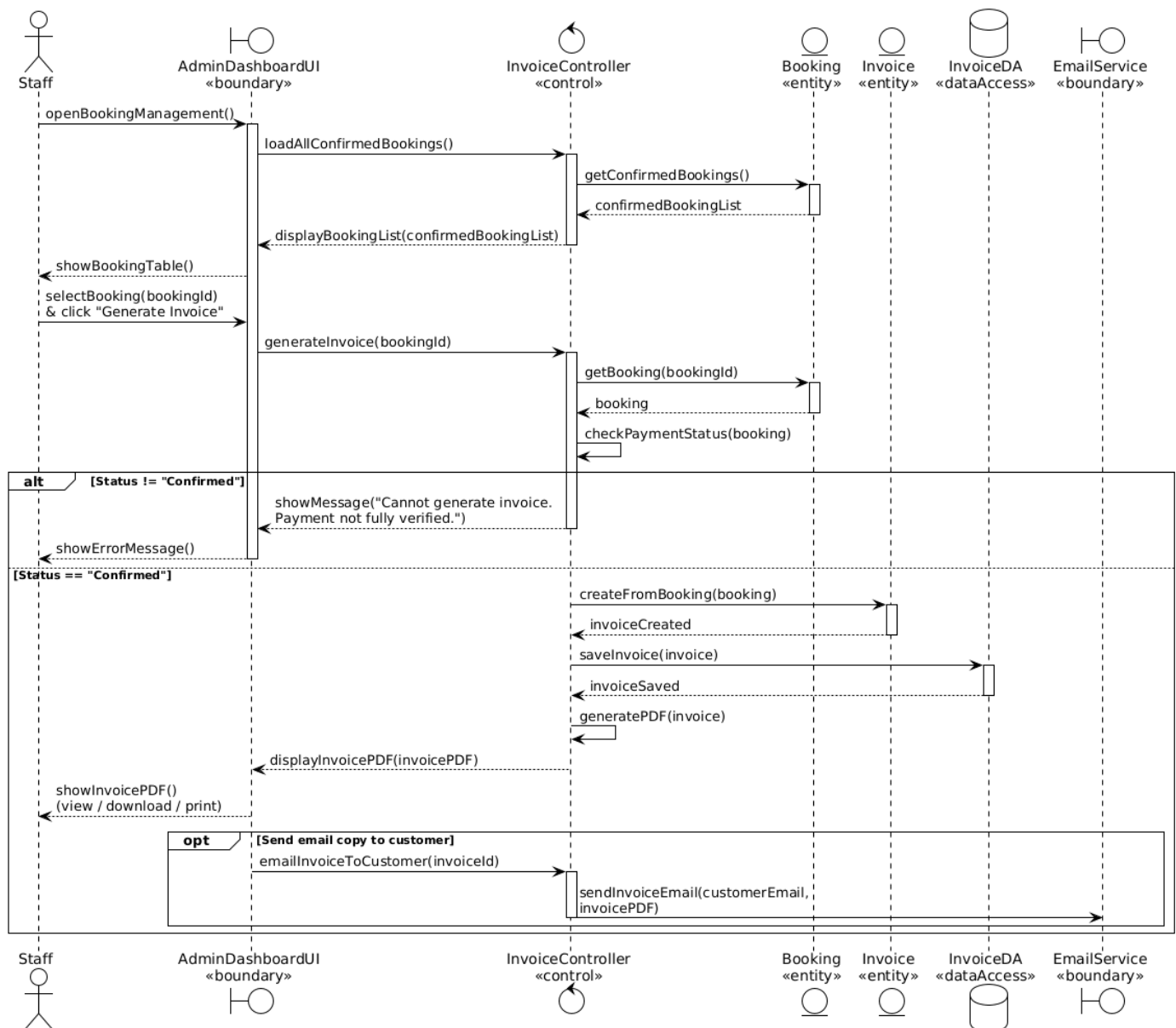


Figure 4.4.8.1: SD008 Sequence Diagram for <Generate Booking invoice scenario>

Activity Diagram - Generate Booking Invoice

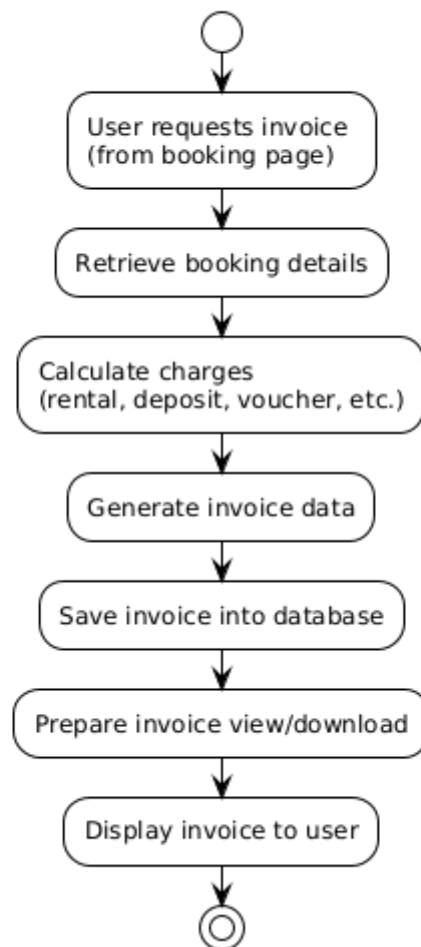


Figure 4.4.8.2: AD008 Activity Diagram for <Generate Booking Invoice scenario>

Figure 4.4.9.1 shows the sequence diagram for the *Verify Payment* scenario, which describes the interaction between the verification interface, the Payment Verifier component, and the payment database when a user or staff member requests payment verification. It illustrates how the system retrieves the payment record, validates the transaction details, and returns the verification result to the interface. Figure 4.4.9.2 is also the activity diagram for the *Verify Payment* scenario, showing the processing logic, the validation step, the decision point for valid or invalid payment, and system actions such as updating the payment status, recording any validation issues, and displaying the verification outcome to the user.

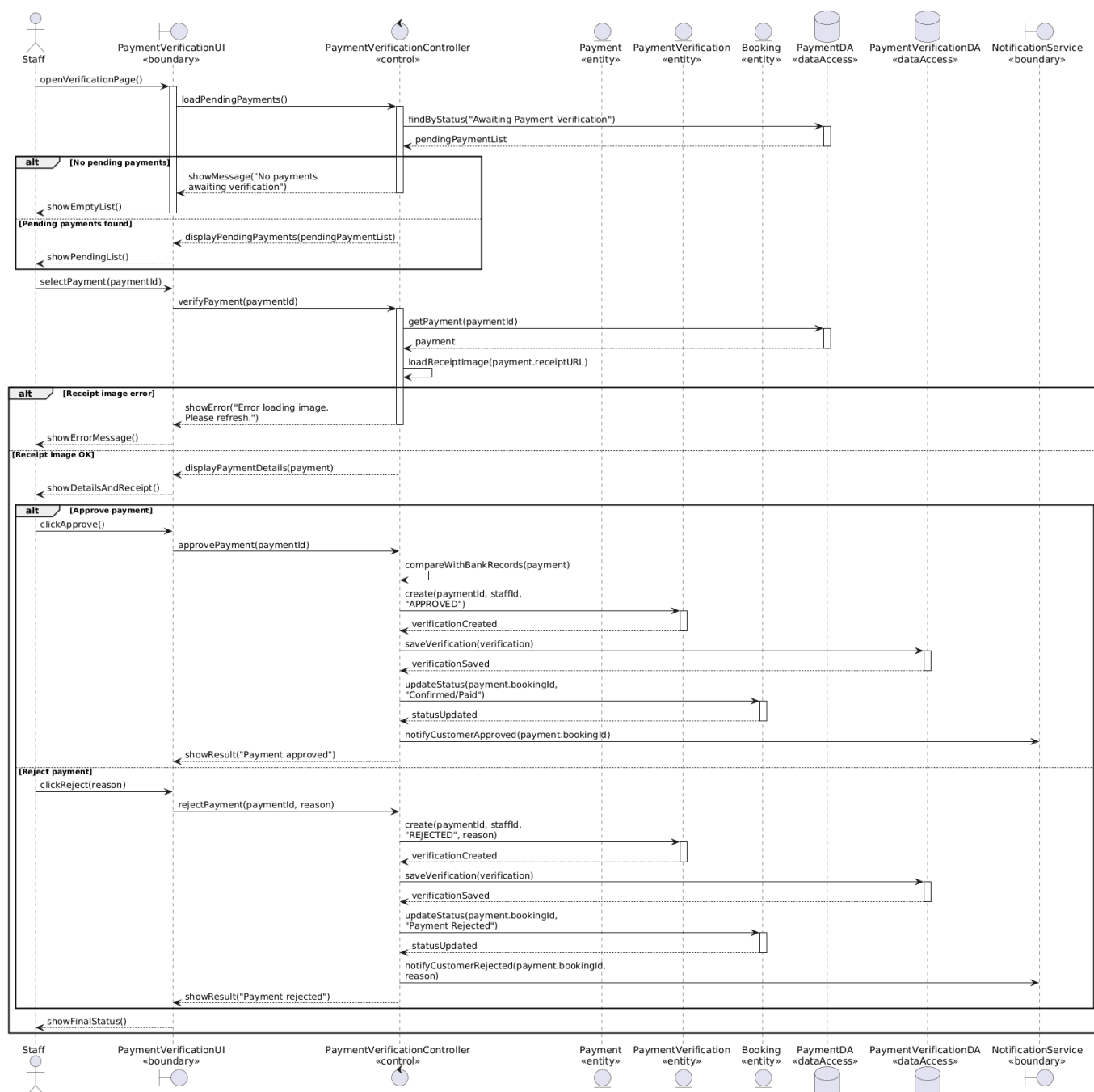


Figure 4.4.9.1: SD009 Sequence Diagram for <Verify payment scenario>

Activity Diagram - Verify Payment

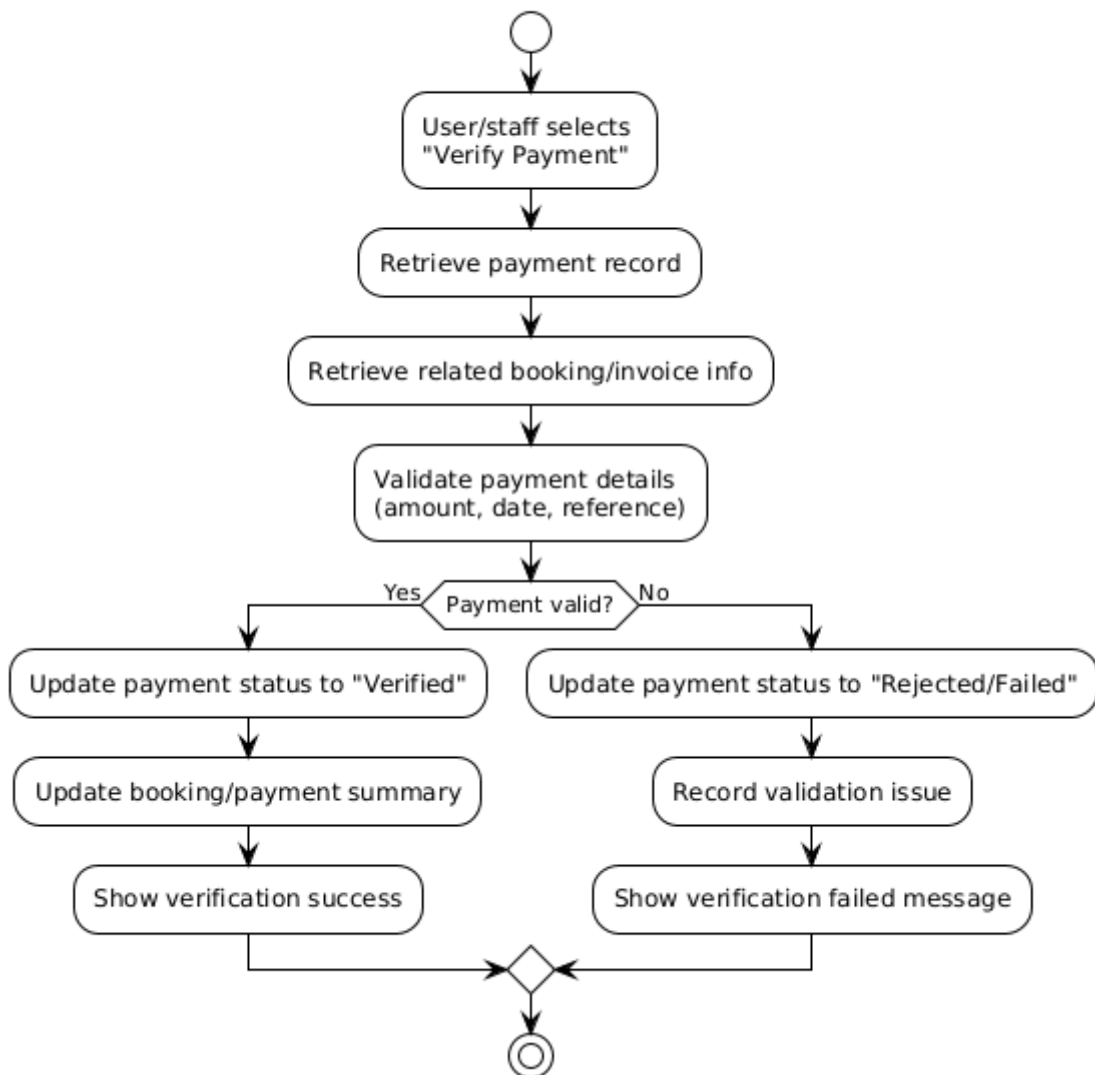


Figure 4.4.9.2: AD009 Activity Diagram for <Verify payment scenario>

4.5 Physical/ Deployment View

Figure 4.10 below illustrates the deployment view of the Hasta Travel system, showing how software components are distributed across hardware infrastructure and how they communicate with each other.

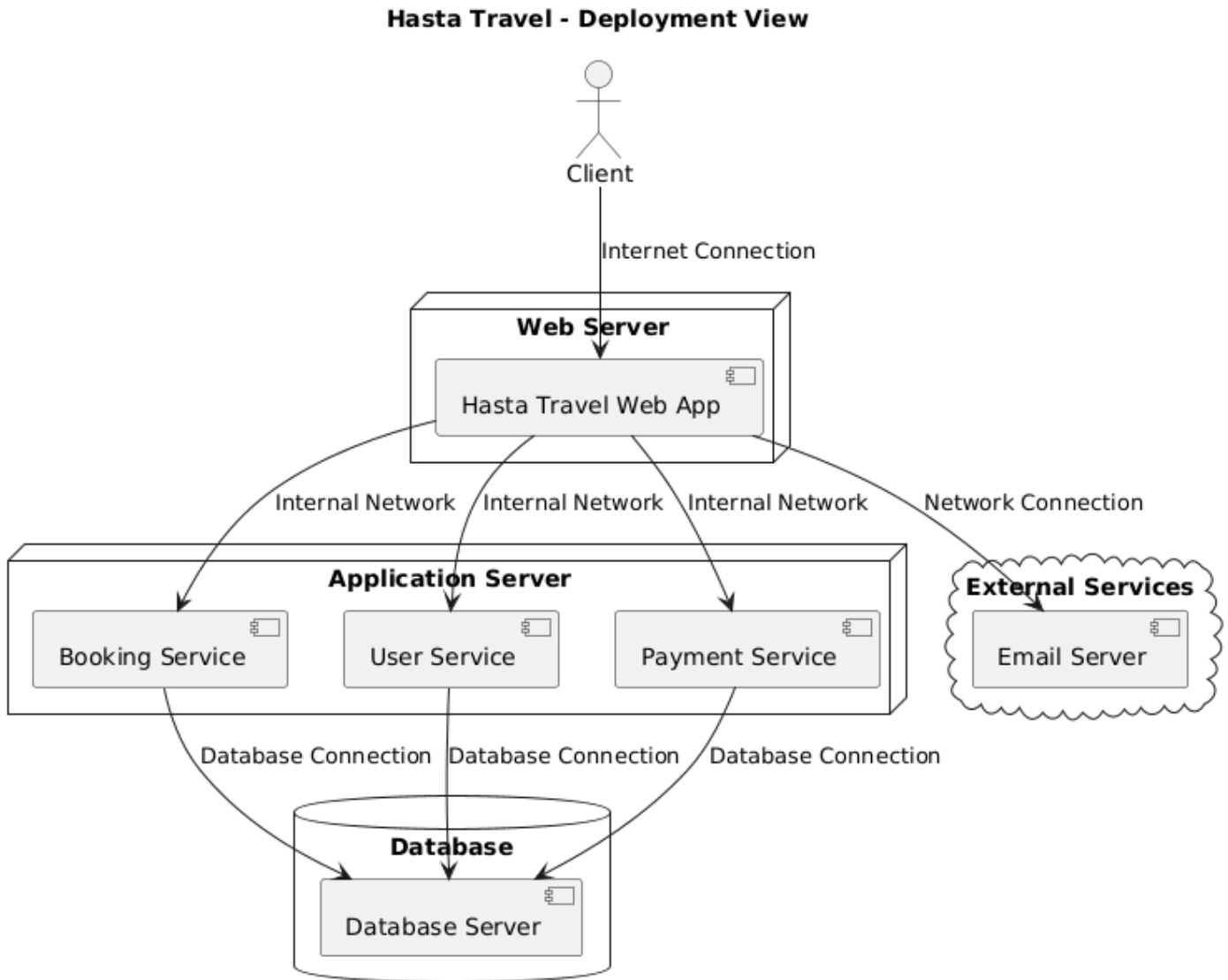


Figure 4.0: Deployment diagram for Internet-based System

5. Data Design

The information domain of the Hasta Car Rental System is transformed into structured data that represents the core entities identified during requirements and domain analysis. These entities are modeled as classes in the object-oriented design and are stored as tables in a single centralized database. Each entity stores specific information required to support system operations such as account management and booking.

The system uses one main database, which contains all tables required for the application. Data is processed through the domain layer, where business rules are applied, and accessed through the data access layer, which handles storage and retrieval. Relationships between entities are maintained using identifiers to ensure data consistency and integrity. This design allows the system to efficiently store, retrieve, and manage information while supporting future system expansion.

Table 5.1: Description of Entities in the Database

No.	Entity Name	Description
1	Admin	Stores administrator records linked to user accounts for system management and oversight.
2	Booking	Represents a rental agreement between a customer and the system, including rental period, charges, deposit, and booking status.
3	Car	Stores car-specific attributes for vehicles classified as cars, such as seating capacity, transmission type, and car category.
4	Car_img	Stores image information related to cars, including image type and descriptive details.
5	Customer	Stores registered customer profiles and serves as the central entity for bookings, payments, wallets, and loyalty programs.
6	International	Stores identification details specific to international customers, such as passport information and country of origin.
7	InternationalStudent	Associates international customers with student academic information.
8	International_UTMStaff	Identifies international customers who are UTM staff members.
9	Invoice	Stores official invoices generated after booking confirmation and payment completion.
10	Local	Stores identification details specific to local customers, such as identity card information and state of origin.
11	LocalStudent	Associates local customers with student academic information.
12	Local_UTMStaff	Identifies local customers who are UTM staff members.
13	LoyaltyCard	Tracks customer participation in the loyalty program, including accumulated stamps and update history.
14	Motorcycle	Stores motorcycle-specific attributes for vehicles classified as motorcycles.

15	OwnerCar	Stores information about third-party vehicle owners, including contact and banking details for leasing and payouts.
16	Payment	Records payment transactions made for bookings, including amount, status, reference number, and proof of payment.
17	persondetails	Stores personal identity information shared by staff, administrators, or owners, such as full name and identity card number.
18	runner	Stores runner-specific information, including commission earned for handling assigned tasks or deliveries.
19	staff	Stores staff records linked to system users for operational, administrative, and verification tasks.
20	staffit	Stores IT staff-specific information such as salary details.
21	user	Stores authentication and profile information for all system users, including customers, staff, and administrators.
22	vehicle	Stores general vehicle information such as plate number, brand, model, rental price, and availability status.
23	vehicleconditionform	Records the physical condition of a vehicle during pickup and return, including mileage, fuel level, and damage notes.
24	vehicleconditionimage	Stores images related to vehicle condition reports for verification and evidence purposes.
25	vehicledocument	Stores metadata and file references for legal and regulatory vehicle documents.
26	vehiclemaintenance	Records vehicle maintenance and servicing history, including service type, cost, and next service date.
27	voucher	Stores discount vouchers that can be applied to bookings.
28	walletaccount	Stores virtual wallet accounts for customers, including balances and outstanding amounts.

5.1 Data Dictionary

This section provides a listing of the major system of entities in the Hasta Car Rental System. Each entity represents a domain-level class identified in the domain model. The table describes the purpose of each class and its key attributes. These classes are used to store and manage system information and are implemented as tables in the database.

5.1.1 Entity: < Admin >

Attribute Name	Type	Description
adminID	UUID	Admin identifier. PK
ic_no	VARCHAR	Reference to PersonDetails
userID	INT	Reference to User. FK

5.1.2 Entity: < Booking >

Attribute Name	Type	Description
bookingID	UUID	Unique booking identifier. PK
rental_start_date	DATETIME	Rental start date
rental_end_date	DATETIME	Rental end date
duration	INT	Rental duration (days)
pickup_point	VARCHAR	Pickup location
return_point	VARCHAR	Return location
pickup_surcharge	DECIMAL	Pickup surcharge
return_surcharge	DECIMAL	Return surcharge
pickup_custom_location	VARCHAR	Custom pickup location
return_custom_location	VARCHAR	Custom return location
deposit_amount	DECIMAL	Deposit amount
rental_amount	DECIMAL	Rental amount
booking_status	VARCHAR	Booking status
deposit_refund_status	VARCHAR	Deposit refund status
deposit_customer_choice	VARCHAR	Refund or wallet choice
deposit_refund_amount	DECIMAL	Refund amount
lastUpdateDate	DATETIME	Last update timestamp
customerID	INT	Reference to Customer. FK
vehicleID	INT	Reference to Vehicle. FK

5.1.3 Entity: < Car >

Attribute Name	Type	Description
vehicleID	UUID	Reference to Vehicle. PK/FK
ownerID	UUID	Reference to OwnerCar. FK
seating_capacity	INT	Number of seats
transmission	VARCHAR	Transmission type (Automatic / Manual)
model	VARCHAR	Car model
car_type	VARCHAR	Type of car (Sedan, SUV, etc.)

5.1.4 Entity: < Customer >

Attribute Name	Type	Description
customerID	UUID	Customer identifier. PK
userID	INT	Reference to User. FK
phone_number	VARCHAR	Contact number
address	VARCHAR	Customer address
customer_license	VARCHAR	Driving license
customer_license_img	VARCHAR	License image
emergency_contact	VARCHAR	Emergency contact
default_bank_name	VARCHAR	Default bank
default_account_no	VARCHAR	Default account
booking_times	INT	Total bookings

5.1.5 Entity: < car_img >

Attribute Name	Type	Description
imgID	INT	Customer identifier. PK
imageType	VARCHAR	The specific view or type of image
img_description	VARCHAR	Optional text description of the image content.
vehicleID	INT	Reference to Vehicle. FK
documentID	INT	Reference to vehicledocument . FK
customer_license_img	VARCHAR	License image

5.1.6 Entity: < Fuel >

Attribute Name	Type	Description
grantID	UUID	Grant document identifier. PK
grantNo	VARCHAR	Grant number
grantType	VARCHAR	Grant type
grant_expirydate	DATE	Grant expiry date
documentID	INT	Reference to VehicleDocument. FK

5.1.7 Entity: < International >

Attribute Name	Type	Description
customerID	UUID	Reference to Customer. PK/FK
passport_no	VARCHAR	Passport number
passport_img	VARCHAR	Passport image
countryOfOrigin	VARCHAR	Country of origin

5.1.8 Entity: < InternationalStudent >

Attribute Name	Type	Description
customerID	UUID	Reference to International. PK/FK
matric_number	VARCHAR	Reference to StudentDetails. FK

5.1.9 Entity: < International_utmstaff >

Attribute Name	Type	Description
customerID	UUID	Reference to International. PK/FK
staff_number	VARCHAR	Reference to StaffDetails. FK

5.1.10 Entity: < Invoice >

Attribute Name	Type	Description
invoiceID	UUID	Invoice identifier. PK
invoice_number	VARCHAR	Invoice number
issue_date	DATE	Issue date
totalAmount	DECIMAL	Invoice total
bookingID	INT	Reference to Booking. FK

5.1.11 Entity: < Local >

Attribute Name	Type	Description
customerID	UUID	Reference to Customer. PK/FK
ic_no	VARCHAR	Identity card number
ic_img	VARCHAR	Identity card image
stateOfOrigin	VARCHAR	State of origin

5.1.12 Entity: < LocalStudent >

Attribute Name	Type	Description
customerID	UUID	Reference to Local. PK/FK
matric_number	VARCHAR	Reference to StudentDetails. FK

5.1.13 Entity: < Local_UTMStaff >

Attribute Name	Type	Description
customerID	UUID	Reference to Local. PK/FK
staff_number	VARCHAR	Reference to StaffDetails. FK

5.1.14 Entity: < LoyaltyCard >

Attribute Name	Type	Description
loyaltyCardID (PK)	UUID	Unique identifier for the loyalty card record.
total_stamps	INT	The current count of accumulated stamps or points earned from completed bookings.
last_updated	DATE	The date when the stamp count was last modified (used to track activity or expiration).
customerID	INT	Reference to Customer. FK

5.1.15 Entity: < Motorcycle >

Attribute Name	Type	Description
vehicleID	UUID	Reference to Vehicle. PK/FK
motor_type	VARCHAR	Type of motorcycle engine

5.1.16 Entity: <OwnerCar>

Attribute Name	Type	Description
ownerID	INT	Vehicle owner identifier. PK
ic_no	VARCHAR	Owner identity card number
contact_number	VARCHAR	Owner contact number
email	VARCHAR	Owner email
bankname	VARCHAR	Bank name
bank_acc_number	VARCHAR	Bank account number
registration_date	DATE	Registration date
license_expirydate	DATE	License expiry date
isActive	BOOLEAN	Owner status
leasing_due_date	DATE	Leasing due date
leasing_price	DECIMAL	Leasing price
leasing_end_month	INT	Leasing end month
license_img	VARCHAR	License image
leasing_end_year	INT	Leasing end year
leasing_end_year	INT	Leasing end year

5.1.17 Entity: <Payment>

Attribute Name	Type	Description
paymentID	UUID	Payment identifier. PK
payment_bank_name	VARCHAR	Bank name
payment_bank_account_no	VARCHAR	Bank account
payment_date	DATETIME	Payment date
total_amount	DECIMAL	Payment amount
payment_status	VARCHAR	Payment status
transaction_reference	VARCHAR	Reference number
isPayment_complete	BOOLEAN	Completion status
payment_isVerify	BOOLEAN	Verification status
latest_Update_Date_Time	DATETIME	Last update
proof_of_payment	DATETIME	Payment proof
bookingID	INT	Reference to Booking. FK

5.1.18 Entity: < runner >

Attribute Name	Type	Description
staffID	UUID	Reference to staff. PK/FK
commission	DECIMAL	Commission amount earned by the runner

5.1.19 Entity: < Staff >

Attribute Name	Type	Description
staffID	UUID	Staff identifier. PK
ic_no	VARCHAR	Reference to PersonDetails
userID	INT	Reference to User. FK

5.1.20 Entity: < staffit >

Attribute Name	Type	Description
staffID	UUID	Reference to Staff. PK/FK
salary	DECIMAL	Staff salary

5.1.21 Entity: < user >

Attribute Name	Type	Description
userID	UUID	User identifier. PK
username	VARCHAR	Username
password	VARCHAR	Encrypted password
email	VARCHAR	Email
phone	VARCHAR	Phone number
name	VARCHAR	Full name
DOB	DATE	Date of birth
age	INT	Age
isActive	BOOLEAN	Account status
lastLogin	DATETIME	Last login
dateRegistered	DATETIME	Registration date

5.1.22 Entity: <Vehicle>

Attribute Name	Type	Description
vehicleID	UUID	Vehicle identifier. PK
plate_number	VARCHAR	Plate number
availability_status	VARCHAR	Availability
vehicle_brand	VARCHAR	Brand
vehicle_model	VARCHAR	Model
manufacturing_year	INT	Manufacturing year
color	VARCHAR	Vehicle color
engineCapacity	DECIMAL	Engine capacity
vehicleType	VARCHAR	Vehicle type
rental_price	DECIMAL	Rental price
isActive	BOOLEAN	Vehicle status
ownerID	INT	Reference to OwnerCar. FK

5.1.23 Entity: < VehicleConditionForm >

Attribute Name	Type	Description
formID	UUID	Form identifier. PK
form_type	ENUM	RECEIVE / RETURN
rental_agreement	VARCHAR	Agreement
odometer_reading	INT	Odometer
fuel_level	ENUM	Fuel level
fuel_img	VARCHAR	Fuel image
scratches_notes	TEXT	Notes
reported_dated_time	DATETIME	Report time
bookingID	INT	Reference to Booking. FK

5.1.24 Entity: < VehicleConditionImage >

Attribute Name	Type	Description
imageID	UUID	Image identifier. PK
image_path	VARCHAR	Image path
image_taken_time	DATETIME	Capture time
formID	INT	Reference to VehicleConditionForm. FK
created_at	TIMESTAMP	Created time
updated_at	TIMESTAMP	Updated time

5.1.25 Entity: <VehicleDocument>

Attribute Name	Type	Description
documentID	UUID	Document identifier. PK
upload_date	DATE	Upload date
verification_date	DATE	Verification date
fileURL	VARCHAR	File location
document_type	VARCHAR	Document type
leasing_document_url	VARCHAR	Leasing document
vehicleID	INT	Reference to Vehicle. FK

5.1.26 Entity: <VehicleMaintenance>

Attribute Name	Type	Description
maintenanceID	UUID	Maintenance identifier. PK
mileage	INT	Mileage
service_date	DATE	Service date
service_type	VARCHAR	Service type
next_due_date	DATE	Next due date
cost	DECIMAL	Cost
commission_amount	DECIMAL	Commission
service_center	VARCHAR	Service center
description	TEXT	Description
maintenance_img	VARCHAR	Image
vehicleID	INT	Reference to Vehicle. FK
staffID	INT	Reference to Staff. FK
vehicleID	INT	Reference to Vehicle. FK
staffID	INT	Reference to Staff. FK

5.1.27 Entity: <Voucher>

Attribute Name	Type	Description
voucherID	UUID	Voucher identifier. PK
loyaltyCardID	INT	Reference to LoyaltyCard. FK
discount_type	TEXT	Discount type
discount_amount	DECIMAL	Discount amount
voucher_isActive	BOOLEAN	Voucher status

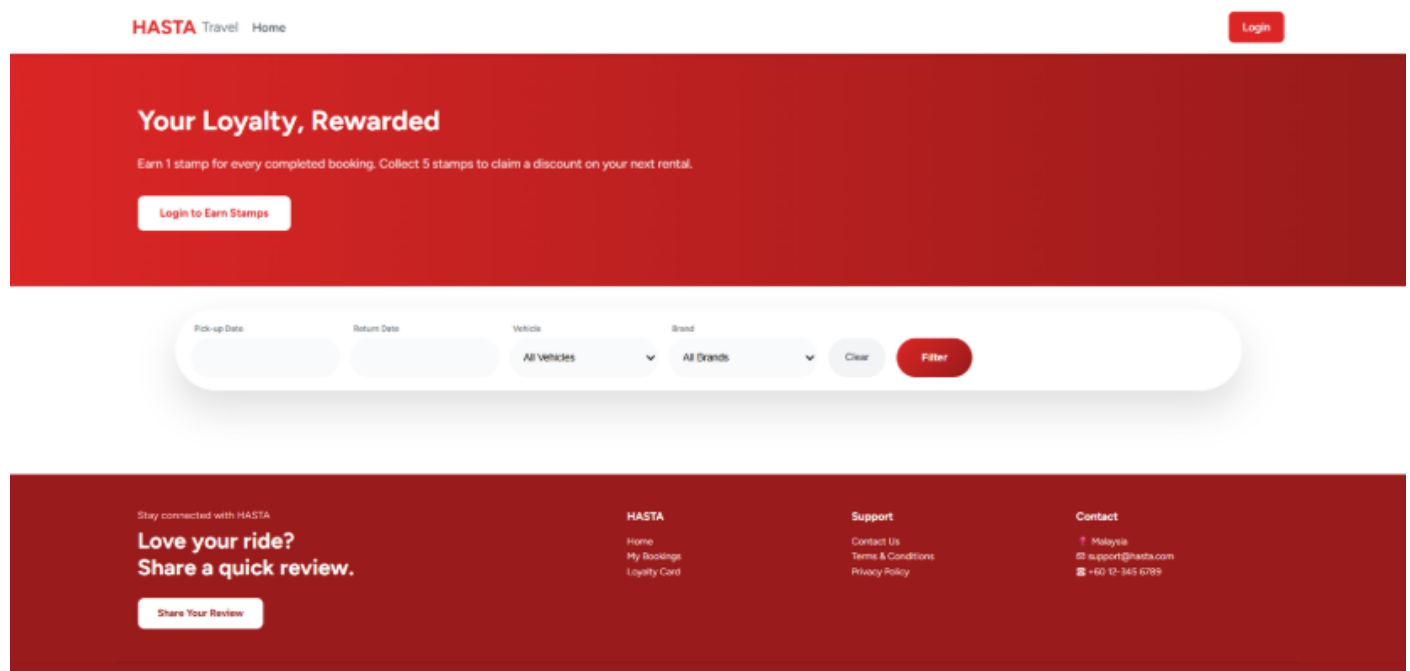
5.1.28 Entity: <WalletAccount>

Attribute Name	Type	Description
walletAccountID	UUID	Wallet identifier. PK
wallet_balance	DECIMAL	Wallet balance
outstanding_amount	DECIMAL	Outstanding amount
wallet_status	VARCHAR	Wallet status
wallet_lastUpdate_Date_Time	DATETIME	Last update
customerID	INT	Reference to Customer. FK

6. User Interface Design

The user interface is designed to be intuitive and user friendly, allowing users to easily interact with the system to perform all expected functions. Each screen provides clear navigation and guidance to ensure a smooth user experience.

6.1 Screen Images



+

Figure 6.1.1: Interface for <Browse>

Perodua Axia (1st Gen)

Hatchback



Vehicle Specifications

RM 50.00 /day

Premium Rental Rate

Pick-up Details

Pick-up Point Type

☐ Office (No surcharge)

☒ Faculty (+RM10 surcharge)

☐ College (+RM10 surcharge)

☐ Others (+RM10 surcharge)

Computing

Date

2026-01-14

Time

08:30

Return Details

Figure 6.1.2: Interface for <Book Form>

Pick-up Date

2026-01-14

Return Date

2026-01-15

Vehicle

All Vehicles

▼

Brand

All Brands

▼

Clear

Filter



Honda CR-V

Compact

●

Black

Pickup

14 Jan 2026

Return

15 Jan 2026

Available

MYR 66.40

View



Honda Vario 160 CC

Motorcycle

●

Orange

Pickup

14 Jan 2026

Return

15 Jan 2026

Available

MYR 50.00

View



Perodua Axia (2nd Gen)

Hatchback

Auto

5 seats

●

Blue

Pickup

14 Jan 2026

Return

15 Jan 2026

Available

MYR 50.00

View



Perodua Axia (1st Gen)

Hatchback

Auto

5 seats

☐White

Pickup

14 Jan 2026

Return

15 Jan 2026

Available

MYR 50.00

View



Perodua Axia (1st Gen)

Hatchback

Auto

5 seats

☐White

Pickup

14 Jan 2026

Return

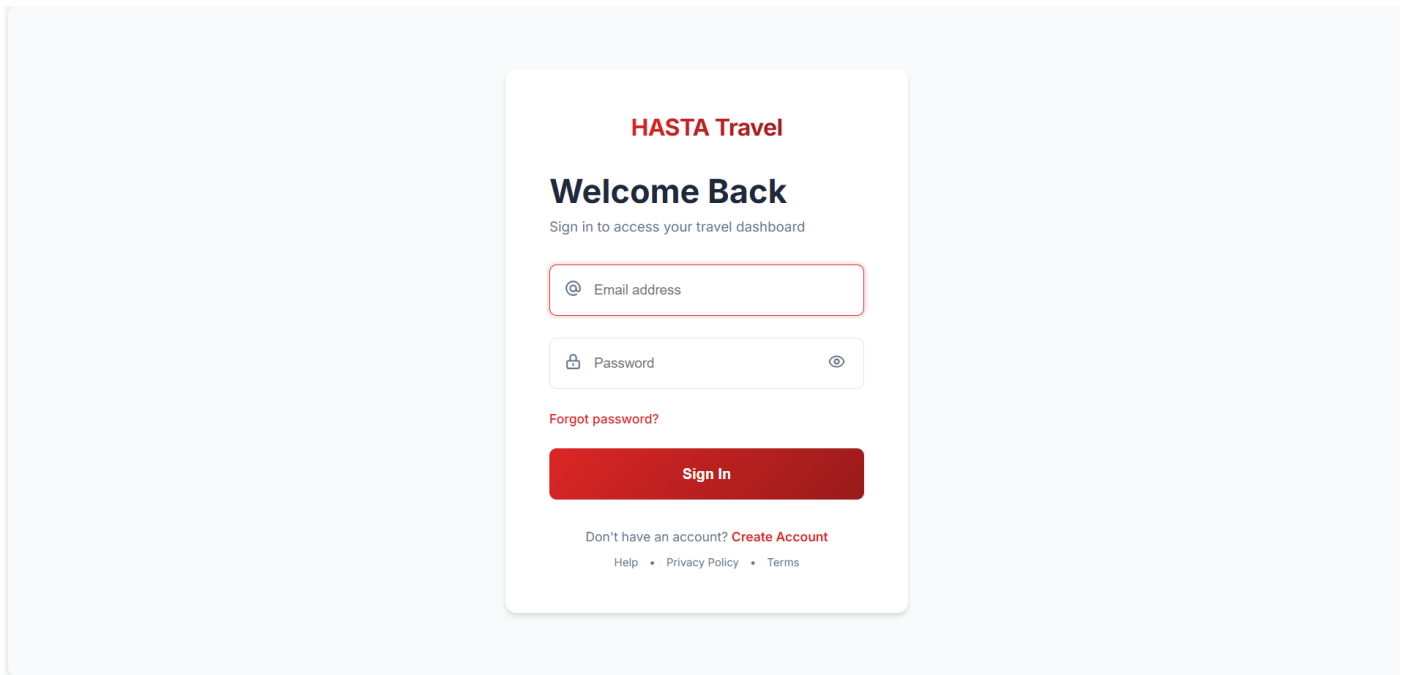
15 Jan 2026

Available

MYR 50.00

View

Figure 6.1.3: Interface for <Browse>



The image shows a login interface for HASTA Travel. It features a white card on a light blue background. The card has the HASTA Travel logo at the top, followed by the heading 'Welcome Back' and a subtext 'Sign in to access your travel dashboard'. Below this are two input fields: 'Email address' with an '@' icon and 'Password' with a lock icon and a toggle eye icon. A 'Forgot password?' link is positioned below the password field. A prominent red 'Sign In' button is centered below the inputs. At the bottom of the card, there is a link 'Don't have an account? Create Account' and a footer with links for 'Help', 'Privacy Policy', and 'Terms'.

HASTA Travel

Welcome Back

Sign in to access your travel dashboard

[Forgot password?](#)

Sign In

Don't have an account? [Create Account](#)

[Help](#) • [Privacy Policy](#) • [Terms](#)

Figure 6.1.4: Interface for <Login Account>

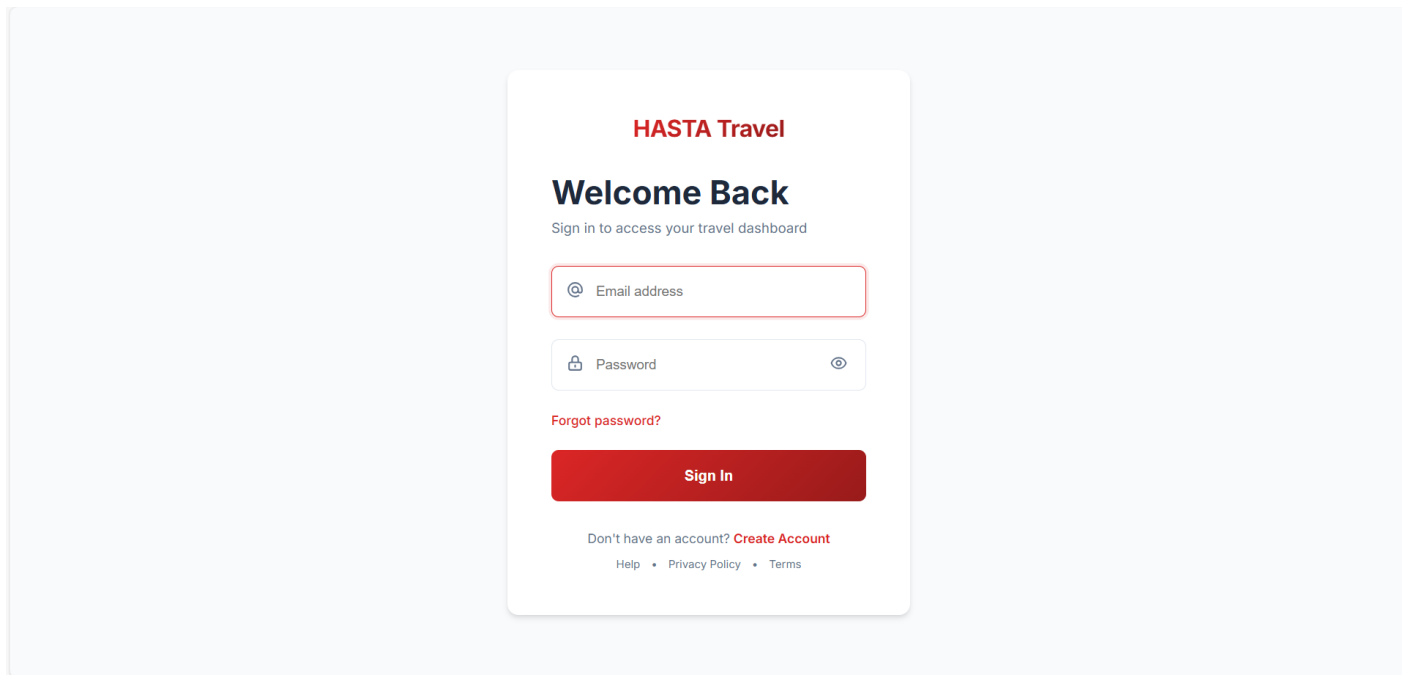


Figure 6.1.5: Interface for <Register Account>

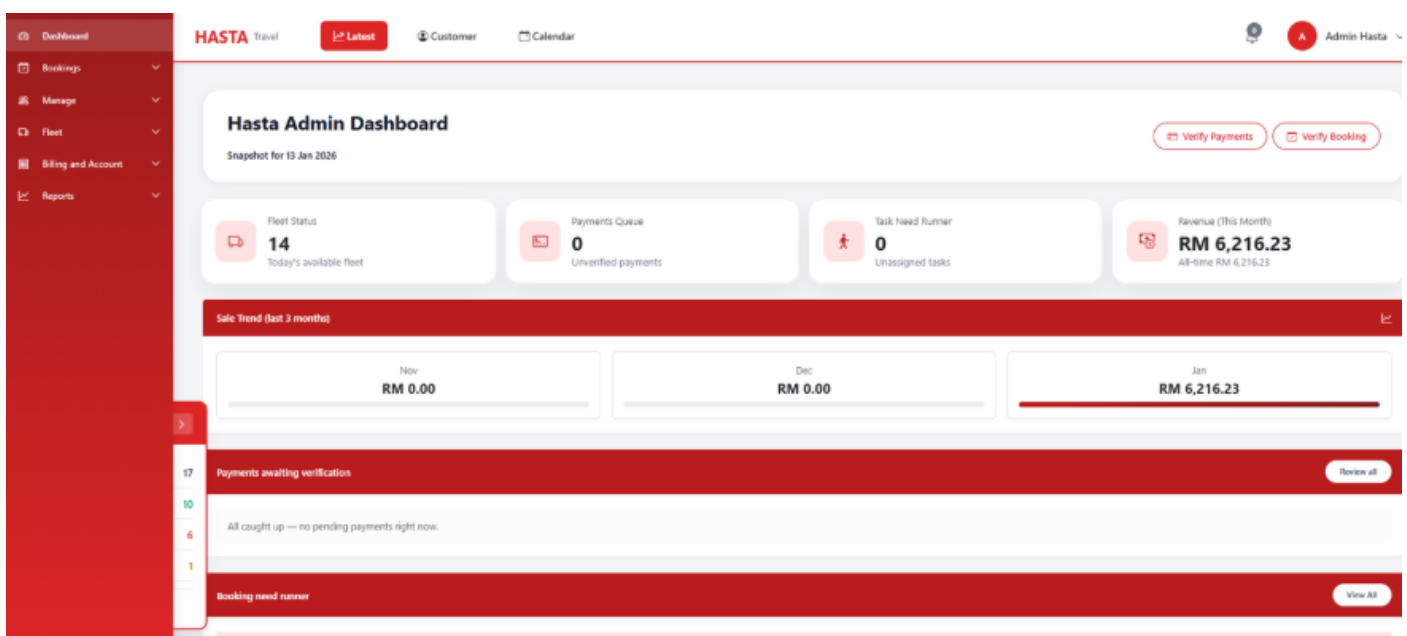


Figure 6.1.6: Interface for <Admin Dashboard>

My Bookings

NO.	VEHICLE	RENTAL PERIOD	BOOKING STATUS	PAYMENT PROGRESS	ACTIONS
2	Honda Vario 160 CC JWD9496	Jan 13, 2026 to Jan 14, 2026	Reserved	Wallet Deposit Applied 31% RM 50.00 / RM 160.00 <i>* Includes RM 50.00 from wallet</i>	Actions ▾
1	Perodua Alza 12NSK	Jan 27, 2026 to Jan 31, 2026	Completed	Fully Verified 100% RM 1,221.50 / RM 1,221.50	Actions ▾

Figure 6.1.7: Interface for <Booking History>

HASTA Travel

Latest

Customer

Calendar

Admin Hasta

Vehicle

Voucher

Reward

Vehicle Management

Manage all vehicles — 12 Jan 2026

Total Vehicles: 17

Cars: 14

Motorcycles: 2

Create

Export

Delete

SEARCH

Plate No

TYPE

IS ACTIVE

Filter

Vehicles

17 total

	VEHICLE ID	BRAND	MODEL	PLATE NO	TYPE	CREATED DATE	MANUFACTURING YEAR	ENGINE CAPACITY	RENTAL PRICE	IS ACTIVE	ACTIONS
☐	#1	Perodua	Alza	12N5K	Car	09 Dec 2025	2021	1.50L	RM 234.30	Active	View Edit Delete
☐	#2	Honda	CR-V	FNH45	Car	01 Dec 2025	2020	2.00L	RM 66.40	Active	View Edit Delete

Figure 6.1.8: Interface for <Manage Car>

HASTA TRAVEL & TOURS SDN. BHD.
Secure Payment Submission

Booking Summary (ID: #10)

Car: Vario 160 CC

Total Price: RM 160.00

Already Paid: RM 50.00 (from wallet)

Balance Due: RM 110.00

Step 1: Scan DuitNow QR to Pay



MALAYSIA NATIONAL QR

HASTA TRAVEL TOURS SDN. BHD.
Maybank: 5513 0654 1568

Step 2: Upload Proof & Details

Payment Type: Remaining Balance
Finalizing your payment.

Amount to Pay (RM)

110.00

Payment Date

01/14/2026

Upload Receipt

Choose File No file chosen

Bank Details (For Refund)

Bank Name

CIMB (Default)

Account Number

12345678

Cancel

Upload Receipt

Figure 6.1.9: Interface for <Receipt>

103

7. Traceability

The traceability matrix links each sprint, subsystem and user story ID. It ensures that all functionalities are accounted for and properly assigned throughout the development process. By mapping sprints, subsystems, and user stories, the matrix helps track progress and identify gaps or missing implementations.

Table 7.1: Traceability

Sprint #	Subsystem/Package	User Story ID
Sprint # 2 <u>1</u> : Booking Module	• Booking subsystem • Booking Cancellation subsystem	UC001: Book Vehicle
Sprint #2: Payment Module	• Payment Calculation subsystem • Payment Status & Billing subsystem • Invoice & Receipt • Abstract Wallet • Financial Analytics • Payment Interface Subsystem	UC002: Process Payment
Sprint #3: Login Module	• Customer Management	<u>UC003</u> : Register and Login
Sprint #4: Management Module	• Vehicle subsystem	UC004: Manage Vehicle
Sprint #5: Admin Module	• Staff Management subsystem • Admin and Staff Dashboard Module subsystem • Reporting & Analytics subsystem	UC005: Generate Report

8. Test Cases

Test cases are a critical component of the software testing process which providing a structured approach to verify whether the system meets its specified requirements. Each test case defines a unique scenario including the inputs, actions and expected results to ensure that individual features function correctly under both normal and edge conditions.

8.1 TC001: Test <Customer Management> Subsystem: <Register Account (US003)>

This test focuses on the customer registration workflow, where new users create an account to access the system. This test contains the following test cases:

(a) TC001_01: Test Register Account Scenario (SD001)

8.1.1 TC001_01: Test <Register Account Scenario (SD001)>

This test contains the following test cases:

- (a) TC001_01_01: Test Successful Registration
- (b) TC001_01_02: Test Registration Validation (Existing Email)

TC001_01_01: Test Successful Registration

DESCRIPTION	VERSION	TEST DATE
Test the normal account creation process for a new customer.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that a new visitor can successfully create an account by providing valid details.	AHMAD IRFAN	

Test Case ID (TC001_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to the landing page and click "Register".	URL	Registration form is displayed.	As expected	Pass		
2	Enter valid customer details into the form.	Name: John Doe Email: john@test.com Password: Pass123!	Form fields accept the input.	As expected	Pass		

3	Click the "Register" button.	Valid form data.	System creates the account, displays "Registration Successful", and redirects to Login page.	As expected	Pass		
---	------------------------------	------------------	--	-------------	------	--	--

TC001_01_02: Test Registration Validation (Existing Email)

DESCRIPTION	VERSION	TEST DATE
Test system behavior when registering with an email that already exists.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To ensure the system prevents duplicate accounts and displays an appropriate error message.	AHMAD IRFAN	

Test Case ID (TC001_01_02)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to the Registration page.	URL	Registration form is displayed.	As expected	Pass		
2	Enter details using an email already stored in the database.	Name: John Doe Email: john@test.com Password: Pass123!	Form fields filled.	As expected	Pass		
3	Click the "Register" button.	Duplicate Email.	System prevents submission and displays error: "Account with this email already exists."	As expected	Pass		

8.2 TC002: Test <Customer Management> Subsystem: <Login Account (US004)>

This test focuses on the authentication workflow, where registered users verify their identity to access the system. This test contains the following test cases:

8.2.1 TC002_01: Test <Login Scenario (SD001)>

(a) TC002_01: Test Login Scenario (SD001)

This test contains the following test cases:

(a) TC002_01_01: Test Successful Login

(b) TC002_01_02: Test Login Validation (Invalid Credentials)

TC002_01_01: Test Successful Login

DESCRIPTION	VERSION	TEST DATE
Test the normal login process for a registered customer.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that a registered user can successfully log in and access the dashboard.	AHMAD IRFAN	

Test Case ID (TC002_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to the Login page.	URL	Login form is displayed.	As expected	Pass		
2	Enter valid credentials	Email: john@test.com Password: Pass123!	Form fields accept the input.	As expected	Pass		
3	Click the "Login" button.	Valid credentials.	System authenticates the user and redirects to the Customer Dashboard.	As expected	Pass		

TC002_01_02: Test Login Validation (Invalid Credentials)

DESCRIPTION	VERSION	TEST DATE
Test system behavior when logging in with incorrect password or email.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To ensure the system denies access to unauthorized users and provides feedback.	AHMAD IRFAN	

Test Case ID (TC002_01_02)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to the Login page.	URL	Login form is displayed.	As expected	Pass		
2	Enter valid email but incorrect password.	Email: john@test.com Password: Pass123!	Form fields filled.	As expected	Pass		
3	Click the "Login" button.	Invalid Password.	System stays on Login page and displays error: "Invalid email or password."	As expected	Pass		

8.3 TC003: Test Customer Management Subsystem: View Booking History (US005)

This test focuses on the customer's ability to view their past and current reservations. This test contains the following test cases:

(a) TC003_01: Test <View History Scenario (SD001)>

8.3.1 TC003_01: Test View History Scenario (SD001)

This test contains the following test cases:

(a) TC003_01_01: Test View History List

(b) TC003_01_02: Test Empty History

TC003_01_01: Test View History List

DESCRIPTION	VERSION	TEST DATE
Test the display of booking history for a user with existing reservations.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that the system correctly retrieves and displays a list of past and active bookings for the logged-in user.	AHMAD IRFAN	

Test Case ID (TC003_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Log in as a customer with existing bookings.	User: active_user@test.com Password: Pass123!	Customer dashboard is displayed.	As expected	Pass		
2	Navigate to the "Booking History" page.	Click "My Bookings" menu option.	Customer booking history is displayed.	As expected	Pass		
3	Observe the list displayed.	Database has all past customer bookings	A table showing all customer past bookings is displayed.	As expected	Pass		
4	Verify the sorting order.	Bookings from different dates.	The list is sorted by date	As expected		Ahmad Irfan bin Azahan	

TC003_01_02: Test Empty History

DESCRIPTION	VERSION	TEST DATE
Test the display when a user has no booking history.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that the system displays an appropriate message when no records are found.	AHMAD IRFAN	

Test Case ID (TC003_01_02)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Log in as a new customer.	User: active_user@test.com Password: Pass123!	Customer dashboard is displayed.	As expected	Pass		
2	Navigate to the "Booking History" page.	Click "My Bookings" menu option.	Customer booking history is displayed.	As expected	Pass		
3	Observe the list displayed.	User has 0 bookings in database.	System displays a message: "No bookings "	As expected	Pass		

8.4 TC004: Test Booking Subsystem: <Book Vehicle(US002)>

This test contains the following test cases:

- (a) TC004_01: Test <Create New Booking Scenario (SD002)>
- (b) TC004_02: Test <Return Car (SD003)>

8.4.1 TC004_01: Test <Create New Booking Scenario (SD002)>

This test contains the following scenarios:

- (a) TC004_01_01 :Test < Successful Booking>

TC004_01_01 :Test < Successful Booking>

DESCRIPTION	VERSION	TEST DATE
Customer book a vehicle	3.0	1-Jan-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Customer booking successfully	Hafiz	

Test Case ID (TC004_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS / FAIL	Tested By	Tester Comments
1	Customer open website	N/A	The homepage should be open	As expected	Pass		
2	Customer choose desire vehicle to book	Click book now	The page of car details of chosen vehicle appears.	As expected	Pass		

3	Customer fill in details in booking form	Date of start and end rent, pick-up location and return location, and add ons.	Booking confirmation page appear	As expected	Pass		
4	Customer click proceed if the booking details are true	N/A	Payment proceed the payment section	As expected	Pass		
5	Customer complete deposit payment booking	Upload receipt payment	The button upload is functional	As expected	Pass		
6	Customer submit payment	N/A	Customer direct back to homepage. Booking successful.	As expected	Pass		
7	Customer wait deposit payment verified	N/A	Deposit verified by staff	As expected	Pass		
8	Customer can pay balance	Upload receipt payment	The button upload is functional	As expected	Pass		
9	Customer can continue to agreement	Customer can tick the agreement	Checkbox functional and	As expected	Pass		

		and download the agreement pdf	agreement downloadabl e	As expecte d			
10	Customer upload agreement	Agreement uploaded with signed	Can upload agreement file	As expecte d	Pass		
11	Customer click next	Customer fill pickup details Eg, front,back, left and right of car image.	Customer able to upload image	As expecte d	Pass		
12	Customer click submit	Customer click action and return form	Page return form is loaded	As expecte d	Pass		
13	Customer fill the return form	Customer fill return form Eg, front,back, left and right of car image. And click submit	Deposit action loaded	As expecte d	Pass		
14	Customer choose deposit action	N/A	Booking successful	As expecte d	Pass		

8.4.2 TC004_02 Test <Return Car (SD003)>

This test contains the following scenarios:

(b) TC004_02_01 :Test < Succesful Return Car (SD003)>

TC004_02_01 :Test < Successful Return Car>

DESCRIPTION	VERSION	TEST DATE
Customer successful submit return vehicle form	3.0	1-Jan-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Customer submit return vehicle form	Hafiz	

Test Case ID (TC004_02_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS / FAIL	Tested By	Tester Comments
1	Customer open website	N/A	The homepage should be open	As expected	Pass		
2	Customer choose desire vehicle to book	Click book now	The page of car details of chosen vehicle appears.	As expected	Pass		
3	Customer fill in details in booking form	Date of start and end rent, pick-up location and return location,	Booking confirmation page appear	As expected	Pass		

		and add ons.					
4	Customer click proceed if the booking details are true	N/A	Payment proceed the payment section	As expected	Pass		
5	Customer complete deposit payment booking	Upload receipt payment	The button upload is functional	As expected	Pass		
6	Customer submit payment	N/A	Customer direct back to homepage. Booking successful.	As expected	Pass		
7	Customer wait deposit payment verified	N/A	Deposit verified by staff	As expected	Pass		
8	Customer can pay balance	Upload receipt payment	The button upload is functional	As expected	Pass		
9	Customer can continue to agreement	Customer can tick the agreement and download the agreement pdf	Checkbox functional and agreement downloadable	As expected	Pass		

10	Customer upload agreement	Agreement uploaded with signed	Can upload agreement file	As expected	Pass		
11	Customer click next	Customer fill pickup details Eg, front,back, left and right of car image.	Customer able to upload image	As expected	Pass		
12	Customer click submit	Customer click action and return form	Page return form is loaded	As expected	Pass		
13	Customer fill the return form	Customer fill return form Eg, front,back, left and right of car image. And click submit	Deposit action loaded. Successful submit return form.	As expected	Pass		

8.5 TC005: Test <Vehicle> Subsystem: <Manage Vehicle (US001)>

This test contains the following test cases:

- (a) TC005_01: Test <Add Vehicle (SD004)>

8.5.1 TC005_01: Test <Add Vehicle (SD004)>

This test contains the following scenarios:

- (a) TC005_01_01: Test <Add new vehicle>
- (b) TC005_01_02: Test <Cancel add>

TC005_01_01: Test <Add new vehicle>

DESCRIPTION	VERSION	TEST DATE
Admin to add vehicle	3.0	1-Jan-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To Add vehicle in system	Hafiz	

Test Case ID (TC005_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS / FAIL	Tested By	Tester Comments
1	Staff logs into system	Email and password	The interface page for Staff should be open	As expected	Pass		
2	Staff navigates to Car Management section	N/A	Landing in Car Management section	As expected	Pass		

3	Staff can select one following options: Add, Edit, Delete Car	Click Add vehicle	Landing in add vehicle page.	As expected	Pass		
4	Enter Car details	Brand, model, plate number, rental price, manufacturing year, car type	Car details can be fill	As expected	Pass		
5.	Click submit	N/A	Going back to manage car section.	As expected	Pass		

TC005_01_02: Test <Cancel add>

DESCRIPTION	VERSION	TEST DATE
Admin cancel add vehicle	3.0	1-Jan-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Cancel add vehicle	Hafiz	

Test Case ID (TC005_01_02)	Test Steps	Input Data	Expected Results	Actual Results	PASS / FAIL	Tested By	Tester Comments
1	Staff logs into system	Email and password	The interface page for Admin should be open	As expected	Pass		
2	Staff navigates to Car Management section	N/A	Landing in Car Management section	As expected	Pass		
3	Staff can select one following options: Add, Edit, Delete Car	Click Add vehicle	Landing in add vehicle page.	As expected	Pass		
4	Enter Car details	Brand,model, manufacturing year,	Car details can be fill	As expected	Pass		

5.	Click cancel	N/A	Going back to manage car section.	As expected	Pass		
----	--------------	-----	-----------------------------------	-------------	------	--	--

8.6 TC006: Test <Booking> Subsystem: <Cancel Order (US006)>

This test contains the following test cases:

1. TC006_01: Test < Cancel Order scenario (SD005)>
- 2.

8.6.1 TC006_01: Test <Cancel Order Scenario (SD005)>

This test contains the following scenarios:

- a. TC006_01_01: Test normal scenario – Cancel upcoming booking (SD005)

DESCRIPTION	VERSION	TEST DATE
Test the Booking Cancellation Functionality	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Verify that the booking cancellation process works correctly for allowed, denied, and system failure scenarios	Ahmad Irfan bin Azahan	

Test Case ID (TC006_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to Booking History Page	Customer logged in	Booking History page should open	As Expected	Pass		
2	Select a booking and click	bookingId = 12345,	Action drop down menu show	As Expected	Pass		

	action button	customer id = 1001					
3	Click view detail	-	Detail booking information is displayed	As expected	Pass		
4	System retrieves booking details	booking id = 12345	BookingService calls BookingRepositoryDA.findByld() and booking details are returned	As Expected	Pass		
5	Click Cancel Booking	-	System ask for confirmation	As Expected	Pass		
6	Confirm Cancellation	Click "Confirm"	Booking status updated to "Pending Cancellation Approval"	As Expected	Pass		
6	Notify staff about cancellation	booking id = 12345	NotificationService sends notification to staff	As Expected	Pass		
7	Show pending approval message to customer		Customer sees "Cancellation Pending Approval" message	As Expected	Pass		

8.7 TC007 Test <Generate Report Scenario> Subsystem: <Generate Report (US007)>

This test contains the following test cases:

- a. TC007_01: Test < Sequence Diagram for <Generate Report (SD006) >

8.7.1 TC006_01: Test <Generate Report (SD006)>

This test contains the following scenarios:

- a. TC007_01_01: Test normal report generation flow (Sequence Diagram SD006)
- b. TC007_01_02: Test report export functionality (Sequence Diagram SD006)
- c. TC007_01_03: Test alternative flow – view report without export (SD006)
- d. TC007_01_04: Test exception flow – no data found (SD006)

TC007_01_01: Test normal report generation flow (Sequence Diagram SD008)

DESCRIPTION	VERSION	TEST DATE
Generate report based on booking record	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Ensure reports can be generated and exported correctly	Cheong Yi Shien	

Test Case ID (TC007_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to Admin Dashboard	Admin-dashboard	Dashboard page should be displayed	As Expected	Pass		
2	Click on Reports menu	-	Reports submenu expands	As Expected	Pass		
3	Choose Reports Type	Report Type = "Rentals"	Rentals window is displayed	As Expected	Pass		

			with filter option				
4	Apply filter	Filters = Date Range: Monthly, Booking Status: Verified	Filters are selected	As Expected	Pass		
5	Click "Apply filter"	Filters = Date Range: Monthly, Booking Status: Verified	Report is generated and displayed if data exists	As Expected	Pass		
6	System display report	-	Report display correctly	As expected	Pass		

TC007_01_02: Test report export functionality (Sequence Diagram SD006)

DESCRIPTION	VERSION	TEST DATE
Generate report based on booking record	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Ensure reports can be generated and exported correctly	Cheong Yi Shien	

Test Case ID (TC007_01_02)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to Admin Dashboard	Admin-dashboard	Dashboard page should be displayed	As Expected	Pass		
2	Click on Reports menu	-	Reports submenu expands	As Expected	Pass		
3	Choose Reports Type	Report Type = "Rentals"	Rentals window is displayed with filter option	As Expected	Pass		
4	Apply filter	Filters = Date Range: Monthly, Booking Status: Verified	Filters are selected	As Expected	Pass		
5	Click "Apply filter"	Filters = Date Range: Monthly,	Report is generated and displayed	As Expected	Pass		

		Booking Status: Verified	if data exists				
6	System display report	-	Report display correctly	As expected	Pass		
7	Click "Export PDF" or "Export Excel"	-	Button is clickable	As expected	Pass		
8	System download exported report	Report template	File is able to download in correct format	As Expected	Pass		

TC007_01_03: Test alternative flow – view report without export (SD006)

DESCRIPTION	VERSION	TEST DATE
Generate report based on booking record	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Ensure reports can be generated and exported correctly	Cheong Yi Shien	

Test Case ID (TC007_01_03)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to Admin Dashboard	Admin-dashboard	Dashboard page should be displayed	As Expected	PASS		
2	Click on Reports menu	-	Reports submenu expands	As Expected	Pass		
3	Choose Reports Type	Report Type = "Rentals"	Rentals window is displayed with filter option	As Expected	Pass		
4	Apply filter	Filters = Date Range: Monthly, Booking Status: Verified	Filters are selected	As Expected	Pass		
5	Click "Apply filter"	Filters = Date Range: Monthly,	Report is generated and displayed	As Expected	Pass		

		Booking Status: Verified	if data exists				
6	System display report	-	Report display correctly	As expected	Pass		

TC007_01_04: Test exception flow – no data found (SD006)

DESCRIPTION	VERSION	TEST DATE
Generate report based on booking record	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
Ensure reports can be generated and exported correctly	Cheong Yi Shien	

Test Case ID (TC007_01_04)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Navigate to Admin Dashboard	Admin-dashboard	Dashboard page should be displayed	As Expected	Pass		
2	Click on Reports menu	-	Reports submenu expands	As Expected	Pass		
3	Choose Reports Type	Report Type = "Rentals"	Rentals window is displayed with filter option	As Expected	Pass		

4	Apply filter	Filters = Date Range: Monthly, Booking Status: Verified	Filters are selected	As Expecte d	Pass		
5	Click “Apply filter”	Filters = Date Range: Monthly, Booking Status: Verified	Report is generated and displayed if data exists	As Expecte d	Pass		
6	Generate report with no data	Filters = Date Range: 12-21- 2025	Show message: “No bookings found with the selected filters”	As Expecte d	Pass		

8.8 TC008: Test <Invoice & Receipt> Subsystem: Make Payment (US009)

This test focuses on the customer payment workflow, where users upload proof of payment for their bookings. The behavior is modelled in Sequence Diagram SD009.

This test contains the following test cases:

(a) TC008_01: Test Payment Scenario (SD007)

8.8.1 TC008_01: Test Payment Scenario (SD007)

This test contains the following scenarios:

- (a) TC008_01_01: Test Normal Payment Submission
- (b) TC008_01_02: Test Deposit Calculation
- (c) TC008_01_03: Test Invalid Receipt Format

TC008_01_01: Test Normal Payment Submission

DESCRIPTION	VERSION	TEST DATE
Test the normal payment receipt upload for a confirmed car rental booking.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that a customer can upload a valid payment receipt and submit payment successfully.	LING YU QIAN	

Test Case ID (TC008_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PAS S/FA IL	Tested By	Tester Comments
1	The customer navigates to the landing page. The customer logs in.	Valid email and password.	Customer dashboard is displayed.	As expected	Pass		
2	The customer navigates to the "My Bookings" page. The customer clicks the menu.	Click on the 'My Bookings' menu.	List of customer bookings is displayed.	As expected	Pass		
3	Select booking with status 'Pay now' Payment'.	Click on the relevant booking row.	Booking details page is displayed.	As expected	Pass		
4	Click 'Upload Receipt' and select receipt file.	Valid JPG/PNG receipt image.	Receipt file is attached or previewed successfully.	As expected	Pass		

5	Click 'Submit Payment'.	Attached valid receipt.	System saves receipt and updates booking status to 'Awaiting Payment Verification' with a success message.	As expected	Pass		
---	-------------------------	-------------------------	--	-------------	------	--	--

TC008_01_02: Test Deposit Calculation

DESCRIPTION	VERSION	TEST DATE
Test the deposit calculation logic based on rental duration.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To confirm that the system calculates the correct deposit for short-term and long-term rentals.	LING YU QIAN	

Test Case ID (TC008_01_02)	Test Steps	Input Data	Expected Results	Actual Results	PAS S/FA IL	Tested By	Tester Comments
1	Customer create a booking with rental duration of 5 days.	Start and end dates that result in 5 rental days.	System calculates deposit amount as RM50 and displays it in the	As expected	Pass		

			booking summary.				
2	Create a booking with rental duration of 20 days.	Start and end dates that result in 20 rental days.	System calculates deposit amount as full rental price and displays it in the booking summary.	As expected	Pass		
3	Review booking summaries for both bookings.	Open each booking summary page.	Correct deposit values are displayed for both bookings.	As expected	Pass		

TC008_01_03: Test Invalid Receipt Format

DESCRIPTION	VERSION	TEST DATE
Test system behaviour when a customer uploads an invalid receipt file format.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To ensure the system rejects non-image files and displays an appropriate error message.	LING YU QIAN	

Test Case ID (TC008_01_03)	Test Steps	Input Data	Expected Results	Actual Results	PAS S/FA IL	Tested By	Tester Comments
1	Customer log in and open booking with status 'Awaiting Payment'.	Valid customer credentials.	Booking details page is displayed.	As expected	Pass		
2	Click 'Upload Receipt' and select a excel file.	Receipt file in excel format.	File is selected in the file chooser.	As expected	Pass		

3	Click 'Submit Payment'.	Selected excel file.	System displays error message: 'Invalid file format. Please upload an image (JPG/PNG).'	As expected	Pass		
4	Check booking status after error.	Refresh or reopen the booking.	Booking status remains in 'Pending' and no receipt is saved.	As expected	Pass		

8.9 TC009: Test <Billing Module> Subsystem: Generate Invoice (US010)

This test verifies the generation of invoices/receipts for confirmed and fully paid bookings. The behaviour is modelled in Sequence Diagram SD012.

This test contains the following test case:

(a) TC009_01: Test <Generate Invoice Scenario (SD008)

8.9.1 TC009_01: Test <Generate Invoice Scenario (SD008)

This test contains the following scenario:

(a) TC009_01_01: Test Generate Invoice After Successful Payment

TC009_01_01: Test Generate Invoice After Successful Payment

DESCRIPTION	VERSION	TEST DATE
Test invoice generation for a confirmed and fully paid booking.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that the system generates a PDF invoice that can be viewed and downloaded by the customer.	LING YU QIAN	

Test Case ID (TC009_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PAS S/F A IL	Tested By	Tester Comments
1	Log in as customer.	Valid email and password.	Customer dashboard is displayed.	As expected	Pas s		

2	Navigate to Booking History.	Click 'Booking History' menu option.	List of current and past bookings is displayed.	As expected	Pass		
3	Select booking with status 'Pending'.	Click relevant booking row.	Booking details page is displayed.	As expected	Pass		
4	Click 'View Receipt / Invoice'.	Invoice action button.	System generates and displays invoice in PDF format.	As expected	Pass		
5	Download or print the invoice from the viewer.	Use PDF download/print option.	Invoice downloads/prints successfully with correct booking, customer, and payment details.	As expected	Pass		

8.10 TC010: Test <Payment Status & Billing> Subsystem: <Verify Payment (US008)>

This test focuses on the staff workflow for verifying customer payments. The behaviour is modelled in Sequence Diagram SD009.

This test contains the following test cases:

- (a) TC010_01: Test <Approve Payment Scenario (SD009)>
- (b) TC010_02: Test <Reject Payment Scenario (SD009)>

8.10.1 TC010_01: Test Approve Payment Scenario (SD009)

This test contains the following scenario:

- (a) TC010_01_01: Test Approve Valid Payment

TC010_1_01: Test Approve Valid Payment

DESCRIPTION	VERSION	TEST DATE
Test the staff approval process for a valid payment receipt.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that staff can approve a valid receipt and confirm the booking.	LING YU QIAN	

Test Case ID (TC010_01_01)	Test Steps	Input Data	Expected Results	Actual Results	PAS S/F A IL	Tested By	Tester Comments
1	Log in as staff user.	Valid staff ID and password.	Staff dashboard is displayed.	As expected	Pass		

2	Open Payment Verification page.	Select 'Payment Verification' from the menu.	List of bookings with status 'Awaiting Payment Verification' is displayed.	As expected	Pass		
3	Select a booking and open the receipt image.	Click the booking row and view receipt.	Receipt image loads clearly for review.	As expected	Pass		
4	Verify receipt details against QR transaction .	QR reference and paid amount.	Receipt information matches bank transaction records.	As expected	Pass		
5	Click 'Approve Payment'.	Approval action for selected booking.	Booking status is updated to 'Confirmed/Paid' and customer notification is sent.	As expected	Pass		

8.10.2 TC010_02: Test Reject Payment Scenario (SD009)

This test contains the following scenario:

(a) TC010_02_01: Test Reject Payment with Reason

TC010_02_01: Test Reject Payment with Reason

DESCRIPTION	VERSION	TEST DATE
Test the staff rejection process for an invalid or suspicious payment receipt.	1.0	29-Dec-2025
TEST OBJECTIVE	AUTHOR	REVIEWER
To verify that staff can reject a receipt with a reason and notify the customer to resubmit.	LING YU QIAN	

Test Case ID (TC010_02_01)	Test Steps	Input Data	Expected Results	Actual Results	PASS/FAIL	Tested By	Tester Comments
1	Log in as staff and open Payment Verification page.	Valid staff account.	List of payments awaiting verification is displayed.	As expected			
2	Select booking with suspicious receipt.	Booking whose receipt is unclear or incorrect.	Receipt image is displayed for inspection.	As expected			

3	Click 'Reject Payment'.	Initiate reject action.	System prompts for a rejection reason.	As expected			
4	Enter reason (e.g., 'Wrong amount') and confirm rejection.	Text reason typed by staff.	System saves reason and updates booking status to 'Payment Rejected'.	As expected			
5	Check customer notification .	Log in as the customer .	Customer receives notification to re-upload a valid payment receipt.	As expected			

9. Traceability Matrix

A traceability matrix is a key tool in software testing and quality assurance that establishes a clear link between requirements and their corresponding test cases. It ensures that every functional and non-functional requirement is adequately tested thus helping to identify any missing or redundant test cases. By mapping requirements to test scenarios, the traceability matrix facilitates comprehensive coverage, simplifies impact analysis for changes, and provides a clear record for stakeholders to verify that all project objectives are met.

Table 9.1: Example of Traceability Matrix for < Hasta Travel Car Rental System >

Sprint No	Package Item	User Story ID	User Story Description	Sequence Diagram ID	Sequence Diagram Description	Test Case ID
Sprint 1	Vehicle Subsystem	US001	Add, read and update vehicle records	SD004	Illustrates the steps for admin to add new vehicle	TC005_01_01
						TC005_01_02
Sprint 2	Booking Subsystem	US002	Customer to book vehicle	SD002	Shows the process of booking	TC004_01_01
	Booking Subsystem	US002	Customer to book vehicle	SD003	Shows process of return car	TC004_02_01
Sprint 3	Customer Management Subsystem	US003	Register Account	SD001	Shows the process of validating user input and creating a new customer record.	TC001_01_01
						TC001_01_02

		US004	Login Account	SD001	Shows the authentication process and session creation for valid credentials.	TC002_01_01
						TC002_01_02
		US005	View Booking History	SD001	Shows the retrieval of booking records associated with the logged-in user.	TC003_01_01
						TC003_01_02
	Booking Cancellation	US006	Cancel Booking	SD005	Shows the status update process and deposit refund/forfeit logic based on time.	TC006_01_01
						TC006_01_02
Sprint 4	Payment Status & Billing	US008	Verify Payment	SD009	Shows the process for verifying customer payments by staff.	TC0010_01_01
						TC0010_02_01
	Invoice & Receipt Subsystem	US009	Make Payment	SD007	Shows the process of submitting payment and uploading receipt for a booking.	TC008_01_01
						TC008_01_02
						TC008_01_03
	Invoice & Receipt Subsystem	US010	Generate invoice	SD008	Shows the process for	TC009_01_01

					generating an invoice/receipt after successful payment.	
Sprint 5	Reporting & Analytics subsystem	US007	Generate Report	SD006	Ensure reports can be generated and exported correctly	TC007_01_01
						TC007_01_02
						TC007_01_03
						TC007_01_04